Università degli Studi di Napoli
Federico II

A.A 2025--2026

Elementi di Informatica Teorica

Indice

0 Introduzione 4

1 Calcolabilità e Linguaggio S 5

1.1 Concetto di calcolabilità 5

1.2 Linguaggio S 6

1.2.1 Sintassi 6

1.2.2 Esempi di S-Programmi e Macro 6

1.3 Caratterizzazione di un S-programma 8

1.4 Esercizi svolti e approfondimenti 11

2 Operazioni sulle funzioni e Paradigma Funzionale 15

2.1 Paradigma Funzionale 15

2.2 Operazioni fondamentali e Classi PRC 15

2.2.1 Composizione 15

2.2.2 Ricorsione Primitiva 16

2.2.3 Classi PRC e Funzioni Ricorsive Primitive 17

2.3 Proprietà di chiusura delle Classi PRC 19

2.3.1 Chiusura per congiunzione, disgiunzione e negazione 19

2.3.2 Definizione per Casi 19

2.3.3 Operatori di Sommatoria e Produttoria e Quantificatori Limitati 20

2.3.4 Minimalizzazione limitata 21

2.4 Funzioni parzialmente calcolabili 23

2.4.1 Minimalizzazione illimitata 23

2.4.2 Caratterizzazione delle funzioni parzialmente calcolabili 23

2.5 Esercizi e approfondimenti 23

3 Problema della Fermata e Funzione Universale 26

3.1 Concetti Introduttivi Generali 26

3.2 Funzione di Pairing e Numeri di Gödel 27

3.2.1 Funzione di Pairing 27

3.2.2 Numeri di Gödel 28

3.3 Codifica d'Istruzioni, Programmi e Stati 29

3.3.1 Codifica d'Istruzioni 29

3.3.2 Codifica di Programmi 30

3.3.3 Codifica degli Stati 32

3.4 Funzioni parziali non calcolabili e Halt Problem 32

3.5 Funzione universale e Predicato STP 34

3.5.1 Funzione universale 34

3.5.2 Predicato STP 37

3.6 Diagramma Riassuntivo sulle Funzioni Parziali 38

3.7 Come utilizzare le funzioni HALT, STP e Universale 39

3.8 Esercizi dalle prove d'esame su STP, HALT e Funzione Universale 40

3.8.1 Esercizio 1 40

3.8.2 Esercizio 2 40

3.9 Esercizi e approfondimenti 40

4 Insiemi Ricorsivi e Ricorsivamente Enumerabili 43

4.1 Da funzioni a insiemi 43

4.1.1 Analisi degli Insiemi Ricorsivi 43

4.1.2 Insiemi Ricorsivamente Enumerabili 45
4.2 Esercizi ricorsivamente enumerabili 49
4.2.1 Esercizio 1 49
4.2.2 Esercizio 2 50
4.3 Esercizi ed approfondimenti 51

5 Macchine di Turing e Automi a Stati Finiti 53
5.1 Macchine di Turing 53
5.2 Automi a Stati Finiti 55
5.2.1 Automi Finiti Deterministici 55
5.2.2 Linguaggi Regolari 56
5.2.3 Automi Finiti Non Deterministici 58
5.2.4 Equivalenza tra DFA e NFA 59
5.3 Esercizi ed approfondimenti 59

6 Operazioni sui Linguaggi regolari 63
6.1 DFA Non-Restarting 63
6.2 Chiusura dei Linguaggi Regolari 63
6.2.1 Unione 64
6.2.2 Prodotto per Concatenazione 65
6.2.3 Iterazione 67
6.3 Espressioni Regolari 70
6.3.1 Semantica delle Espressioni Regolari 70
6.4 Pumping Lemma per Linguaggi Regolari 71
6.5 Esercizi ed approfondimenti 73

7 Grammatiche context-free e Automi Pushdown 76
7.1 Grammatiche context-free 76
7.1.1 Albero di derivazione e ambiguità 77
7.2 Proprietà di chiusura dei linguaggi context-free 79
7.2.1 Grammatiche regolari 80
7.3 Automi Pushdown 81

8 Forme Normali di Chomsky e Pumping Lemma 85
8.1 Forme normali di Chomsky 85
8.2 Pumping Lemma per i linguaggi context-free 87
8.3 Gerarchia di Chomsky-Schützenberger 88

9 Esercizi della 1° prova intercorso 89
9.1 2023 89
9.1.1 20 Gennaio 2023 89
9.1.2 03 Marzo 2023 90
9.1.3 13 Giugno 2023 90
9.2 2022 91
9.2.1 16 Marzo 2022 91
9.2.2 07 Aprile 2022 91
9.2.3 20 Luglio 2022 92
9.2.4 13 Settembre 2022 94
9.3 2021 95
9.3.1 22 Giugno 2021 95
9.3.2 20 Luglio 2021 95
9.3.3 19 Gennaio 2021 96
9.4 2020 96
9.4.1 20 Gennaio 2020 96
9.4.2 18 Febbraio 2020 97
9.4.3 23 Giugno 2020 98
9.4.4 16 Ottobre 2020 98
9.5 2019 98
9.5.1 20 Giugno 2019 98
9.5.2 17 Luglio 2019 99
9.6 2018 99
9.6.1 2 Luglio 2018 100

Frammenti di Codice

Codice 1.1 Esempio di S-Programma . . . 6
Codice 1.2 Funzione identica . . . 6
Codice 1.3 Macro di salto incondizionato . . . 7
Codice 1.4 Macro di azzeramento . . . 7
Codice 1.5 Macro di assegnazione . . . 7
Codice 1.6 Macro di salto generalizzato . . . 7
Codice 1.7 Predicato logico (uguaglianza) . . . 8
Codice 1.8 Funzione parzialmente calcolabile (sottrazione) . . . 8
Codice 1.9 Macro . . . 10
Codice 1.10 Funzione identica . . . 11
Codice 1.11 Funzione somma . . . 11
Codice 1.12 Funzione prodotto . . . 12
Codice 1.13 Funzione prodotto con costante . . . 12
Codice 1.14 Predicato di Parità . . . 12
Codice 1.15 Predicato di Uguaglianza senza Macro . . . 12
Codice 1.16 Funzione potenza . . . 13
Codice 1.17 Programma P . . . 13
Codice 1.18 Predicato Minore Uguale . . . 14
Codice 1.19 Predicato Minore Uguale . . . 14
Codice 2.1 Dimostrazione Teorema di Composizione . . . 16
Codice 2.2 Dimostrazione Teorema di Ricorsione Primitiva . . . 17
Codice 2.3 Minimalizzazione illimitata . . . 23
Codice 3.1 Dimotrazione Problema della Fermata . . . 34
Codice 3.2 Dimostrazione Programma Universale . . . 37
Codice 3.3 Funzione somma . . . 39
Codice 3.4 Funzione somma . . . 39
Codice 3.5 Funzione somma . . . 39
Codice 3.6 Funzione h . . . 40
Codice 3.7 Esercizio Codifica Programmi 1 . . . 40
Codice 3.8 Esercizio Codifica Programmi 1 . . . 41
Codice 3.9 Esercizio Codifica Programmi 3 . . . 41
Codice 3.10 Programma H1 . . . 42
Codice 4.4 S-Programma che calcola g(x) . . . 50
Codice 4.5 S-Programma con tecnica Dovetail . . . 51

INTRODUZIONE

ESTRATTO

Il seguente documento rappresenta una raccolta di appunti, riorganizzata e migliorata nelle vesti grafiche, del corso di Elementi di Informatica Teorica del professor A. De Luca, per il corso di laurea in Informatica A.A 2025--2026, della Federico II di Napoli.

Il corso si articola in due macro sezioni principali. La prima, riguardante il concetto di algoritmo e di calcolabilità, tratterà dell'analisi formale di algoritmi visti come funzioni matematiche, mentre la seconda, riguardante modelli di calcolo semplificati e linguaggi formali, tratterà di macchine sequenziali (automi) e la loro corrispondenza con le grammatiche e i linguaggi regolari

Si tiene a specificare che questo documento non ha lo scopo di **sostituire libri di testo**, note del professore o lezioni frontali, nonostante siano tutte fonti utilizzate per la stesura, ma quello di integrare al materiale più complesso una rilettura degli argomenti da un punto di vista meno tecnico e quindi più lento nel soffermarsi nei passaggi critici.

Si ringraziano **Valentino Bocchetti** e **Pasquale Miranda** ideatori e gestori del progetto *Unina Docs*,

Si ringraziano inoltre tutti i revisori che hanno contribuito al documento con consigli e proposte di modifica, invitando a dare feedback in caso di errori.

Si ringraziano in fine i lettori nella speranza che questo lavoro sia stato utile.

Redazione a cura di **Riccardo Elena**, **Davide Gargiulo**

Revisione testo a cura di Giuseppe Di Martino, Gianluca Fiorentino, Francesco Donnarumma, Mattia Perna

Si ringraziano Gennaro De Gregorio, Benedetta Carandente Giarrusso, Vincenzo di Carluccio, Mattia Perna per il materiale fornito

Calcolabilità e Linguaggio S

1.1 Concetto di calcolabilità

Un problema è detto **calcolabile** se è possibile risolverlo algoritmicamente. Questa misura può essere effettuata in maniera rigorosa formalizzando il problema in esame usando una funzione matematica. L'insieme degli input possibili per tale problema ne costituirà il dominio, mentre quello degli output il codominio. Nel nostro caso ci limiteremo a prendere in considerazione i problemi, e dunque le funzioni, che abbiano come input e output numeri naturali.

Importante distinzione che va effettuata dunque è tra **funzioni totali** e **funzioni parziali**:

- Una funzione associata a un problema è detta **totale** se ha un valore di output definito $\forall a_k \in \mathbb{N}^k$, ovvero è definibile come:

$$\begin{aligned} f : \mathbb{N}^k &\to \mathbb{N},\ k \in \mathbb{N} \\ (x_1, x_2, \cdots, x_k) &\mapsto y \end{aligned} \tag{1.1}$$

- Una funzione associata a un problema è detta **parziale** se ha un valore di output definito $\forall a_k \in D \subseteq \mathbb{N}^k$, ovvero è definibile come:

$$\begin{aligned} f : D \subseteq \mathbb{N}^k &\to \mathbb{N},\ k \in \mathbb{N} \\ (x_1, x_2, \cdots, x_k) &\mapsto y \end{aligned} \tag{1.2}$$

Le **funzioni totali** rappresentano dunque un *sottoinsieme*[1] di quelle **parziali**. Il ricorso a quest'ultime è fondamentale poiché, al prezzo della perdita di *generalità* del problema, possiamo alle volte garantirne la **calcolabilità** altrimenti dimostrabile come falsa.

Per definire dunque il concetto di calcolabilità di una funzione, è necessario enunciare la **tesi di Church-Turing**[2].

Tesi di Church-Turing (Versione S)

Qualsiasi funzione che si possa determinare tramite qualche procedura algoritmica è parzialmente calcolabile.

Questo enunciato è appunto una **tesi** poiché non è dimostrabile, essendo definito in modo non preciso, ma è comunque accettato come vero poiché rappresenta l'idea intuitiva che ogni funzione valutabile in una sequenza finita di passi sia esprimibile tramite un programma. Diremo dunque che:

- Una **funzione** $f : D \subseteq \mathbb{N}^k \to \mathbb{N},\ k \in \mathbb{N}$ è ***parzialmente* calcolabile** se esiste un algoritmo che la rappresenta.
- Una **funzione** è **calcolabile** se essa è **totale *e* parzialmente calcolabile**

Definiamo infine come **predicato** una particolare funzione totale il cui codominio è l'insieme $\{0, 1\} \in \mathbb{N}$. Codificando il valore di 0 come `FALSO` e di 1 come `VERO`, è possibile valutare la calcolabilità di predicati logici a valori booleani.

$$\begin{aligned} p : \mathbb{N}^k &\to \{0, 1\} \subset \mathbb{N},\ k \in \mathbb{N} \\ (x_1, x_2, \cdots, x_k) &\mapsto y \in \{0, 1\} \end{aligned} \tag{1.3}$$

[1] Essendo il dominio di una funzione parziale $D \subseteq \mathbb{N}^k$, ne fanno parte anche i casi particolari di $D = \mathbb{N}^k$, ovvero le funzioni totali.

[2] La versione enunciata è quella relativa al **Modello S**, che utilizzeremo per la valutazione della calcolabilità di una funzione. Ogni modello di calcolo però è dimostrabile come equivalente.

1.2 Linguaggio S

Il **linguaggio S** è un linguaggio formale per la scrittura di algoritmi utilizzabile per la verifica della **calcolabilità** di un problema.

Questo linguaggio infatti è *minimale* nelle istruzioni, possedendo solo quelle eseguibili da qualsiasi compilatore e con le quali è possibile, per quanto in molte più linee, ricostruire un qualsiasi algoritmo.

1.2.1 Sintassi

Variabili. Nel **linguaggio S** le variabili si dividono in tre gruppi:

1. Variabili d'*Input* (`X1,X2, ...`), di numero potenzialmente infinito[3] inizializzate con i valori dell'input di una data istanza;
2. Variabile di *Output* (`Y`), unica e inizializzata al valore `0`;
3. Variabili *Temporanee* (`Z1,Z2, ...`), di numero potenzialmente infinito inizializzate col valore `0`.

Etichette. Il **linguaggio S** permette di utilizzare delle etichette alfabetiche, *label*, per demarcare specifiche linee su cui poter effettuare dei salti, *jump*.

Vengono utilizzate le lettere dell'alfabeto dalla `A` alla `D` numerate, con il caso speciale dell'etichetta `E`, usata per la terminazione del programma, che spiegheremo meglio in seguito.

Statement. Il **linguaggio S** contempla solo 4 tipi di **statement**, o *asserzioni*, che possono essere combinate o meno a **etichette** per formare **istruzioni**. Nello specifico questi statement sono:

- **Pigra**: `V←V`, asserzione nulla;
- **Incremento**: `V←V+1`, incrementa una variabile qualsiasi `V` di `1`;
- **Decremento**: `V←V-1`, decrementa una variabile qualsiasi `V` di `1`, se essa contiene il valore `0` l'istruzione viene ignorata;
- **Salto**: `IF V≠0 GOTO L`, passa alla prima linea etichettata dall'etichetta generica `L` se la variabile generica `V` è diversa da `0`, altrimenti l'istruzione viene ignorata. Nel caso l'etichetta `L` non esista, il programma termina.
  Solitamente questo meccanismo è effettuato con l'etichetta `E`, che viene riservata allo scopo di terminazione.

Esempio: S-Programma

Un primo esempio di **S-Programma** può dunque essere il seguente:

```
[A] X ← X - 1
    Y ← Y + 1
    IF X≠0 GOTO A
```

Codice 1.1: Esempio di S-Programma

Nello specifico, questo **S-Programma** rappresenta la funzione:

$$f(x) = \begin{cases} 1, & x = 0 \\ x, & x \neq 0 \end{cases} \tag{1.4}$$

Poiché il programma ripete l'operazione d'incremento della variabile `Y` un numero `X` di volte, per $x \neq 0$, mentre una volta sola per $x = 0$. Il programma termina infine, dopo l'esecuzione dell'ultima linea.

1.2.2 Esempi di S-Programmi e Macro

Consideriamo invece adesso un **S-Programma** che implementi proprio la *funzione identica*[4] $f(x) = x$:

```
[A] IF X≠0 GOTO B
    Z ← Z + 1
    IF Z ≠ 0 GOTO E
[B] X ← X - 1
    Y ← Y + 1
    Z ← Z + 1
```

[3] Il numero è potenzialmente infinito per non inserire limitazione di numero, ma chiaramente ogni programma ne utilizzerà un numero finito.
[4] Implementazione alternativa 1.10.

```
7      IF Z ≠ 0 GOTO A
```

Codice 1.2: Funzione identica

Possiamo notare come le righe `6--7` effettuano un *salto incondizionato*. È possibile isolarle sfruttando il concetto di **macro**.

Macro

Ogni programma associato a una **funzione** ***(parzialmente)*** **calcolabile**, con **stato** corrente **iniziale**, può essere riutilizzato per il calcolo di un'altra **funzione**, sostituito da un *alias*. Quest'ultimo viene detto **macro**.

Nello specifico definiamo la **macro di salto incondizionato** `GOTO L` come[5]:

```
[L] ...
    ...
    V ← V + 1
    IF V ≠ 0 GOTO L
```

Codice 1.3: Macro di salto incondizionato

Esempi

- Definiamo innanzitutto la **macro di azzeramento** `V←0`:

```
[A] V ← V - 1
    IF V ≠ 0 GOTO A
```

Codice 1.4: Macro di azzeramento

  Questa **macro** corrisponde alla **funzione costante** $f(x) = 0$, ed è di fondamentale importanza per il corretto funzionamento di altre macro.
  Ogni **macro** che d'ora in avanti andremo a definire, conterrà implicitamente la **macro di azzeramento** per garantire lo stato iniziale delle variabili.

- Definiamo infine un'altra importante **macro**, ovvero la **macro di assegnazione** `V←V1`:

```
[C] IF V1 ≠ 0 GOTO A
    GOTO E
[A] V ← V + 1
    V1 ← V1 - 1
    GOTO C
```

Codice 1.5: Macro di assegnazione

Predicati

La codifica di **predicati** invece può essere utilizzata per la definizione di **macro** di *salto* con condizione diversa da `≠0`.

Potendo infatti conservare in una variabile generica `V` l'output di una qualsiasi **funzione k-aria (parzialmente) calcolabile** grazie alla macro di assegnazione, è possibile definire la **macro** di **salto generalizzato**:

```
V ← P(x1,x2, ... , xk)
IF V ≠ 0 GOTO L
```

Codice 1.6: Macro di salto generalizzato

Nel caso infatti che il predicato p sia `VERO`, avremo che la variabile `V` sarà pari a `1`, la condizione dello statement di salto sarà verificata, e avverrà il salto. In caso contrario, si avrà che la variabile `V` sarà pari a `0`, la condizione dello statement di salto non sarà verificata, e il programma procederà all'istruzione successiva.

[5] Approfondimento 3.5. Esercizi 1.12 1.13

Esempo: Funzione totale predicato

Prendiamo ad esempio il predicato d'uguaglianza fra due numeri $p(x_1, x_2) \iff x_1 = x_2$. Possiamo esprimere questo **predicato** usando la **funzione totale**:

$$f(x_1, x_2) = \begin{cases} 1, & x_1 = x_2 \\ 0, & x_1 \neq x_2 \end{cases} \tag{1.5}$$

Un **programma** che implementa questo predicato è:

```
1       Z1 ← X1
2       Z2 ← X2
3   [A] IF Z1 ≠ 0 GOTO B
4       IF Z2 ≠ 0 GOTO E
5       Y ← Y + 1
6       GOTO E
7   [B] Z1 ← Z1 - 1
8       IF Z2 ≠ 0 GOTO C
9       GOTO E
10  [C] Z2 ← Z2 - 1
11      GOTO A
```

Codice 1.7: Predicato logico (uguaglianza)

Funzioni parzialmente calcolabili

Possiamo anche scrivere programmi per le **funzioni parzialmente calcolabili**, come per esempio:

```
1       Y ← X1
2       Z ← X2
3   [C] IF Z ≠ 0 GOTO A
4       GOTO E
5   [A] IF Y ≠ 0 GOTO B
6       GOTO A
7   [B] Y ← Y - 1
8       Z ← Z - 1
9       GOTO C
```

Codice 1.8: Funzione parzialmente calcolabile (sottrazione)

Nello specifico, questo **S-programma** rappresenta:

$$f(x_1, x_2) = \begin{cases} x_1 - x_2, & \text{se } x_1 \geqslant x_2 \\ \uparrow, & \text{altrimenti} \end{cases} \tag{1.6}$$

Dove il simbolo $\uparrow$ indica *l'inesistenza di output*, normalmente chiamata **divergenza** della funzione.
Questa funzione è quindi parzialmente calcolabile dato che può eseguire la sottrazione *se e solo se* il primo numero è maggiore o uguale del secondo[6].

In **linguaggio S** l'inesistenza di output è codificata con l'impossibilità di terminazione del programma. Nel caso specifico possiamo infatti notare che, se verificato il caso $x_1 < x_2$, l'esecuzione rimane bloccata tra le righe `5--6` non giungendo mai alla fine del programma.

1.3 Caratterizzazione di un S-programma

Una volta afferrate così le basi del **linguaggio S** e visti alcuni esempi notevoli, andiamo a formalizzare il concetto di **S-Programma**. Chiamiamo **S-Programma** una qualsiasi *lista finita*[7] d'**istruzioni**.

Una volta definito formalmente il concetto di **S-Programma** possiamo, usando le proprietà che derivano da questa definizione, andarlo a caratterizzare fino a cogliere il parallelismo suddetto tra **S-Programmi** e **funzioni calcolabili**.

Innanzitutto, per descrivere un **S-Programma** è importante tenere traccia dell'aggiornamento delle sue variabili. Per fare ciò ci avvaliamo del concetto di **Stato di un Programma**.

[6] Esercizio 1.14
[7] Importante sottolineare che anche la *lista vuota* è contemplata, rappresentata infatti dal *programma vuoto*.

Stato di un Programma

Per **Stato** di un **Programma** $\mathcal{P}$ si intende un insieme finito di **equazioni** della forma $V = n$, dove V è una *variabile* del **linguaggio S** e $n \in \mathbb{N}$, contenente **una e una sola equazione** per ogni variabile che *compare* in $\mathcal{P}$.

Lo **stato** però non è sufficiente a descrivere un **programma**, in quanto non tiene traccia dell'ordine di esecuzione delle istruzioni. Per avere una descrizione puntuale di un **programma** di *'l' istruzioni*[8], è però necessario contestualizzare il suo **stato** in un determinato momento dell'esecuzione. Per fare questo ci avvaliamo del concetto d'**istantanea**, o *snapshot*.

Istantanea

Definiamo **istantanea** di un **programma** $\mathcal{P}$ di *lunghezza* l una qualsiasi coppia ordinata del tipo:
(i, σ) con $i \in \{1, \cdots, l+1\}$ e $\sigma \in \Sigma$ insieme degli stati.

Dove con $l + 1$ codifichiamo la terminazione del programma, essendo un'istruzione con tale indice non esistente. Chiamiamo l'**istantanea** con questo indice **terminale**.

Le **istantanee** permettono in oltre di descrivere consequenzialmente le **istantanee successive** di un **programma** se abbinate alla istruzione indicizzata da i, a patto che essa non sia un'**istantanea terminale**.

In particolare, partendo da una qualsiasi **istantanea** (i, σ) esistono 4 diverse configurazioni di possibili **istantanee successive** (j, τ) in base all'istruzione in questione.

1. $j = i + 1$ e $\tau = \sigma$, se l'i-esima istruzione è:
   - pigra (`V←V`);
   - decremento (`V←V-1`) con $V = 0$;
   - salto (`IF V≠0 GOTO L`) con $V = 0$.
2. $j = i + 1$ e τ ottenuta da σ sostituendo l'equazione $V = n$ con $V = n + 1$, se l'i-esima istruzione è:
   - incremento (`V←V+1`);
3. $j = i + 1$ e τ ottenuta da σ sostituendo l'equazione $V = n$ con $V = n - 1$, se l'i-esima istruzione è:
   - decremento (`V←V-1`) e $V > 0$;
4. (a) $j = l + 1$ e $\tau = \sigma$, se l'i-esima istruzione è:
   - salto (`IF V≠0 GOTO L`), con $\mathcal{P}$ non contenente istruzioni etichettate con `L` e $V > 0$;

   (b) $j = k$ e $\tau = \sigma$, se l'i-esima istruzione è:
   - salto (`IF V≠0 GOTO L`), con k indice della prima istruzione di $\mathcal{P}$ etichettata con `L` e $V > 0$;

In ultimo chiamiamo **istantanea iniziale** una qualunque coppia (i, σ) con $i = 1$ e σ contenente equazione $V = 0$ per ogni V locale o di output.

Avvalendoci di queste definizioni e caratterizzazioni, possiamo in definitiva definire[9] formalmente cosa si intende per **calcolo terminante di** $\mathcal{P}$.

Calcolo Terminante di P

Sequenza finita d'istantanee $S_1, S_2, S_3 \cdots, S_k$, con S_k terminale, $S_1, S_2, S_3 \cdots, S_{k-1}$ non terminale e, se $1 < i \leq k$, allora S_i è successiva di S_{i-1}.

Date queste definizioni formali, è possibile trarre dei parallelismi tra la vista del programma come funzione e la vista del programma come sequenza d'istruzioni.

Infatti, dato $(r_1, \cdots, r_k) \in \mathbb{N}$ come n-upla d'input di una funzione f, lo **Stato Iniziale** dell'**S-Programma** $\mathcal{P}$ corrispondente sarà $\sigma_1 = \{x_1 = r_1, \cdots, x_k = r_k, y = 0, z_1 = 0, \cdots, z_n = 0\}$, con n numero di variabili temporanee, mentre l'**istantanea iniziale** sarà rappresentata dalla coppia $(1, \sigma_1)$.

[8] Dove l è detta di norma *lunghezza del programma*.

[9] A chiunque turbi l'accoppiata delle parole "definitiva" e "definire" si ricorda la figura retorica di suono della consonanza, ampiamente utilizzata e apprezzata nella poesia da secoli.

Esempo: Calcolo Terminante di $\mathcal{P}$

Prendiamo come esempio il programma 1.1 e andiamo ad analizzare l'andamento del programma per input pari a 2:

0. **Stato Iniziale**: $\sigma_1 = \{X = 2, Y = 0\}$
1. **Istantanea 1**: $(1, \sigma_1)$;
2. **Istantanea 2**: $(2, \sigma_2)$ con $\sigma_2 = \{X = 1, Y = 0\}$;
3. **Istantanea 3**: $(3, \sigma_3)$ con $\sigma_3 = \{X = 1, Y = 1\}$;
4. **Istantanea 4**: $(1, \sigma_3)$;
5. **Istantanea 5**: $(2, \sigma_4)$ con $\sigma_4 = \{X = 0, Y = 1\}$;
6. **Istantanea 6**: $(3, \sigma_5)$ con $\sigma_5 = \{X = 0, Y = 2\}$;
7. **Istantanea 7**: $(4, \sigma_5)$ **terminale**;
8. **Stato Finale**: $\sigma_4 = \{X = 0, Y = 2\}$;

È possibile notare che se venissero forniti più input di quelli necessari, la successione d'istantanee rimarrebbe invariata, ignorando gli input in eccesso.
Se ad esempio prendiamo come input la n-upla $(2, 3, 4)$, lo **stato iniziale** sarà $\sigma_1 \cup \{X_1 = 3, X_2 = 4\}$, ma si comporterà come se fosse stato fornito solo 2 come input.
Inoltre è possibile generalizzare il calcolo ulteriormente, contemplando anche la possibilità d'input sottodimensionati rispetto al numero di variabili richieste.
Considerando infatti le variabili d'input senza inizializzazione esplicita come poste a `0`, l'avanzamento delle istantanee procederà normalmente fino alla terminazione in maniera analoga a quelle appena viste.

Formalizziamo dunque la **funzione k-aria** calcolata da $\mathcal{P}$ come:

$$\Psi_{\mathcal{P}}^{(k)}(x_1, x_2, \cdots, x_k) = \begin{cases} y \text{ nello stato terminale}, & \text{se esiste un calcolo terminante di } \mathcal{P} \text{ con input } x_1, x_2, \cdots, x_k \\ \uparrow, & \text{altrimenti} \end{cases} \tag{1.7}$$

Possiamo finalmente dare una definizione formale del concetto di **funzione parzialmente calcolabile** accennato all'inizio di questo capitolo:

Funzione (Parzialmente) Calcolabile

Una **funzione** $f : D \subseteq \mathbb{N}^k \to \mathbb{N},\ k \in \mathbb{N}$ è detta ***parzialmente*** **calcolabile** se esiste un **S-Programma** $\mathcal{P}$ tale che:

$$\forall (x_1, x_2, \cdots, x_k) \in \mathbb{N}^k, f(x_1, x_2, \cdots, x_k) = \Psi_{\mathcal{P}}^{(k)}(x_1, x_2, \cdots, x_k) \tag{1.8}$$

È necessario, logicamente, che la funzione f sia definita per *tutte e sole* le n-uple per cui $\Psi_{\mathcal{P}}^{(k)}$ termina.

Formalizziamo dunque anche il processo di *sostituzione di macro* vista in precedenza. Sia f funzione n-aria calcolata da $\mathcal{P} = \mathcal{P}(y, x_1, x_2, \cdots, x_n, z_1, z_2, \cdots, z_k, E, A_1, \cdots, A_l)$. Allora la macro $V' \leftarrow f(V_1, V_2, \cdots, V_n)$ all'interno di un altro programma $\mathcal{P}_2$ va sostituita con:

```
1        Zm ← 0
2        Zm+1 ← V1
3        ⋮
4        Zm+n ← Vn
5        Zm+n+1 ← 0
6        Zm+n+2 ← 0
7        ⋮
8        Zm+n+k ← 0
9        Q = P(Z_m, Z_{m+1}, Z_{m+2}, ..., Z_{m+n+k}, E_m, A_{m+1}, A_{m+l})
10   [Em] V'← Zm
```

Codice 1.9: Macro

Dove per la linea `9` si intende aver sostituito in $\mathcal{P}_2$ le istruzioni del programma $\mathcal{P}$, e m è sufficientemente grande da non creare conflitti con le variabili di $\mathcal{P}_2$. È importante inoltre notare la presenza della **label** `[Em]` che permette la corretta assegnazione del valore di output nella variabile `V'` anche in caso la macro termini per *salto* a etichetta `[E]`[10].

[10] Esercizi da 1.17 a 1.18

Approfondimento.

Programma alternativo per l'implementazione della funzione identica $f(x) = x$ con utilizzo di **macro** di **salto incondizionato**.

```
[A] IF X ≠ 0 GOTO B
    GOTO C
[B] X ← X - 1
    Y ← Y + 1
    Z ← Z + 1
    GOTO A
[C] IF Z ≠ 0 GOTO D
    GOTO E
[D] X ← X + 1
    Z ← Z - 1
    GOTO C
```

Codice 1.10: Funzione identica

Questa implementazione della funzione identica ha il vantaggio di **preservare** il valore delle variabili d'input, buona pratica generale per l'implementazione di qualsiasi funzione.

Approfondimento.

Programma per l'implementazione della funzione somma $f(x_1, x_2) = x_1 + x_2$.

```
    Y ← X1
    Z ← X2
[A] IF Z ≠ 0 GOTO B
    GOTO E
[B] Z ← Z - 1
    Y ← Y + 1
    GOTO A
```

Codice 1.11: Funzione somma

Faremo riferimento a questo programma da ora in poi con la macro `V ← V1 + V2`

Esercizio.

Si scriva un S-Programma che implementi la funzione $f(x_1, x_2) = x_1 x_2$

```
    Z ← X1
[A] Y ← Y + X2
    Z ← Z - 1
    IF Z ≠ 0 GOTO A
```

Codice 1.12: Funzione prodotto

Faremo riferimento a questo programma da ora in poi con la macro `V ← V1 * V2`

Esercizio.

Si scriva un S-Programma che implementi la funzione $f(x_1) = 3x_2$

```
Z ← Z + 1
Z ← Z + 1
Z ← Z + 1
Y ← X1 * Z
```

Codice 1.13: Funzione prodotto con costante

Esercizio.

Si scriva un S-Programma che implementi il predicato di parità

$$p(x) = \begin{cases} 1, & x \equiv_2 0 \\ \uparrow, & altrimenti \end{cases} \tag{1.9}$$

```
    Z1 ← Z1 + 1
    Z2 ← Z2 + 1
    Z2 ← Z2 + 1
[A] IF X ≠ 0 GOTO B
    Y ← Y + 1
    GOTO E
[B] IF X == Z1 GOTO B
    X ← X - Z2
    GOTO A
```

Codice 1.14: Predicato di Parità

Esercizio.

Si calcoli la successione delle istantanee relative al programma 1.7 per gli input $(x_1, x_2, x_3) = (2, 1, 3)$ *e* $x_1 = 5$.

Programma 1.7 in **linguaggio S** *"puro"*.

```
[A] IF X1 ≠ 0 GOTO B
    IF X2 ≠ 0 GOTO E
    Y ← Y + 1
    IF Y ≠ 0 GOTO E
[B] X1 ← X1 - 1
    IF X2 ≠ 0 GOTO C
    Z ← Z + 1
    IF Z ≠ 0 GOTO E
[C] X2 ← X2 - 1
    Z ← Z + 1
    IF Z ≠ 0 GOTO A
```

Codice 1.15: Predicato di Uguaglianza senza Macro

Input: $(x_1, x_2, x_3) = (2, 1, 3)$;

1. **Stato Iniziale** $\sigma_1 = \{x_1 = 2, x_2 = 1, x_3 = 3, y = 0, z = 0\}$;
2. **Istantanea 1** $(1, \sigma_1)$;
3. **Istantanea 2** $(5, \sigma_1)$;
4. **Istantanea 3** $(6, \sigma_2)$, con $\sigma_2 = \{x_1 = 1, x_2 = 1, x_3 = 3, y = 0, z = 0\}$;

5. **Istantanea 4** $(9, \sigma_2)$;
6. **Istantanea 5** $(10, \sigma_3)$, con $\sigma_3 = \{x_1 = 1, x_2 = 0, x_3 = 3, y = 0, z = 0\}$;
7. **Istantanea 6** $(11, \sigma_4)$, con $\sigma_4 = \{x_1 = 1, x_2 = 0, x_3 = 3, y = 0, z = 1\}$;
8. **Istantanea 7** $(1, \sigma_4)$;
9. **Istantanea 8** $(5, \sigma_4)$;
10. **Istantanea 9** $(6, \sigma_5)$, con $\sigma_5 = \{x_1 = 0, x_2 = 0, x_3 = 3, y = 0, z = 1\}$;
11. **Istantanea 10** $(7, \sigma_5)$;
12. **Istantanea 11** $(8, \sigma_6)$, con $\sigma_6 = \{x_1 = 0, x_2 = 0, x_3 = 3, y = 0, z = 2\}$;
13. **Istantanea 12** $(12, \sigma_6)$, **terminale**;
14. **Stato Finale** $\sigma_6 = \{x_1 = 0, x_2 = 0, x_3 = 3, y = 0, z = 2\}$.

Input: $x_1 = 5$;

1. **Stato Iniziale** $\sigma_1 = \{x_1 = 5, x_2 = 0, y = 0, z = 0\}$;
2. **Istantanea 1** $(1, \sigma_1)$;
3. **Istantanea 2** $(5, \sigma_1)$;
4. **Istantanea 3** $(6, \sigma_2)$, con $\sigma_2 = \{x_1 = 4, x_2 = 0, y = 0, z = 0\}$;
5. **Istantanea 4** $(7, \sigma_2)$;
6. **Istantanea 5** $(8, \sigma_3)$, con $\sigma_3 = \{x_1 = 4, x_2 = 0, y = 0, z = 1\}$;
7. **Istantanea 6** $(12, \sigma_3)$, **terminale**;
8. **Stato Finale** $\sigma_3 = \{x_1 = 4, x_2 = 0, y = 0, z = 1\}$.

Esercizio

Si scriva un S-Programma che implementi la funzione potenza $f(x_1, x_2) = x_1^{x_2}$.

```
    Y ← Y + 1
[A] IF X2 ≠ 0 GOTO B
    GOTO E
[B] X2 ← X2 - 1
    Y ← Y * X1
    GOTO A
```

Codice 1.16: Funzione potenza

Esercizio.

Considerando il seguente programma $\mathcal{P}$*, determinare* $\Psi_{\mathcal{P}}^{(1)}(r_1), \Psi_{\mathcal{P}}^{(2)}(r_1, r_2), \Psi_{\mathcal{P}}^{(3)}(r_1, r_2, r_3)$.

```
    Y ← X1
[A] IF X2 == 0 GOTO E
    Y ← Y + 1
    Y ← Y + 1
    X2 ← X2 - 1
    GOTO A
```

Codice 1.17: Programma P

- $\Psi_{\mathcal{P}}^{(1)}(r_1) = r_1$;
- $\Psi_{\mathcal{P}}^{(2)}(r_1, r_2) = r_1 + 2r_2$;
- $\Psi_{\mathcal{P}}^{(3)}(r_1, r_2, r_3) = r_1 + 2r_2$.

Esercizio.

1. $\forall k \geq 0$, *sia* $f_k(x) = k$. *Dimostrare che* f_k *è calcolabile*[11].
2. *Un programma è* ***straightline*** *se non contiene istruzioni di salto. Mostrare per induzione su* k *che se* $\mathcal{P}$ *è* ***straightline*** *e ha lunghezza* k *allora* $\Psi_{\mathcal{P}}^{(1)}(x) \leq k, \forall x \in \mathbb{N}$.
3. *Mostrare che se* $\mathcal{P}$ *è straightline e calcola* f_k, *allora* $\mathcal{P}$ *ha lunghezza* $\geq k$.
4. *Mostrare che nessun programma straightline calcola la funzione* $f(x) = x + 1$, *e quindi dedurne che la classe delle funzioni calcolate dai programmi straightline è* ***propriamente*** *contenuta nella classe delle funzioni calcolabili.*

1. Dimostriamo per induzione.

Base induttiva $k = 0$. Vero poiché $f_0(x) = 0$ è calcolata dal **programma vuoto**.
Passo induttivo $k \to k + 1$. Vero poiché $f_{k+1}(x) = k + 1$ è calcolata aggiungendo al programma che calcola f_k, esistente per ipotesi d'induzione, l'istruzione `Y ← Y + 1`.

2. Dimostriamo per induzione.

Base induttiva $k = 0$. Vero avremo il **programma vuoto**, dunque con output $0 \leq k$

Passo induttivo $k \to k + 1$. Vero poiché avendo per ipotesi di induzione che $\Psi_{\mathcal{P}}^{(1)}(x) \leq k$, il programma di lunghezza $k + 1$, dovendo essere **straightline** dovrà avere come istruzione in più necessariamente una di queste 3 istruzioni:

- **Pigra.** Porta a un output uguale al programma precedente, dunque $y_{k+1} = y_k \leq k \leq k + 1$;
- **Decremento.** Porta a un output minore di quello precedente di 1, dunque $y_{k+1} = y_k - 1 \leq y_k \leq k \leq k + 1$;
- **Incremento.** Porta a un output maggiore di quello precedente di 1, ma essendo $y_k \leq k$ allora $y_{k+1} = y_k + 1 \leq k + 1$.

3. Dimostriamo per costruzione.

Essendo il programma **straightline** potrà avere solo istruzioni **pigre**, di **incremento** o di **decremento**.
Essendo la variabile di output `Y` inizializzata a 0, per calcolare f_k sono necessario come minimo k istruzioni di incremento, dunque la lunghezza del programma sarà almeno k. In caso ci siano istruzioni non di incremento la lunghezza sarà maggiore di k avendo bisogno di al meno k istruzioni di incremento.

4. Dimostriamo per costruzione.

Preso un programma $\mathcal{P}$ di k istruzioni, se questo è **straightline** avrà come output $y \leq k$, come visto nel punto 2. Ma la funzione $f(x) = x + 1$ ha $y > k, \forall x \geq k$. Dunque nessun programma **straightline** può calcolare la funzione $f(x) = x + 1$.

Esercizio.

Mostrare che il predicato $P(x_1, x_2) \iff x_1 \leq x_2$ *è calcolabile costruendo un S-Programma.*

```
     IF X1 ≠ X2 GOTO A
     Y ← Y + 1
     GOTO E
[A]  IF X1 ≠ 0 GOTO B
     Y ← Y + 1
     GOTO E
[B]  IF X2 ≠ 0 GOTO C
     GOTO E
[C]  X1 ← X1 - 1
     X2 ← X2 - 1
     GOTO A
```

Codice 1.18: Predicato Minore Uguale

Implementazione alternativa:

```
[C]  IF X2 ≠ 0 GOTO A
     IF X1 ≠ 0 GOTO B
     Y ← Y + 1
[B]  GOTO E
[A]  X2 ← X2 - 1
     X1 ← X1 - 1
     GOTO C
```

Codice 1.19: Predicato Minore Uguale

[11] Da ora in avanti le funzioni costanti saranno considerate tutte calcolabili, e dunque usate in S-Programmi

Operazioni sulle funzioni e Paradigma Funzionale

2.1 Paradigma Funzionale

Fino a ora abbiamo utilizzato un approccio *diretto* per la valutazione della calcolabilità di una funzione. Questo approccio presenta svariati limiti, primo dei quali è la necessità di dimostrare come calcolabile una singola funzione alla volta, senza poter trarre conclusioni di carattere generale o di caratterizzare insiemi o classi di funzioni.

Per avere un approccio più generale e sistematico alla valutazione della calcolabilità delle funzioni è necessario dunque approcciare il problema in maniera diversa, utilizzando un paradigma diverso che sfrutti[1] le proprietà e le operazioni delle funzioni stesse per valutarne la calcolabilità.

2.2 Operazioni fondamentali e Classi PRC

Con **operazioni sulle funzioni** facciamo riferimento a operazioni algebriche definite nell'insieme delle funzioni, ovvero operazioni che hanno sia come operando che come risultato delle funzioni. Definiamo dunque le due operazioni fondamentali sulle funzioni: la **composizione** e la **ricorsione primitiva**.

2.2.1 Composizione

Composizione di Funzioni

Sia f una funzione k-aria e $g_1, g_2, \cdots, g_k$ funzioni n-arie. Allora la funzione h n-aria è **composizione** di $f, g_1, g_2, \cdots, g_k$ se:

$$h(x_1, \cdots, x_n) = f(g_1(x_1, \cdots, x_n), \cdots, g_k(x_1, \cdots, x_n)) \tag{2.1}$$

È importante notare che la composizione di funzioni è ben definita anche per *funzioni parziali*.
Infatti si avrà che, data $(x_1, \cdots, x_n) \in \mathbb{N}$:

$$h(x_1, \cdots, x_n) \uparrow \iff \exists i \in \{1, \cdots, k\} : g_i(x_1, \cdots, x_n) \uparrow \vee f(g_1(x_1, \cdots, x_n), \cdots, g_k(x_1, \cdots, x_n)) \uparrow \tag{2.2}$$

Con ragionamento simile è anche possibile dimostrare che la composizione di **funzioni parzialmente calcolabili** è necessariamente **parzialmente calcolabile**.

[1] Il passaggio dall'approccio diretto a uno di carattere più generale per la valutazione della calcolabilità di una funzione è assimilabile al passaggio dal paradigma imperativo a quello funzionale.

Teorema di Composizione

Enunciato. Sia h ricavata per composizione di $f, g_1, g_2, \cdots, g_k$ **parzialmente calcolabili**, allora h è **parzialmente calcolabile**.

Dimostrazione. Per definizione di **composizione** si ha $h = \Psi_{\mathcal{P}}^{(n)}$, dove $\mathcal{P}$ è:

```
Z1 ← g_1(x_1, ···, x_n)
⋮
Zk ← g_k(x_1, ···, x_n)
Y ← f(Z_1, ···, Z_k)
```

Codice 2.1: Dimostrazione Teorema di Composizione

Inoltre come notato in precedenza il programma non arriva a terminazione *se e solo se* uno dei programmi g_i o f non termina, dunque:

$$h \text{ non totale} \implies \exists i \in \{1, \cdots, k\} : g_i \text{ non totale} \vee f \text{ non totale} \tag{2.3}$$

Non è necessariamente vero il viceversa però. Infatti se il codominio di una funzione totale f che compone h è un sottoinsieme del dominio di un'altra parziale g che compone a sinistra[2], la funzione composta sarà totale, poiché quella totale escluderà i casi gli input di non terminazione dalla prima. Vediamo qualche esempio al riguardo per chiarire il concetto.

Esempi

- Sia $h_1(x) = 2x$. Possiamo considerare h come funzione composta da $f_1(x_1, x_2) = x_1 \cdot x_2$, $g_1(x) = 2$ e $g_2(x) = x$, o alternativamente, notando che $h_1(x) = 2x = x + x$ ovvero composizione di $f_2(x_1, x_2) = x_1 + x_2$ e $g_2(x) = x$, dunque h_1 è calcolabile.
- Sia $h_2(x) = 4x^2$. Possiamo considerare h_2 come funzione composta dalle funzioni f_1 e h_1 sopracitate, infatti $h_2(x) = 4x^2 = 2x \cdot 2x$, dunque h_2 è calcolabile.
- Sia $h_3(x) = 4x^2 - 2x$. Si ha $h_3(x) \downarrow \forall x \in \mathbb{N}, 4x^2 \geq 2x$. Abbiamo dunque che la funzione h_3 è totale. Ma $h_3(x) = f_3(h_2(x), h_1(x))$, dove $f_3(x_1, x_2) = x_1 - x_2$ **non è totale**[a]. Abbiamo dunque che l'implicazione non vale in entrambi i sensi come precedentemente detto.

[a] In caso di dubbi controllare qui 1.8

2.2.2 Ricorsione Primitiva

L'operazione di **ricorsione primitiva** è un'operazione che permette di definire una funzione in maniera ricorsiva, ovvero definendo un caso base e un passo ricorsivo, formalizzando matematicamente il concetto di algoritmo ricorsivo. Nonostante sia definibile anche per le **funzioni parziali**, noi tratteremo solo il caso delle **funzioni totali**.

Per una funzione h unaria diremo che è ricavata per **ricorsione primitiva** da $k \in \mathbb{N}$ e g funzione binaria **totale** se:

$$\begin{cases} h(0) = k \\ h(t+1) = g(t, h(t)), \quad \forall t \in \mathbb{N}^* \end{cases} \tag{2.4}$$

Nonostante affronteremo prevalentemente funzioni unarie, è importante dare una definizione generale per qualsiasi funzione.

Ricorsione Primitiva

Una funzione h $n-aria$ si ricava per ricorsione primitiva da f $(n-1)$-aria e da g $(n+1)$-aria, se $\forall x_1, \cdots, x_{n-1} \in \mathbb{N}$ vale:

$$\begin{cases} h(x_1, \cdots, x_{n-1}, 0) = f(x_1, \cdots, x_{n-1}) \\ h(x_1, \cdots, x_{n-1}, t+1) = g(x_1, \cdots, x_{n-1}, t, h(x_1, \cdots, x_{n-1}, t)), \quad \forall t \in \mathbb{N}^* \end{cases} \tag{2.5}$$

[2] In notazione algebrica $f \circ g$ equivale a $f(g(x))$, dunque è l'operando sinistro a poter essere parziale, non essendo la composizione commutativa.

Teorema di Ricorsione primitiva

Enunciato. Sia h ricavata per ricorsione primitiva da f e g calcolabili, allora h è calcolabile.

Dimostrazione. Per definizione di **ricorsione primitiva** si ha $h = \Psi_{\mathcal{P}}^{(n)}$, dove $\mathcal{P}$ è:

```
1        Y ← f(X_1, ···, X_{n-1})
2    [A] IF Xn == 0 GOTO E
3        Y ← g(X_1, ···, X_{n-1}, Y, Z)
4        Z ← Z + 1
5        Xn ← Xn - 1
6        GOTO A
```

Codice 2.2: Dimostrazione Teorema di Ricorsione Primitiva

Questo programma calcola esattamente la funzione h poiché, se ci trovassimo nel primo caso della definizione 2.2.2 l'output sarebbe dato dalle linee `1--2`, altrimenti, se ci trovassimo nel secondo caso, le linee `3--6` calcolerebbero ricorsivamente l'output di h, dimostrata così calcolabile.

2.2.3 Classi PRC e Funzioni Ricorsive Primitive

Le due operazioni appena definite sono alla base del paradigma funzionale e definiscono, insieme ad alcune specifiche funzioni, una classe di funzioni chiamate **PRC**.

Classi PRC di funzioni

Un insieme di funzioni totali $\mathcal{C}$ è detto **PRC** se è chiuso rispetto alle operazioni di composizione e ricorsione primitiva e contiene:

- La funzione nulla $n(x) = 0$;
- La funzione successore $s(x) = x + 1$;
- Le funzioni proiezione $u_i^{(n)}(x_1, \cdots, x_n) = x_i$, $\forall (n \geq 1 \wedge 1 \leq i \leq n)$.

Dato che le funzioni iniziali[3] sono facilmente calcolabili e visti i risultati dei teoremi 2.1 e 2.2, possiamo notare che l'insieme delle **funzioni calcolabili** è una **classe PRC**.

Funzioni Ricorsive Primitive

Una funzione è detta **ricorsiva primitiva** se è ottenibile dalle funzioni iniziali mediante un numero finito di applicazioni di composizione e ricorsione primitiva.

È importante notare che, dalla definizione di **classe PRC** e dalle proprietà degli **insiemi chiusi**, l'insieme delle **funzioni ricorsive primitive** è contenuto in tutte le **classi PRC**, ed è dunque in particolare la più piccola **classe PRC**.

In particolare, essendo **l'insieme delle funzioni calcolabili** una **classe PRC**, si ha che **ogni funzione ricorsiva primitiva è calcolabile**.

[3] Si noti che le funzioni proiezioni sono un **insieme infinito** e non **una sola funzione**. Infatti ne esiste una $\forall (n, i)$ con $n \in \mathbb{N}$ e $i \in \{1, \cdots, n\}$.

Andiamo ora a elencare una serie di esempi significativi di funzioni ricorsive primitive, che verranno poi usate sia per definire nuove funzioni che per dimostrare l'appartenenza di altre funzioni alla stessa classe.

Esempi

1. Consideriamo la funzione $a(x_1, x_2) = x_1 + x_2$. Possiamo vedere che questa è **ricorsiva primitiva** da:

$$\begin{cases} a(x_1, 0) = x_1 = u_1^{(1)}(x) \\ a(x_1, t+1) = x + t + 1 = a(x_1, t) + 1 = s(a(x_1, t)) = g(t, a(x, t), x), \quad \forall t \in \mathbb{N}^* \end{cases} \tag{2.6}$$

   Dove $g(x, y, z) = s(y) = s(u_2^{(3)}(x, y, z))$ è ottenuta per composizione di funzioni iniziali, ed è dunque **ricorsiva primitiva**.

2. Consideriamo la funzione $m(x, y) = x \cdot y$. Possiamo vedere che questa è **ricorsiva primitiva** da:

$$\begin{cases} m(x, 0) = 0 = n(x) \\ m(x, t+1) = m(x, t) + x = a(m(x, t), x) = g'(t, m(x, t), x), \quad \forall t \in \mathbb{N}^* \end{cases} \tag{2.7}$$

   Dove $g'(x, y, z) = a(u_2^{(3)}(x, y, z), u_3^{(3)}(x, y, z))$ è ottenuta per composizione di funzioni iniziali, ed è dunque **ricorsiva primitiva**.

3. Consideriamo la funzione **fattoriale** $x!$, definita come:

$$x! = \begin{cases} 1, & x = 0 \\ x \cdot (x-1)!, & x > 0 \end{cases} \tag{2.8}$$

   Possiamo vedere che questa è **ricorsiva primitiva** da:

$$\begin{cases} 0! = 1 = s(0) \\ (t+1)! = (t+1) \cdot t! = m(t+1, t!) = g(t+1, t!), \quad \forall t \in \mathbb{N}^* \end{cases} \tag{2.9}$$

4. Consideriamo la funzione **precedente**:

$$p(x) = \begin{cases} 0, & x = 0 \\ x - 1, & x > 0 \end{cases} \tag{2.10}$$

   Possiamo vedere che questa è **ricorsiva primitiva** da:

$$\begin{cases} p(0) = 0 = n(x) \\ p(t+1) = t = u_1^{(2)}(t, p(t)) = g(t, p(t)), \quad \forall t \in \mathbb{N}^* \end{cases} \tag{2.11}$$

5. Consideriamo ora un'alternativa alla funzione Sottrazione:

$$x \dot{-} y = \begin{cases} 0, & x < y \\ x - y, & x \geq y \end{cases} \tag{2.12}$$

   Possiamo vedere che questa è **ricorsiva primitiva** da:

$$\begin{cases} x \dot{-} 0 = x = u_1^{(1)}(x) \\ x \dot{-} (t+1) = p(x \dot{-} t) = g(x, x \dot{-} t, t), \quad \forall t \in \mathbb{N}^* \end{cases} \tag{2.13}$$

   Dove $g(x, y, z) = p(u_2^{(3)}(x, y, z))$ è ottenuta per composizione di funzioni iniziali, ed è **ricorsiva primitiva**[a].

6. Consideriamo ora il predicato[b] $\alpha(x) \iff x = 0$, esprimibile anche come:

$$\alpha(x) = \begin{cases} 1, & x = 0 \\ 0, & x > 0 \end{cases} = 1 \dot{-} x \tag{2.14}$$

   Questo predicato è **ricorsivo primitivo**, essendo composizione di funzioni dimostrate come tali.

7. Consideriamo ora il predicato $\beta(x, y) \iff x = y$, esprimibile anche come[c]:

$$\beta(x, y) = \begin{cases} 1, & x = y \\ 0, & x \neq y \end{cases} = \alpha(|x - y|) \tag{2.15}$$

   Questo predicato è ricorsivo primitivo, essendo composizione di funzioni dimostrate come tali[d].

[a] Per dimostrare che una funzione è **ricorsiva primitiva** è sufficiente dimostrare che è ottenuta da **funzioni ricorsive primitive**. Da ora in avanti non ci si ricondurrà più a funzioni iniziali per le dimostrazioni.

[b] Questo predicato è detto *rilevatore di zeri*, è applicato a un predicato può svolgere il ruolo della *negazione logica*

[c] Funzione *Sottrazione in Valore Assoluto*: 2.42

[d] La funzione $f(x, y) = |x - y|$ è definita e dimostrata come ricorsiva primitiva nell'esercizio 2.42. Esercizi consigliati: 2.42, 2.43

Definite le classi PRC e, notato che l'insieme delle funzioni calcolabili ne è un esempio, è naturale chiedersi se esse siano chiuse rispetto ad altre operazioni oltre a quelle che le definiscono.

Proprietà di chiusura di questo tipo sono importanti per poter caratterizzare le classi PRC e per poter dimostrare l'appartenenza di una funzione a una di esse, e dunque la sua calcolabilità.

Andiamo dunque a elencare e dimostrare le proprietà di chiusura delle classi PRC.

2.3.1 Chiusura per congiunzione, disgiunzione e negazione

Teorema di Chiusura per congiunzione, disgiunzione e negazione

Enunciato. Siano p e q predicati n-ari appartenenti a $\mathcal{C}$ **classe PRC**, allora anche i predicati $p \wedge q$, $p \vee q$, $\neg p$, $\neg q$ appartengono a $\mathcal{C}$.

Dimostrazione.

- $(p \wedge q)(x_1, \cdots, x_n) = p(x_1, \cdots, x_n) \cdot q(x_1, \cdots, x_n)$, è dunque **ricorsivo primitivo** da composizione di funzioni ricorsive primitive.
- $(\neg p)(x_1, \cdots, x_n) = \alpha(p(x_1, \cdots, x_n))$, è dunque **ricorsivo primitivo** da composizione di funzioni ricorsive primitive, vale il ragionamento analogo per $\neg q$.
- $(p \vee q)(x_1, \cdots, x_n) \iff \neg(\neg p(x_1, \cdots, x_n) \wedge \neg q(x_1, \cdots, x_n))$, è dunque **ricorsivo primitivo** dalle leggi di De Morgan e dal ragionamento precedente.

Avendo dimostrato che i predicati $x \leq y$ e $x = y$ sono **ricorsivi primitivi** e utilizzando il teorema appena dimostrato, sappiamo che sono ricorsivi primitivi anche i **predicati** $x < y \iff x \leq y \wedge \neg(y = x)$, $x \geq y \iff \neg(x < y)$ e $x > y$, dalle precedenti.

2.3.2 Definizione per Casi

Andiamo ora a individuare un ulteriore metodo per definire funzioni che, come vedremo, è una proprietà di chiusura per le classi PRC, ovvero la **definizione per casi**.

Definizione per Casi

Siano g e h funzioni n-arie e p predicato n-ario allora la funzione n-aria f è **definita per composizione** se definita come:

$$f(x_1, \cdots, x_n) = \begin{cases} g(x_1, \cdots, x_n), & p(x_1, \cdots, x_n) \\ h(x_1, \cdots, x_n), & \neg p(x_1, \cdots, x_n) \end{cases} \tag{2.16}$$

In altre parole una funzione **definita per casi** è una funzione la cui immagine dipende da un predicato, ovvero da una condizione che può essere vera o falsa riguardante le incognite della funzione stessa. Possiamo trovare una proprietà di chiusura per le **Classi PRC** per la **definizione per casi** simile a quelle viste per composizione e ricorsione primitiva.

Teorema di Definizione per Casi

Enunciato. Siano g e h funzioni n-arie in $\mathcal{C}$ **classe PRC** e p predicato n-ario in $\mathcal{C}$, allora a $\mathcal{C}$ appartiene anche la funzione n-aria f **definita per casi** da g, h e p.

Dimostrazione. Si ha:

$$f(x_1, \cdots, x_n) = g(x_1, \cdots, x_n)p(x_1, \cdots, x_n) + h(x_1, \cdots, x_n)(\neg p)(x_1, \cdots, x_n) \tag{2.17}$$

Per la proprietà di chiusura per **composizione** e **ricorsione primitiva** dimostrata in precedenza, $f \in \mathcal{C}$

È possibile generalizzare la definizione di **definizione per casi** a più casi, ovvero a più funzioni e più predicati, la cui proprietà di chiusura è a sua volta dimostrabile[4].

[4] Dimostrazione alternativa: 2.5. Esercizi consigliati: 2.48.

Corollario al Teorema di Definizione per Casi

Enunciato. Se $g_1, \cdots, g_k, h, p_1, \cdots, p_k$ sono funzioni e predicati n-ari in $\mathcal{C}$ **classe PRC**, allora a $\mathcal{C}$ appartiene anche la funzione n-aria f definita come:

$$f(x_1, \cdots, x_n) = \begin{cases} g_1(x_1, \cdots, x_n), & p_1(x_1, \cdots, x_n) \\ \vdots & \\ g_k(x_1, \cdots, x_n), & p_k(x_1, \cdots, x_n) \\ h(x_1, \cdots, x_n), & \text{altrimenti} \end{cases} \tag{2.18}$$

Dimostrazione. Si può dimostrare che f è **ricorsiva primitiva** per **induzione** su k. Il **caso base** è rappresentato dal teorema precedente. Per quando riguarda il **passo induttivo** $k \implies k+1$, consideriamo la funzione f' definita come:

$$f'(x_1, \cdots, x_n) = \begin{cases} g_k(x_1, \cdots, x_n), & p_k(x_1, \cdots, x_n) \\ h(x_1, \cdots, x_n), & \text{altrimenti} \end{cases} \tag{2.19}$$

Per il teorema precedente $f' \in \mathcal{C}$. A questo punto possiamo riscrivere f come:

$$f(x_1, \cdots, x_n) = \begin{cases} g_1(x_1, \cdots, x_n), & p_1(x_1, \cdots, x_n) \\ \vdots & \\ g_{k-1}(x_1, \cdots, x_n), & p_{k-1}(x_1, \cdots, x_n) \\ f'(x_1, \cdots, x_n), & \text{altrimenti} \end{cases} \tag{2.20}$$

Per ipotesi di induzione, essendo ora f definita da k funzioni appartenenti a $\mathcal{C}$ e k predicati appartenenti a $\mathcal{C}$, $f \in \mathcal{C}$.

2.3.3 Operatori di Sommatoria e Produttoria e Quantificatori Limitati

Come abbiamo già visto, la combinazione di funzioni $f \in \mathcal{C}$ **classe PRC**, mediante somma e prodotto appartiene ancora $\mathcal{C}$. Possiamo generalizzare questo processo di composizione mediante gli operatori di **sommatoria e produttoria**.

Sommatoria e Produttoria di funzioni

Sia f una funzione $(n+1)$-aria, allora definiamo come **operatori di sommatoria e produttoria** le funzioni:

$$\begin{aligned} \sigma_f(x_1, \cdots, x_n, y) &= \sum_{i=0}^{y} f(x_1, \cdots, x_n, i) \\ \pi_f(x_1, \cdots, x_n, y) &= \prod_{i=0}^{y} f(x_1, \cdots, x_n, i) \end{aligned} \tag{2.21}$$

Come le operazioni precedenti, anche per gli **operatori di sommatoria e produttoria** è possibile dimostrare una proprietà di chiusura per le **classi PRC**.

Teorema di chiusura per Sommatoria e Produttoria delle classi PRC

Enunciato. Sia f una funzione $(n+1)$-aria in $\mathcal{C}$ **classe PRC**, allora appartengono a $\mathcal{C}$ anche le funzioni σ_f e π_f.

Dimostrazione. Dimostriamo per **ricorsione primitiva**. Si ha:

$$\begin{cases} \sigma_f(x_1, \cdots, x_n, 0) = f(x_1, \cdots, x_n, 0) \\ \sigma_f(x_1, \cdots, x_n, t+1) = \sigma_f(x_1, \cdots, x_n, t) + f(x_1, \cdots, x_n, t+1), \forall t \in \mathbb{N}^* \end{cases} \tag{2.22}$$

Si può dunque vedere che σ_f è **ricorsiva primitiva** da composizione di funzioni iniziali. Per ragionamento analogo si dimostra anche che π_f è **ricorsiva primitiva**.

Gli **operatori di sommatoria e produttoria** sono utili per dimostrare come appartenente a **classi PRC** ulteriori costrutti matematici. Un primo esempio sono i **quantificatori limitati esistenziale e universale**.

Quantificatori limitati

Sia p un predicato $(n+1)$-ario, allora definiamo i **quantificatori limitati esistenziale e universale** E_p e U_p come:

$$\begin{aligned} E_p(x_1, \cdots, x_n, y) &\iff \exists t \leq y : p(x_1, \cdots, x_n, t) \\ U_p(x_1, \cdots, x_n, y) &\iff \forall t \leq y : p(x_1, \cdots, x_n, t) \end{aligned} \tag{2.23}$$

È fondamentale definire questi quantificatori **limitati superiormente**, poiché altrimenti non sarebbe possibile dimostrare la proprietà di chiusura delle **classi PRC** a loro relativa.

Teorema di chiusura per quantificatori limitati delle classi PRC

Enunciato. Sia p una funzione $(n+1)$-ario in $\mathcal{C}$ **classe PRC**, allora appartengono a $\mathcal{C}$ anche i predicati E_p e U_p $(n+1)$-ari definiti come:

$$E_p(x_1, \cdots, x_n, y) \iff \exists t \leq y : p(x_1, \cdots, x_n, t) U_p(x_1, \cdots, x_n, y) \iff \forall t \leq y : p(x_1, \cdots, x_n, t) \tag{2.24}$$

Dimostrazione. Per E_p si ha che:

$$E_p(x_1, \cdots, x_n, y) \iff \sum_{t=0}^{y} p(x_1, \cdots, x_n, y) > 0 \tag{2.25}$$

poiché la sommatoria darà 0 nel caso non esista un t tale che $p(x_1, \cdots, x_n, t)$, e viceversa un valore positivo nel caso esista. Per U_p si ha che:

$$U_p(x_1, \cdots, x_n, y) = \prod_{t=0}^{y} p(x_1, \cdots, x_n, y) \tag{2.26}$$

poiché in caso esista un predicato che restituisce 0 la produttoria verrebbe annullata.

Esempi

Grazie a questi quantificatori, è possibile caratterizzare altri predicati aritmetici come **ricorsive primitive**.

1. Predicato di divisibilità:

$$x|y \iff \exists z \leq y : x \cdot z = y \tag{2.27}$$

   È possibile in questo caso **limitare superiormente** la ricerca di z poiché per far si che $x|y$ sia vero, z deve essere minore o uguale a y.

2. Predicato numero primo:

$$\text{Primo}(x) \iff x > 1 \wedge \forall z \leq x : (\neg(z|x) \vee z = 1 \vee z = x) \tag{2.28}$$

   In questo modo esprimiamo proprio che x è primo *se e solo se* ha come divisori solo 1 e se stesso.

2.3.4 Minimalizzazione limitata

Un ulteriore operazione per cui è dimostrabile una proprietà di chiusura delle **classi PRC** è la **minimalizzazione limitata**.

Minimalizzazione limitata

Sia p un predicato $(n+1)$-ario, allora la funzione $(n+1)$-aria $\min_{t \leq y}$ è definita come:

$$\min_{t \leq y}(p(x_1, \cdots, x_n, y)) = \begin{cases} \min(Z_y) = \{t \leq y | p(x_1, \cdots, x_n, t)\}, & \exists t \leq y : p(x_1, \cdots, x_n, t) \\ 0, & \text{altrimenti} \end{cases} \tag{2.29}$$

Dove $\min(Z_y)$ è il minimo elemento dell'insieme Z_y, insieme dei valori $t \leq$ y per cui $p(x_1, \cdots, x_n, t)$ è vero. Questa funzione dunque, dato un predicato p e un **limite superiore** y restituisce, se esiste[5], il minimo $t \in \{0, \cdots, y\}$ per cui il predicato p è vero, altrimenti restituisce 0.

[5] È importante specificare nella definizione di minimalizzazione limitata i due casi possibili tramite il quantificatore esistenziale, poiché il **non esiste minimo dell'insieme vuoto**.

Teorema di chiusura per minimalizzazione limitata delle classi PRC

Enunciato. Sia p un predicato $(n+1)$-ario in $\mathcal{C}$ **classe PRC**, allora appartiene a $\mathcal{C}$ anche la funzione $(n+1)$-aria $\min_{t \le y}$ definita per minimalizzazione limitata da p.

Dimostrazione. Consideriamo la seguente funzione g:

$$g(x_1, \cdots, x_n, y) = \sum_{u=0}^{y} \prod_{t=0}^{u} \alpha(p(x_1, \cdots, x_n, t)) \tag{2.30}$$

Analizziamo g dall'interno. La funzione α agisce sui predicati come negazione. Dunque si avrà:

$$\alpha(p(x_1, \cdots, x_n, t)) = \begin{cases} 1, & \neg p(x_1, \cdots, x_n, t) \\ 0, & p(x_1, \cdots, x_n, t) \end{cases} \tag{2.31}$$

Inevitabilmente si avrà che la produttoria in g si annullerà nel caso in cui $p(x_1, \cdots, x_n, t)$ sia vera per qualche $t \le u$, ovvero:

$$\prod_{t=0}^{u} \alpha(p(x_1, \cdots, x_n, t)) = 1 \iff \forall t \le u, \neg p(x_1, \cdots, x_n, t) \tag{2.32}$$

Di conseguenza, iterando su u, otterremo che le varie **produttorie** varranno 1 fino a quando non si avrà il primo $u = t_0$ tale che $p(x_1, \cdots, x_n, u)$ risulti vera, e sarà nulla da quel momento in poi, essendoci l'iterazione t_0-esima ad annullarla.
Dovendo infine la **sommatoria esterna** fare esattamente t_0 passi prima di raggiungere tale u minimo, si avrà che $g(x_1, \cdots, x_n, y) = t_0$, che abbiamo dimostrato essere il **minimo cercato**.
Si avrà dunque che la funzione $g = \min Z_y \in \mathcal{C}$ essendo composizione di funzioni appartenenti a $\mathcal{C}$. Di conseguenza anche la funzione $\min_{t \le y} \in \mathcal{C}$, poiché **definita per casi**.

Esempi

Grazie alla minimalizzazione limitata è possibile definire altri predicati aritmetici come **ricorsive primitive**.

1. **Divisione con resto** $\left\lfloor \frac{x}{y} \right\rfloor$. Questo predicato è definibile come:

$$\left\lfloor \frac{x}{y} \right\rfloor = \min_{t \le x}((t+1)y > x) \tag{2.33}$$

   Infatti il minimo t per cui $(t+1)y > x$ è proprio il quoziente cercato, poiché per t più piccoli il prodotto con y sarà minore di x e dunque non potrà essere risultato della divisione.

2. **Operatore di resto** $x \mod y$. Questo predicato è definibile come:

$$x \mod y = x \dot{-} \left\lfloor \frac{x}{y} \right\rfloor y \tag{2.34}$$

3. **N-esimo primo.** Definiamo $P_0 = 0$ e sia per $n > 0, P_n$ l'n-esimo numero primo. Dimostriamo che questa funzione è **ricorsiva primitiva**. Consideriamo la funzione definita come:

$$P_{n+1} = \min_{t \le P_n!+1} (\text{Primo}(t) \wedge t > P_n) \tag{2.35}$$

   Questa restituisce il minimo numero primo maggiore di P_n, ovvero il successivo numero primo. Infatti si ha:

$$\forall i \in \{1, \cdots, n\}, \neg(P_i | (P_n! + 1)) \tag{2.36}$$

   poiché[a] $(P_n! + 1) \mod (P_i) = 1$, ovvero nessun primo $\le P_n$ divide $P_n! + 1$. Dunque qualsiasi fattore primo di $P_n! + 1$ sarà maggiore di P_n, e in particolare[b] si avrà $P_n! + 1 \ge P_{n+1}$.
   Definiamo dunque:

$$h(y, z) := \min_{t \le y}(\text{Primo}(t) \wedge t > y) \tag{2.37}$$

   questa è ricorsiva primitiva per il teorema di minimalizzazione limitata.
   Definiamo infine:

$$k(x) := h(x, x! + 1) \tag{2.38}$$

   Questa è ricorsiva primitiva per composizione di funzioni ricorsive primitive. Possiamo infine definire per ricorsione primitiva P_n come:

$$\begin{cases} P_0 = 0 \\ P_{n+1} = k(P_n) \end{cases} \tag{2.39}$$

[a] Si noti che $P_n! + 1 = (1 \cdot 2 \cdot \cdots \cdot P_i \cdot \cdots \cdot P_n) + 1$, dunque $\forall i \in \{1, \cdots, n\} : P_i | P_n!$, e dunque si ha che $(P_n! + 1) \mod (P_i) = 1$.

[b] Questo limite superiore che imponiamo è molto più grande di quanto necessario, è dimostrabile che $P_{n+1} \le 2P_n$. Si è scelto questo limite superiore essendo più facile da mostrare.

2.4 Funzioni parzialmente calcolabili

Abbiamo analizzato le **classi PRC** nelle loro proprietà poiché abbiamo dimostrato che la classe delle **funzioni calcolabili è PRC**.
Dal capitolo 1 però, sappiamo che le **funzioni calcolabili** non sono le uniche di cui si può valutare la calcolabilità, infatti esse sono un sotto insieme delle **funzioni parzialmente calcolabili**.
Queste però sono escluse dalle caratterizzazioni delle **classi PRC** poiché definite solo per funzioni totali.
Per ovviare a questa lacuna possiamo fornirci di un ulteriore operatore, quello di **minimalizzazione illimitata** che, come vedremo, ci permetterà di caratterizzare a pieno la classe delle **funzioni parzialmente calcolabili**.

2.4.1 Minimalizzazione illimitata

Minimalizzazione illimitata

Sia p un predicato $(n+1)$-ario, allora chiamiamo **minimalizzazione illimitata** la funzione $(n+1)$-aria $\min_t$ definita come:

$$\min_t \; p(x_1, \cdots, x_n, t) = \begin{cases} \min\{t \in \mathbb{N} | p(x_1, \cdots, x_n, t)\}, & \exists t \in \mathbb{N} : p(x_1, \cdots, x_n, t) \\ \uparrow, & \text{altrimenti} \end{cases} \tag{2.40}$$

Per far si che quest'operatore sia utile nella caratterizzazione delle **funzioni parzialmente calcolabili** è necessario dimostrare che esso stesso è **parzialmente calcolabile**.

Calcolabilità della Minimalizzazione illimitata

Enunciato. Se p un predicato calcolabile. Allora m **minimalizzazione illimitata** è parzialmente calcolabile.

Dimostrazione. Non potendo utilizzare le proprietà delle **classi PRC** ricorriamo al seguente S-programma:

```
[A] IF p(X_1, ···, X_n, Y) GOTO E
    Y ← Y + 1
    GOTO A
```

Codice 2.3: Minimalizzazione illimitata

Si avrà infatti che se $p(x_1, \cdots, x_n, t)$ è vera per qualche t, il programma terminerà restituendo il minimo t trovato, altrimenti il programma non terminerà mai.

2.4.2 Caratterizzazione delle funzioni parzialmente calcolabili

Grazie all'operatore appena definito, possiamo infine caratterizzare le **funzioni parzialmente calcolabili** grazie a un teorema che, come vedremo, è molto importante per la teoria della calcolabilità.

Caratterizzazione delle funzioni parzialmente calcolabili

Enunciato. Una funzione è parzialmente calcolabile *se e solo se* si può ottenere dalle funzioni iniziali mediante un numero finito di **composizioni**, **ricorsioni primitive** (applicata solo a eventuali funzioni totali ottenute) e **minimalizzazioni illimitate**.

Questo teorema, che non dimostreremo, insieme alla tesi di **Church-Turing**[6], che ci dice che ogni **funzione parzialmente calcolabile** è esprimibile mediante un **S-programma**, esprime un risultato teorico molto importante, ovvero che **ogni procedura algoritmica** può essere **ridotto alle funzioni iniziali** mediante questi **tre operatori**.
Per analizzare meglio questi programmi però, e farli valutare da delle funzioni, sarà necessario che anch'essi possano essere associati univocamente con dei numeri[7].

2.5 Esercizi e approfondimenti

Esercizio.

Consideriamo la funzione $|x - y| = x \dot{-} y + y \dot{-} x$. *Dimostrare che questa è ricorsiva primitiva utilizzando le funzioni iniziali.*

[6] Enunciata nella definizione 1.1
[7] Esercizi Consigliati: 2.50, 2.52 e 2.53

È possibile dimostrare che la funzione $|x - y|$ è ricorsiva primitiva mostrando che essa è composizione di funzioni iniziali. Si ha infatti che le funzioni **somma** e **differenza modificata** sono già state dimostrate essere ricorsive primitive.
Per quanto riguarda la funzione **differenza modificata** con operandi invertiti è sufficiente comporla con le funzioni proiezioni in modo da ottenere il risultato voluto, ovvero:

$$y \dot{-} x = d_m(y, x) = d_m(u_2^{(2)}(x, y), u_1^{(2)}(x, y)) = u_2^{(2)}(x, y) \dot{-} u_1^{(2)}(x, y) \tag{2.41}$$

Si avrà in questo modo che la **differenza modificata** riceverà gli operandi nell'ordine per cui è stata dimostrata essere ricorsiva primitiva.
Alternativamente possiamo dividere la funzione nei casi della definizione di funzione ottenuta per ricorsione primitiva:

$$\begin{cases} |x - 0| = x \dot{-} 0 + 0 \dot{-} x = x + 0 = x = u_1^{(1)}(x) \\ |x - (t + 1)| = p(|x - t|) = g(x, t, |x - t|), \forall t \in \mathbb{N}^* \end{cases} \tag{2.42}$$

Dove $g(x, y, z) = p(u_3^{(3)}(x, y, z))$ è composizione di funzioni iniziali.

Esercizio.

Consideriamo il predicato $p(x, y) \iff x \leq y$. Dimostrare che questo è ricorsivo primitivo. Consideriamo il predicato dato come:

$$p(x, y) = \begin{cases} 1, & x \leq y \\ 0, & \text{altrimenti} \end{cases} = \begin{cases} 1, & x \dot{-} y = 0 \\ 0, & \text{altrimenti} \end{cases} = \alpha(x \dot{-} y) \tag{2.43}$$

Dunque il predicato p è ricorsivo primitivo in quanto composizione di funzioni ricorsive primitive.

Approfondimento.

Dimostrazione alternativa del corollario 2.3.2.

Corollario al Teorema di Definizione per Casi

Dimostrazione. Dal teorema 2.3.1 sappiamo che tutti i *predicati composti* a partire da $p_1, \cdots, p_n$ usando i connettivi $\wedge, \vee, \neg$ appartengono a $\mathcal{C}$. Per ogni caso $i \in \{1, \cdots, k\}$ della definizione di f possiamo dunque definire un predicato p_i' come:

$$p_i'(x_1, \cdots, x_n) = \neg p_1 \wedge \cdots \wedge \neg p_{i-1} \wedge p_i \tag{2.44}$$

In oltre possiamo definire il caso $(k + 1)$-esimo come:

$$p_{k+1}'(x_1, \cdots, x_n) = \neg p_1 \wedge \cdots \wedge \neg p_{n-1} \wedge \neg p_n \tag{2.45}$$

Possiamo dunque definire la funzione f come:

$$f(x_1, \cdots, x_n) = g_1(x_1, \cdots, x_n)p_1'(x_1, \cdots, x_n) + \cdots + g_k(x_1, \cdots, x_n)p_k'(x_1, \cdots, x_n) + h(x_1, \cdots, x_n)p_{k+1}'(x_1, \cdots, x_n) \tag{2.46}$$

Essendo dunque f composizione di **funzioni ricorsive primitive** e **funzioni appartenenti** a $\mathcal{C}$ è **ricorsiva primitiva**.

Esercizio

*Sia f una funzione totale unaria. Definiamo le **iterate di** f come:*

$$f^{(0)}(x) = x, f^{(1)}(x) = f(x), \cdots, f^{(n+1)}(x) = f(f^{(n)}(x)), \forall n \in \mathbb{N} \tag{2.47}$$

Definiamo allora la funzione t_f binaria come:

$$t_f(x, n) = f^{(n)}(x) \tag{2.48}$$

*Mostrare che se $f \in \mathcal{C}$ **classe PRC**, allora anche $t_f \in \mathcal{C}$*

Dimostriamo per Ricorsione Primitiva

$$\begin{cases} t_f(x, 0) = x = u_1^{(2)}(x) \\ t_f(x, n + 1) = f(t_f(x, n)) = g(x, n, t_f(x, n)), \forall n \in \mathbb{N} \end{cases} \tag{2.49}$$

Dove $g(x, y, z) = f(u_3^{(3)}(x, y, z))$ è composizione di funzioni iniziali e appartenenti a $\mathcal{C}$ e dunque ricorsiva primitiva.

Esercizio:

Sia $\pi(x)$ il numero di primi $\leq x$. Mostrare che $\pi(x)$ è ricorsiva primitiva.

Consideriamo la funzione data come:

$$\pi(x) = \sum_{i=0}^{x} Primo(i) \tag{2.50}$$

Essendo dunque la funzione $\pi(x)$ ottenuta tramite **l'operatore sommatoria** applicato a un predicato ricorsivo primitivo, essa è ricorsiva primitiva.

Esercizio:

Sia $h(x) = \lfloor \sqrt{2}x \rfloor$, ovvero che restituisca $n \in \mathbb{N} : n \leq \sqrt{2}x < n + 1$. Mostrare che $h(x)$ è ricorsiva primitiva.

Essendo $\sqrt{2}$ un numero *irrazionale* non è compreso nel dominio delle funzioni che stiamo contemplando. Dobbiamo dunque limitarlo superiormente.

$$n \leq \sqrt{2}x < n + 1 \iff n^2 \leq 2x^2 < (n+1)^2 \tag{2.51}$$

Possiamo dunque considerare la funzione data come:

$$h(x) = \min_{t<2x}\left((t+1)^2 > 2x^2\right) \tag{2.52}$$

La funzione è dunque ricorsiva primitiva poiché ottenuta per **minimizzazione limitata** del predicato ricorsivo primitivo $(t+1)^2 > 2x^2$ composizione di funzioni ricorsive primitive.

Esercizio:

Sia $R(x, y)$ un predicato binario ricorsivo primitivo. Definiamo la massimalizzazione limitata $\max_{t \leq y} R(x, t)$, ponendo $g(x, y) = 0$ se $\neg\exists t \leq y : R(x, t)$. Dimostrare che $\max_{t \leq y} R(x, t)$ è ricorsiva primitiva.

Dimostrazione per Ricorsione Primitiva

Possiamo definire la funzione data per ricorsione primitiva come:

$$\begin{cases} \max_{t \leq 0} R(x, t) = g(x, 0) = 0 \\ \max_{t \leq y+1} R(x, t) = \max_{t \leq y} R(x, t) \cdot (\neg R(x, y+1)) + (y+1) \cdot (R(x, y+1)) = g(x, y, \max_{t \leq y} R(x, y)), \forall y \in \mathbb{N} \end{cases} \tag{2.53}$$

Dove $g(x, y, \max_{t \leq y} R(x, y))$ è una funzione ricorsiva primitiva poiché composta di funzioni e predicati ricorsivi primitivi.

Problema della Fermata e Funzione Universale

3.1 Concetti Introduttivi Generali

Fino ad adesso ci siamo concentrati sul concetto di calcolabilità e sulla descrizione delle proprietà delle funzioni calcolabili.

Per quanto siamo riusciti a ottenere ottimi risultati nel caratterizzarle, non siamo ancora riusciti a trarre conclusioni di carattere generale sulle funzioni calcolabili.

In questo capitolo procederemo a trattare due dei più importanti risultati dell'informatica teorica che vanno in questo senso, ovvero il **problema della fermata e l'esistenza di funzioni non calcolabili** e la **funzione universale**.

Il primo dei due risultati risponde alla domanda se **esistono funzioni che non possono essere risolti da un algoritmo**, mentre il secondo risponde alla domanda **se esiste un algoritmo che può risolvere ogni problema risolvibile**.

Per quanto possano sembrare formulazioni molto simili, andremo a dimostrare come nel primo caso otterremo un risultato **in senso negativo**, mentre nel secondo un risultato **in senso positivo**.

Prima di fare ciò però è fondamentale introdurre degli ultimi strumenti utili che ci permetteranno di dimostrare questi risultati, ovvero la **funzione di pairing** e i **numeri di Gödel**, che ci permetteranno di procedere alla **codifica dei programmi in numeri**, fondamentale per permettere a degli algoritmi di valutarli.

3.2 Funzione di Pairing e Numeri di Gödel

Prima di procedere a mostrare come codificare un programma con un numero, è necessario introdurre alcune funzioni che ci saranno utili per la codifica.

Nel dettaglio queste sono:

- **Funzione di pairing**, funzione biettiva che permette di codificare una coppia di numeri in un unico numero.
- **Numeri di Gödel**, funzione che permette di codificare una k-pla di numeri in un unico numero.

3.2.1 Funzione di Pairing

La **funzione di pairing** (detta anche volgarmente *angoletto*) è una funzione che permette di codificare una coppia di numeri in un unico numero.

Funzione di Pairing

Chiamiamo **funzione di Pairing** la funzione $\langle , \rangle : \mathbb{N}^2 \to \mathbb{N}$ definita come:

$$\langle x, y \rangle = 2^x(2y+1) \dot{-} 1 \tag{3.1}$$

La proprietà più evidente di questa funzione è proprio che è **ricorsiva primitive**, infatti è una composizione di funzioni ricorsive primitive viste fino a ora. Oltre a ciò però, la **funzione di pairing** gode di altre due proprietà fondamentali che ci saranno fondamentali nella codifica, la **biettività** e la **ricorsione primitiva** delle **inverse parziali**.

Biettività della funzione di Pairing

Enunciato. La **funzione di Pairing** è **biettiva**.

Dimostrazione. Innanzitutto possiamo ricondurci alla sua forma completa. L'operatore $\dot{-}$ è infatti sostituibile con la normale **sottrazione**, poiché $2^x(2y+1) \geq 1$.
Riscriviamo dunque la **funzione di Pairing** come $\langle x, y \rangle = 2^x(2y+1) - 1$.
Possiamo ora notare che:

$$\langle x, y \rangle = 2^x(2y+1) - 1 \iff \langle x, y \rangle + 1 = 2^x(2y+1) \tag{3.2}$$

Dunque, con un $z \in \mathbb{N}$ e come scegliendo x l'esponente della maggiore potenza di 2 che divide $z+1$, avremo che $\frac{z+1}{2^x}$ è necessariamente dispari e dunque si scrive come $2y+1$ per un unico $y \in \mathbb{N}$, ovvero:

$$\forall z \in \mathbb{N} : \exists! x, y \in \mathbb{N} : z + 1 = 2^x(2y+1) \implies \forall z \in \mathbb{N} : \exists! x, y \in \mathbb{N} : z = \langle x, y \rangle \tag{3.3}$$

Dalla **biettività** della **funzione di pairing** sappiamo che deve esistere una sua **inversa**.
Stando trattando esclusivamente funzioni $\mathbb{N}^k \to \mathbb{N}, k \in \mathbb{N}$, non è possibile considerare propriamente **l'inversa della funzione di Pairing**, poiché questa è una funzione del tipo $\mathbb{N} \to \mathbb{N}^2$.
Per ovviare a questo problema, definiamo l'**inversa parziale sinistra** l (*duale per la destra* r) della **funzione di pairing**:

$$\begin{aligned} &l : \mathbb{N} \to \mathbb{N} \\ &l(z) \mapsto \min_{x \leq z}(\exists y \leq z : z = \langle x, y \rangle) \end{aligned} \tag{3.4}$$

Possiamo dunque subito notare che anche queste due funzioni sono **ricorsive primitive**, infatti definite tramite **minimalizzazione limitata**.
In maniera più pratica, è possibile calcolare i valore di $l(x)$ e $r(x)$ seguendo il processo utilizzato per dimostrare la biettività della **funzione di pairing**.

Infatti per determinare $l(x)$, dobbiamo trovare la più grande potenza di 2 che divide $x+1$, per poi sfruttare il risultato ottenuto per determinare y dalla definizione della **funzione di pairing**.

Esempi

1. Consideriamo la coppia di numeri $(2,3)$. Avremo che:

$$\langle 2,3\rangle = 2^2(2\cdot 3+1)-1 = 2^2(7)-1 = 27 \tag{3.5}$$

   Dunque la coppia ordinata $(2,3)$ è univocamente codificata dal numero 27.
2. Consideriamo la terna di numeri $(1,0,2)$. Avremo che:

$$\langle 1,\langle 0,2\rangle\rangle = \left\langle 1, 2^0(2\cdot 2+1)-1\right\rangle = \langle 1,4\rangle = 2^1(2\cdot 4+1)-1 = 17 \tag{3.6}$$

   Dunque la terna ordinata $(1,0,2)$ è univocamente codificata dal numero 17. In generale è possibile codificare una qualsiasi k-upla di elementi combinando più volte la **funzione di pairing**.
3. Consideriamo il numero $z = 23$. Per determinare $l(z)$ dobbiamo trovare la più grande potenza di 2 che divide $z+1$, ovvero:

$$l(23) = x \iff 2^x|24 \wedge \neg(2^{x+1}|24) \implies x = 3 \tag{3.7}$$

   Dunque $l(23) = 3$. Per determinare $r(23)$ dobbiamo invece sfruttare il risultato ottenuto per $l(23)$ e invertire le operazioni usate nella **funzione di pairing**, ovvero:

$$r(23) = y \iff 23+1 = 2^3(2y+1) \iff 24 = 2^3(2y+1) \iff 3 = 2y+1 \iff y = 1 \tag{3.8}$$

3.2.2 Numeri di Gödel

I **Numeri di Gödel** sono una classe di funzioni che permettono di codificare una k-pla di numeri in un unico numero[1].

Numeri di Gödel

Sia $a_1,\cdots,a_k \in \mathbb{N}$ e P_n la funzione **N-esimo primo**, allora definiamo come **Numero di Gödel** della k-pla come:

$$[a_1,\cdots,a_k] = \prod_{i=1}^{k} P_i^{a_i} \tag{3.9}$$

È importante i **Numeri di Gödel** così definiti non rappresentano una funzione, bensì una classe di funzioni poiché esiste una funzione $\forall k \in \mathbb{N}$. Inoltre, ognuna di queste funzioni è ricorsiva primitiva poiché, $\forall k \in \mathbb{N}$, $[a_1,\cdots,a_k]$ è ottenuta tramite **l'operatore produttoria** applicato a **funzioni ricorsive primitive**[2].

Possiamo però considerare questa classe di funzioni come un unica funzione definita come:

$$\begin{aligned} [\,] &: \mathbb{N}^0 \cup \mathbb{N}^1 \cup \mathbb{N}^2 \cup \cdots \to \mathbb{N} \\ [x_1,\cdots,x_k]_k &\longmapsto \prod_{i=1}^{k} P_i^{x_i} \end{aligned} \tag{3.10}$$

Inoltre essendo ogni numero elevato a 0 uguale a 1, possiamo definire anche $[\,]_0 = 1$.
A differenza della **funzione di pairing** però questa funzione non è biettiva. In particolare non è suriettiva poiché non esiste una k-pla di numeri che codifica il numero 0 e non è iniettiva poiché se prendiamo una qualsiasi k-pla e aggiungiamo a destra degli 0, otterremo sempre lo stesso numero di Gödel.

Ma dal teorema fondamentale dell'aritmetica sappiamo che, presi $n,m \in \mathbb{N}$:

$$n \le m \wedge [a_1,\cdots,a_n]_n = [b_1,\cdots,b_m]_m \implies a_i = b_i, \forall i \in \{1,\cdots,n\} \wedge b_j = 0, \forall j \in \{n+1,\cdots,m\} \tag{3.11}$$

Ovvero che ogni numero naturale è **unicamente fattorizzabile** in numeri primi, se si prescinde dall'ordine dei fattori e dalla presenza di esponenti nulli aggiunti a destra.

Sfruttando questo risultato possiamo definire delle **inverse parziali** per la funzione **Numeri di Gödel**.
Una prima **inversa parziale** che possiamo definire è la funzione che, dato un numero da fattorizzare x e un indice i che indichi l'i-esimo primo, restituisca l'esponente di P_i nella fattorizzazione di x.
Possiamo definire questa funzione come:

$$(x)_i = \min_{t\le x}(\neg(P_i^{t+1}|x)) \tag{3.12}$$

[1] Nella definizione della funzione **Numeri di Gödel** è utilizzata la funzione P_n che dato un $n \in \mathbb{N}$ restituisce l'n-esimo **numero primo**. Questa è ricorsiva primitiva, come dimostrato nell'esercizio 2.35

[2] L'argomento della produttoria è infatti $P_i^{a_i}$, composizione della funzione P_n dimostrata come ricorsiva primitiva nell'esempio 2.35 e della funzione x^y mostrata come calcolabile nell'esercizio 1.16 e facilmente dimostrabile come ricorsiva primitiva.

In particolare questa funzione verifica per un dato primo P_i quale sia il **minimo esponente** t tale che P_i^{t+1} non divida x, ovvero il massimo esponente t tale che P_i^t divida x.

Nel caso $(0)_i, \forall i \in \mathbb{N}$, la funzione restituisce 0 poiché, essendo che ogni numero divide 0, non esiste un minimo, e dunque per definizione di minimalizzazione limitata la funzione minimo esponente restituirà 0.
Questo valore però è restituito anche per ogni i tale che P_i non divide x, poiché in questo l'esponente t tale che P_i^t divida x è proprio 0 e nel caso $(1)_i, \forall i \in \mathbb{N}$, poiché si ha $1 = P_i^0, \forall i \in \mathbb{N}$.
Questa funzione inoltre è chiaramente **ricorsiva primitiva** per **minimalizzazione limitata**.

Un'altra **inversa parziale** che possiamo definire per la funzione **Numeri di Gödel** è la funzione che, dato un numero da fattorizzare x restituisca la lunghezza della minima k-pla tale che $[a_1, \cdots, a_k]_k = x$. In particolare la definiamo come:

$$Lt(x) = \min_{i \leq x}((x)_i \neq 0 \wedge \forall j \leq x : j \leq i \vee (x)_j = 0) \tag{3.13}$$

In particolare questa funzione restituisce il massimo indice che non annulla l'**inversa parziale** definita prima, ovvero l'indice del massimo primo che divide x.
Possiamo dunque andare a mostrare come queste funzioni appena definite, composte con la funzione **Numeri di Gödel**, siano composizioni di proiezioni con l'identica.

$$\begin{gathered}([a_1, \cdots, a_n]_n)_i = \begin{cases} a_i, & \text{se } i \leq n \\ 0, & \text{altrimenti} \end{cases} \\ [(x)_1, \cdots, (x)_n]_n = x \text{ se } n \geq Lt(x)\end{gathered} \tag{3.14}$$

Esempi

1. Consideriamo la sequenza di lunghezza $k = 3$ $[2, 0, 1]$. Avremo che:

$$[2, 0, 1]_3 = P_1^2 \cdot P_2^0 \cdot P_3^1 = 2^2 \cdot 3^0 \cdot 5^1 = 20 \tag{3.15}$$

2. Consideriamo il numero $x = 30$ e l'indice $i = 2$. Avremo che:

$$(30)_2 = y \iff P_2^y | x \wedge \neg(P_2^{y+1} | x) \iff 3^y | 30 \iff y = 1 \tag{3.16}$$

In maniera più pratica possiamo scomporre in fattori primi x e valutare l'esponente dell'i-esimo primo, ovvero:

$$30 = 2^1 \cdot 3^1 \cdot 5^1 \implies (30)_2 = 1 \tag{3.17}$$

3. Consideriamo il numero $x = 25$. Avremo che:

$$Lt(25) = i \iff \left((25)_i \neq 0 \wedge \forall y > i : (25)_y = 0\right) \iff i = 3 \tag{3.18}$$

In maniera più pratica possiamo scomporre in fattori primi x e valutare l'indice del massimo primo che divide x, ovvero:

$$25 = 2^0 \cdot 3^0 \cdot 5^2 \implies Lt(25) = 3 \tag{3.19}$$

3.3 Codifica d'Istruzioni, Programmi e Stati

Definite dunque queste funzioni, possiamo finalmente passare alla codifica d'istruzioni del linguaggio S, i programmi e gli stati.

3.3.1 Codifica d'Istruzioni

Per questa codifica è necessario innanzi tutto assegnare a ogni variabile un numero naturale, in modo da poterle codificare.

Cominciamo ordinando le variabili seguendo lo schema `Y,X1,Z1,X2,Z2, ...` e assegniamo a ogni variabile un numero naturale in base alla sua posizione. In particolare assegniamo a `Y` il numero 1, a `X1` il numero 2, a `Z1` il numero 3, a `X2` il numero 4 ect.[3].
Procediamo poi con la stessa logica per le etichette `A1,B1,C1,D2,E2,A2,B2, ...`, in modo da codificare l'assenza di etichette col numero 0. Infine codifichiamo il tipo d'istruzione con un numero naturale, in particolare:

- 0 se è un'istruzione pigra.
- 1 se è incremento.
- 2 se è decremento.
- $n + 2$ se è di salto alla n-esima etichetta.

[3] Il motivo per cui la numerazione delle variabili parte da 1 invece che da 0 sarà più chiaro in seguito.

A questo punto possiamo procedere con la codifica d'istruzioni sfruttando la **funzione di pairing**.

Codifica Istruzioni Linguaggio S

Ogni istruzione I del linguaggio S è codificabile univocamente dal numero $\#(I) = \langle a, \langle b, c\rangle\rangle$ dove:
- a è il numero di etichetta dell'istruzione o 0 se non ha etichetta.
- b è il numero del tipo d'istruzione che stiamo codificando.
- c è il numero dell'unica variabile che compare nell'istruzione diminuito di 1.

Grazie alle proprietà della funzione pairing possiamo notare che ogni istruzione è codificata da un numero naturale, e che ogni numero naturale codifica un'istruzione. Dalla biettività della **funzione di pairing**, questa operazione è reversibile.

Esempi

1. Consideriamo le seguenti istruzioni:
   - `[A] X ← X + 1`
     Questa istruzione è codificata dalla terna $(1, 1, 1)$ poiché abbiamo l'etichetta 1 = `A`, il tipo di istruzione 1, ovvero incremento, e la variabile 1 =`X`. Possiamo dunque codificare questa istruzione col numero:
     $$\langle 1, \langle 1, 1\rangle\rangle = \left\langle 1, 2^1(2 \cdot 1 + 1) - 1\right\rangle = \langle 1, 5\rangle = 2^1(2 \cdot 5 + 1) - 1 = 21 \tag{3.20}$$
   - `IF Z1 ≠ 0 GOTO E`
     Questa istruzione è codificata dalla terna $(0, 7, 2)$ poiché non abbiamo etichetta, il tipo d'istruzione è di salto all'etichetta 5, ovvero di numero $5 + 2 = 7$, e la variabile 2 =`Z`. Possiamo dunque codificare questa istruzione col numero:
     $$\langle 0, \langle 7, 2\rangle\rangle = \left\langle 1, 2^7(2 \cdot 2 + 1) - 1\right\rangle = \langle 0, 639\rangle = 2^0(2 \cdot 639 + 1) - 1 = 1278 \tag{3.21}$$
2. Viceversa, consideriamo il numero 23.
   Per ricavare l'equazione associata a questo numero, dobbiamo risolvere la seguente equazione:
   $$23 = \langle a, \langle b, c\rangle\rangle \iff a = l(23) \tag{3.22}$$
   Da quanto visto nella sezione precedente, abbiamo che:
   $$a = l(23) = 3 \implies (24 = 2^3(2 \cdot 1 + 1)) \implies r(23) = \langle b, c\rangle = 1 \tag{3.23}$$
   Iterando il ragionamento, abbiamo che:
   $$b = l(1) = 1 \implies (2 = 2^1(2 \cdot 0 + 1)) \implies r(1) = c = 0 \tag{3.24}$$
   Dunque il numero 23 codifica l'istruzione con etichetta 3, ovvero C, l'istruzione 1, ovvero incremento, e la variabile 0, ovvero `Y`. Avremo dunque l'istruzione: `[C] Y ← Y +1`

3.3.2 Codifica di Programmi

Se per la codifica delle istruzioni è stato sufficiente utilizzare esclusivamente la **funzione di pairing**, per la codifica dei programmi sarà necessario usare i risultati di queste codifiche come valori d'input per i **Numeri di Gödel**.

Codifica di S-Programmi

Sia $\mathcal{P}$ un S-Programma avente istruzioni $I_1, \cdots, I_k$ con k lunghezza di $\mathcal{P}$, allora $\#(\mathcal{P}) = [\#(I_1), \cdots, \#(I_k)]_k - 1$.

Sottrarre 1 è necessario per risolvere la non suriettività dei **Numeri di Gödel**. Infatti in questo modo anche il valore 0 è incluso nell'immagine della funzione, e dunque ogni numero naturale codifica un programma.

Per quanto concerne l'iniettività, possiamo restringere la definizione di S-Programma in modo che non possa contenere come ultima istruzione l'istruzione pigra sulla `Y`.

In questo modo, ogni programma avrà $Lt(\#(\mathcal{P})) = k$, poiché essendo l'istruzione `Y ← Y + 1` l'unica codificata col numero 0, avremo sempre che il k-esimo valore dei **Numeri di Gödel** sarà non nullo.

Inoltre grazie a queste restrizioni possiamo distinguere anche il programma vuoto, poiché questo avrà $Lt(\#(\mathcal{P})) = 0$ e dunque sarà codificato dal numero $[\,] - 1 = 0$. In questo modo si avrà che ogni programma è codificato in un unico numero naturale,

ovvero avremo una corrispondenza biunivoca tra programmi e numeri naturali[4].

[4] Esercizi consigliati: 3.7, 3.8, 3.9.

3.3.3 Codifica degli Stati

Per poter infine **codificare gli stati** di un programma è sufficiente utilizzare i **Numeri di Gödel**.

Codifica di Stati

Sia $\sigma = \{V_1 = n_1, V_2 = n_2, \cdots, V_k = n_k\}$ uno stato di un programma $\mathcal{P}$ avente variabili $V_1, \cdots, V_k$ con k lunghezza di $\mathcal{P}$, allora:

$$\#(\sigma) = \prod_{i=1}^{k} P_{\#(V_i)}^{n_i} = [\cdots, n_i, \cdots] \tag{3.25}$$

Dove n_i si trova nella posizione $\#(V_i)$ dei **Numeri di Gödel**.

Esempi

Consideriamo lo stato $\sigma = \{Y = 1, X_1 = 0, X_2 = 1\}$.
Questo stato si codifica come $[1, 0, 0, 1] = 14$, poiché tutte le variabili temporanee sono inizializzate a 0.

3.4 Funzioni parziali non calcolabili e Halt Problem

Fino a ora abbiamo incontrato solo funzioni parzialmente calcolabili, ma ancora non sappiamo se **tutte** le **funzioni parziali** sono **calcolabili** o se esistono **funzioni parziali non calcolabili**.

Delle due è vera la seconda affermazione, che andremo a dimostrare in due modi diversi. Iniziamo da un approccio prima di tipo generale, che sfrutta **l'argomento diagonale di Cantor**[5].

Delle funzioni che vedremo sarà ricorrente il parametro "y" che altro non è che la codifica (vista nelle sezioni precedenti) del programma che stiamo considerando. Se ad esempio stiamo considerando il programma che calcola la funzione somma, y sarà la codifica delle istruzioni del programma S che calcolerà la somma tra x1 e x2.

Esistenza di funzioni non parzialmente calcolabili

Enunciato. Esistono **funzioni parziali non calcolabili**.

Dimostrazione. $\forall n \in \mathbb{N}$, sia $\mathcal{P}_n$ il programma tale che $\#(\mathcal{P}_n) = n$.
Consideriamo la seguente matrice:

$$\begin{pmatrix} \Psi_{\mathcal{P}_0}^{(1)}(0) & \Psi_{\mathcal{P}_0}^{(1)}(1) & \Psi_{\mathcal{P}_0}^{(1)}(2) & \cdots \\ \Psi_{\mathcal{P}_1}^{(1)}(0) & \Psi_{\mathcal{P}_1}^{(1)}(1) & \cdots & \cdots \\ \Psi_{\mathcal{P}_2}^{(1)}(0) & \cdots & \cdots & \cdots \\ \vdots & \vdots & \vdots & \ddots \end{pmatrix} \tag{3.26}$$

Definiamo ora la funzione g come:

$$g(n) = \begin{cases} \Psi_{\mathcal{P}_n}^{(1)}(n) + 1, & \Psi_{\mathcal{P}_n}^{(1)}(n) \downarrow \\ 0, & \text{altrimenti} \end{cases} \tag{3.27}$$

Supponiamo ora **per assurdo** che g sia **calcolabile**. Allora $\exists m \in \mathbb{N} : g = \Psi_{\mathcal{P}_m}^{(1)}$.
Ma calcolando $g(m)$ utilizzando la sua definizione otteniamo, nel caso $\Psi_{\mathcal{P}_m}^{(1)}(m) \uparrow$, che $g(m) = 0$ e $\Psi_{\mathcal{P}_m}^{(1)}(m)$ non termina, mentre nel caso $\Psi_{\mathcal{P}_m}^{(1)}(m) \downarrow$:

$$\Psi_{\mathcal{P}_m}^{(1)}(m) + 1 = g(m) = \Psi_{\mathcal{P}_m}^{(1)}(m), \quad \textbf{assurdo!} \tag{3.28}$$

Si ha dunque $\nexists m \in \mathbb{N} : g = \Psi_{\mathcal{P}_m}^{(1)}$ e dunque g **non è parzialmente calcolabile**.

La proprietà di esistenza di funzioni parziali non calcolabili è molto importante, poiché ci permette di dimostrare che esistono problemi che non possono essere risolti da un algoritmo.

[5] **L'argomento diagonale di Cantor** è la metodologia di dimostrazione usata da **Cantor** per dimostrare che l'insieme $\mathbb{R}$ ha cardinalità strettamente maggiore dell'insieme $\mathbb{N}$.

Nello specifico vediamo una riconferma della proprietà appena dimostrata, attraverso un famoso problema indecidibile, il **problema della fermata**.

Problema della fermata

Enunciato. Per ogni $k \in \mathbb{N}^*$, non è calcolabile il predicato:

$$HALT^{(k)}(x_1, x_2, \cdots, x_k, y) \iff \Psi^{(k)}_{\mathcal{P}_y}(x_1, x_2, \cdots, x_k) \downarrow \tag{3.29}$$

Dimostrazione[a]**. Per assurdo**, sia $HALT$ calcolabile. Potremmo allora scrivere il programma seguente:

```
1      [A] IF HALT(X,X) GOTO A
```

Codice 3.1: Dimotrazione Problema della Fermata

Sia y_0 il numero di questo programma. Allora:

$$\Psi^{(1)}_{\mathcal{P}_{y_0}}(x) = \begin{cases} 0, & \Psi^{(1)}_{\mathcal{P}_x}(x) \uparrow \iff \neg HALT(X, X) \\ \uparrow, & \Psi^{(1)}_{\mathcal{P}_x}(x) \downarrow \iff HALT(X, X) \end{cases} \tag{3.30}$$

Dunque si avrà che $\forall x \in \mathbb{N}, HALT(x, y_0) \iff \neg HALT(x, x)$. Per $x = y_0$ ciò porta all'assurdo, dimostrando che $HALT$ **non è calcolabile**.

[a] Per brevità considereremo solo il caso $k = 1$. Gli altri casi sono dimostrabili in modo analogo. Useremo in oltre la notazione $HALT$ per intendere $HALT^{(1)}$.

Avremo quindi che dal **problema della fermata** che non è possibile trovare un algoritmo generale che permetta di determinare, in **tempo finito**, se un dato programma termina su un certo input.

Aver dimostrato che **esistono funzioni parziali non calcolabili** è molto importante, poiché, alla luce della tesi di **Church-Turing**, ci permette di dimostrare che esistono problemi che non possono essere risolti da un algoritmo. Ciò non significa che non esistano problemi di cui è possibile determinare il risultato a priori, ma che piuttosto ne esistano alcuni per cui non è possibile determinarlo. Il **problema della fermata** è solo un esempio di **problema indecidibile**, ma ne esistono molti altri. Consideriamo ad esempio una congettura matematica ancora non dimostrata, come la **congettura di Goldbach**. La **congettura di Goldbach** afferma che ogni numero pari maggiore di 4 è somma di due numeri primi. A oggi la congettura non è ancora stata dimostrata. Possiamo però facilmente esprimere questa congettura col predicato:

$$SumP(n) \iff \exists x \leq n(\exists y \leq n(Primo(x) \wedge Primo(y) \wedge x + y = n)) \tag{3.31}$$

Di conseguenza il seguente programma è indecidibile, ovvero non è possibile stabilire se terminerà e su quali input:

```
1      Y ← min_n(2|n ∧ n ≥ 4 ∧ ¬SumP(n))
```

3.5 Funzione universale e Predicato STP

Nonostante la prima asserzione generale fatta sulla calcolabilità dei programmi si sia dimostrata come falsa, è possibile ancora dimostrare che esiste una funzione che è calcolabile per ogni programma. Questo risultato, oltre a essere di grande importanza poiché ci dimostrerà che ogni programma può essere simulato utilizzandone un altro, ci permetterà di definire una *versione ristretta* del predicato $HALT$ che sarà calcolabile.

3.5.1 Funzione universale

Funzione universale (N+1)-aria

Sia $\Phi^{(n)}$ la funzione definita come $\Phi^{(n)}(x_1, \cdots, x_n, y) = \Psi^{(n)}_{\mathcal{P}_y}(x_1, \cdots, x_n)$, dove $\mathcal{P}_y$ è il programma di numero y. Questa funzione è detta **funzione universale** $(n+1)$**-aria**.

È importante notare innanzitutto che questa *funzione*, come la *funzione* **Numeri di Gödel**, rappresenti in realtà una **classe di funzioni**, una per ogni $n \in \mathbb{N}^*$.

Questa funzione si può dimostrare come parzialmente calcolabile. Questa cosa non è banale poiché la funzione universale non dipende solo dalle variabili $x_1, \ldots, x_n$ ma anche dal programma y. Equivale essenzialmente a una definizione per casi **per infiniti casi**.

In particolare è importante notare che $\Phi^{(n)}(x_1, \cdots, x_n, y) \uparrow \iff \Psi_{\mathcal{P}_y}^{(n)(x_1,\cdots,x_n)} \uparrow$. Dunque a differenza della funzione *HALT* non abbiamo necessità che sia sempre definita. Anzi nei casi critici in cui il programma scelto dalla variabile y non termina sono tutti e soli i casi in cui questa funzione non termina.

Teorema di Universalità

Enunciato. $\forall n \in \mathbb{N}^*, \Phi^{(n)}$ è **parzialmente calcolabile**.

Dimostrazione. Per dimostrare questo teorema andremo a scrivere un S-Programma che calcoli la funzione universale. Analizzeremo le singole istruzioni una a una prima di mostrare il programma completo, per renderle più comprensibili[a]. Iniziamo col salvare in `Z` il valore di X_{n+1} che conserva il programma da calcolare, in una variabile K un contatore per l'istruzione attuale, partendo da 1 e in una variabile S inseriamo lo stato iniziale. Per farlo basterà usare la codifica degli stati ignorando tutti i posti dispari poiché le variabili di input si trovano solo in posti pari. Avremo quindi:

```
1    Z ← X_{n+1} + 1
2    K ← 1
3    S ← ∏_{i=1}^{n} P_{2i}^{x_i}
```

Andiamo ora a iniziare il ciclo principale del programma che scorrerà lungo tutte le istruzioni del programma da calcolare. Per farlo andiamo a definire un'etichetta `C` che ci permetta di tornare al ciclo principale e un'etichetta `F` che ci permetta di terminare il programma. La condizione di uscita dal ciclo principale sarà che il contatore K sia maggiore del numero di istruzioni del programma da calcolare, ovvero $Lt(Z)+1$, o che sia uguale a 0, nel caso di salto a un'etichetta inesistente.

```
1    [C] IF (K == Lt(Z) + 1 ∨ K==0) GOTO F
```

Iniziamo ora con l'analisi delle istruzioni. In primo luogo andiamo a salvare in `U` l'istruzione senza etichetta corrente, ovvero l'istruzione di numero K del programma da calcolare. Per farlo basterà usare le inverse parziali dei **Numeri di Gödel** e **funzione di pairing**, ovvero:

```
1    U ← r((Z)_K)
```

Dobbiamo ora valutare il tipo di istruzione che abbiamo appena salvato. Quest'ultimo è accessibile calcolando $l(U)$. Da ora in poi ci ricondurremo al caso più difficile, ovvero quello di salto, utilizzando il costrutto `IF`, saltando a parti del programma preposte a risolvere gli altri casi se ci trovassimo con istruzioni di altro tipo.
In primo luogo nel caso avessimo un istruzione pigra, ovvero $l(U) = 0$, andremo a N, ovvero alla parte del programma che si occupa esclusivamente di incrementare il contatore K e tornare al ciclo principale. Avremo quindi:

```
1    IF (l(U) == 0) GOTO N
```

In tutti gli altri casi sarà necessario interagire con la variabile presente nell'istruzione. Per ricavarla sarà sufficiente calcolare il numero primo di indice $r(U)$ che corrisponde proprio alla variabile in questione, ovvero:

```
1    P ← P_{r(U)}
```

Nel caso avessimo un'istruzione di incremento, ovvero $l(U) = 1$, andremo a A, ovvero alla parte del programma che si occupa di incrementare la variabile P e tornare al ciclo principale. Avremo quindi:

```
1    IF (l(U) == 1) GOTO A
```

Abbiamo poi la necessità di un altro salto a N che comprenda due casi, ovvero il caso di decremento su variabile nulla o di non verifica della condizione di un salto. Entrambi i casi sono accomunati dal fatto che la variabile presente nell'istruzione dev'essere 0. Per verificare che sia questo il caso sarà sufficiente controllare se il numero primo associato alla variabile in questione non divida lo stato. In tal caso infatti avrà nella fattorizzazione esponente 0. Avremo che:

```
1    IF ¬(P|S) GOTO N
```

Nel caso avessimo un'istruzione di decremento, ovvero $l(U) = 2$, andremo a M, ovvero alla parte del programma che si occupa di decrementare la variabile P e tornare al ciclo principale. Avremo quindi:

```
1    IF (l(U) == 2) GOTO M
```

Avendo dunque escluso ogni altro caso, possiamo dedicarci a pieno all'istruzione di salto. Questa deve spostare il contatore dell'istruzione attuale alla prima istruzione con etichetta $l(U)-2$ se presente. Per farlo sarà sufficiente calcolare il minimo numero di istruzione tale che l'etichetta di quell'istruzione $l((Z)_i)$ sia uguale a $l(U)-2$. Possiamo fare questo utilizzando al minimalizzazione limitata. Avremo dunque:

```
1    K ← min_{i≤Lt(Z)}(l(U) = l((Z)_i) + 2)
```

La minimalizzazione limitata, per com'è stata definita in 2.3.4, ci garantisce che se non esiste un'istruzione con etichetta $l(U)-2$ il contatore K sarà uguale a 0 e incontreremo dunque una condizione di terminazione. Inoltre dalla definizione di minimalizzazione limitata il conteggio di i parte da 0. Sebbene questo possa sembrare un problema non esistendo l'etichetta 0, in realtà avendo definito $(Z)_0 = 0$ avremo che $l((Z)_0) + 2 = 2$, che non sarà mai uguale a $l(U)$ essendo 2 associato al tipo di istruzione di decremento.
Aggiornato il contatore K andremo a C, ovvero alla parte del programma che si occupa di valutare l'istruzione successiva.

```
1    GOTO C
```

[a] Per praticità e leggibilità verranno utilizzati nomi di variabili ed etichette non contemplati dal linguaggio S. Si può ovviamente riscrivere il programma sostituendo una a una le variabili e le etichette con quelle del linguaggio S ottenendo un risultato analogo.

Teorema di Universalità

Andiamo ora ad analizzare i casi relativi alle altre istruzioni e la chiusura del programma.

A In questo caso abbiamo un'istruzione di incremento. Per incrementare la variabile associata al primo P nello stato sarà sufficiente moltiplicare lo stato per P. In questo modo infatti andremo ad aumentare l'esponente di P nella fattorizzazione di S, che esprime il valore della variabile associata a P, di 1. Avremo quindi:

```
[A] S ← S * P
    GOTO N
```

M In questo caso abbiamo un'istruzione di decremento. Dualmente a quanto abbiamo visto per l'incremento, è sufficiente per il decremento dividere lo stato per P. La divisione sarà intera poiché giungeremo a queste istruzioni mediante il controllo fatto in precedenza che assicura che P divida S. Avremo quindi:

```
[M] S ← ⌊S/P⌋
```

In questo caso possiamo omettere il salto a N poiché N sarà la prossima istruzione da eseguire comunque.

N È possibile arrivare a questa istruzione in diversi modi durante l'esecuzione, ma tutti sono accomunati dal dover semplicemente passare all'operazione successiva. Basterà dunque incrementare il contatore K e tornare al ciclo principale. Avremo quindi:

```
[N] K ← K + 1
    GOTO C
```

F Infine, terminiamo il programma estraendo il valore della variabile di output conservata in $(S)_1$, per porlo in Y. Avremo quindi:

```
[F] Y ← (S)_1
```

Vediamo dunque infine il programma completo:

```
    Z ← X_{n+1} + 1
    K ← 1
    S ← ∏_{i=0}^{n} P_{2i+1}^{x_i}
[C] IF (K = Lt(Z) + 1 ∨ K=0) GOTO F
    U ← r((Z)_K)
    IF (l(U) = 0) GOTO N
    P ← P_{r(U)}
    IF (l(U) = 1) GOTO A
    IF ¬(P|S) GOTO N
    IF l(U) = 2 GOTO M
    K ← min_{i≤Lt(Z)}(l(U) = l((Z)_i)) + 2
    GOTO C
[A] S ← S * P
    GOTO N
[M] S ← ⌊S/P⌋
[N] K ← K + 1
    GOTO C
[F] Y ← (S)_1
```

Codice 3.2: Dimostrazione Programma Universale

3.5.2 Predicato STP

Avendo dimostrato che la funzione universale è parzialmente calcolabile, possiamo definire un predicato che si comporti in maniera simile al predicato $HALT$, ma che sia calcolabile inserendogli delle limitazioni.

Predicato STP (N+2)-ario

$\forall n \in \mathbb{N}^*$ chiamiamo **predicato STP** $(n+2)$**-ario** il predicato:

$$STP^{(n)}(x_1, \cdots, x_n, y, t) \iff \text{il programma } \mathcal{P}_y \text{ su input } (x_1, \cdots, x_n) \text{ termina in al più } t \text{ passi.} \tag{3.32}$$

Possiamo rapidamente dimostrare come questo predicato sia calcolabile. È infatti sufficiente aggiungere un contatore al programma visto nella 3.5.1 che termini il programma nel caso si raggiungano i t passi. È possibile inoltre dimostrare che questo predicato è **ricorsivo primitivo**, ma eviteremo la dimostrazione essendo particolarmente ostica.
Abbiamo dunque ottenuto due fondamentali risultati. Il primo è che esiste una funzione che possa simulare ogni programma dato, e il secondo e che, per quanto non si possa determinare in assoluto se un programma termini o meno, è possibile determinare se termini in un certo numero di passi.

Possiamo finalmente dunque trarre le conclusioni sulla calcolabilità delle funzioni parziali. Abbiamo infatti dimostrato che esistono funzioni parziali non calcolabili, ma che esiste anche una funzione universale che permette di simulare ogni programma dato. Questo ci permette di tracciare un diagramma riassuntivo sulle funzioni parziali che comprenda tutti i sottoinsiemi delle funzioni parziali ed evidenzi in che relazione sono fra loro.

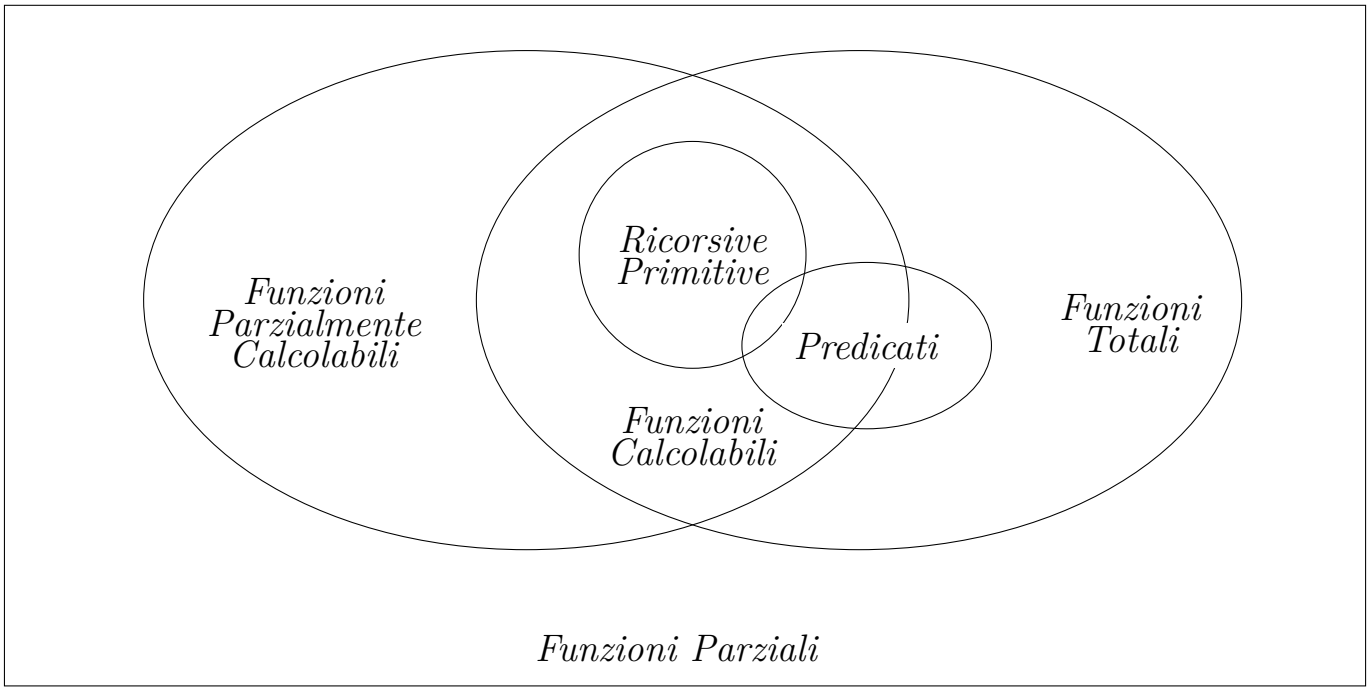


Figura 3.1: Diagramma Riassuntivo sulle Funzioni Parziali

Abbiamo infatti visto come possano esistere *Funzioni parzialmente calcolabili*, come la sottrazione fra due numeri, *Funzioni Parziali non Calcolabili*, tra l'altro anche *Totali* come ***HALT***, e *Funzioni Totali Calcolabili*, come la somma fra due numeri.

Abbiamo poi visto come anche tra i predicati esistano *Predicati Calcolabili* come il **predicato di uguaglianza** e predicati non calcolabili, considerando ad esempio sempre ***HALT***.

Abbiamo poi definito la classe delle *Funzioni ricorsiva primitive*, di cui abbiamo visto esempi anche tra i predicati come la **funzione rilevatrice di zeri**.

Ci manca solo da analizzare il caso di una funzione che sia *calcolabile*, ma che non appartenga alle funzioni *ricorsive primitive*. Per mostrare ciò ci avvarremo della **funzione di Ackermann-Péter**.

Esempio di funzione calcolabile ma non Ricorsiva Primitiva

Funzione di Ackermann-Péter

$$A(x,y) = \begin{cases} y+1, & x = 0 \\ A(x-1,1), & y = 0 \\ A(x-1, A(x,y-1)), & \text{altrimenti} \end{cases} \tag{3.33}$$

Per quanto si possa agilmente scrivere un S-programma che calcoli questa funzione, questa non è ricorsiva primitiva. Vediamo infine un esempio di calcolo di questa funzione per input $(2,1)$:

$$A(2,1) = A(1, A(2,0)) \tag{3.34}$$

Calcoliamo dunque separatamente $A(2,0)$:

$$A(2,0) = A(1,1) = A(0, A(1,0)) = A(1,0) + 1 = A(0,1) + 1 = 3 \tag{3.35}$$

Avremo dunque:

$$A(2,1) = A(1, A(2,0)) = A(1,3) = A(0, A(1,2)) = A(1,2) + 1 = A(0, A(1,1)) + 1 = A(1,1) + 1 + 1 = 3 + 1 + 1 = 5 \tag{3.36}$$

Abbiamo introdotto tre importanti concetti: la funzione universale, il predicato HALT e il predicato STP.

La prima differenza è che HALT non è mai calcolabile (quindi non possiamo usarlo nei programmi S), STP invece è addirittura ricorsiva primitiva quindi possiamo usarlo liberamente, mentre la funzione universale è parzialmente calcolabile quindi possiamo usarla nei programmi S ma non possiamo essere sicuri che termini sempre.

La funzione HALT ci permette di capire se il programma y con i parametri x1, x2, ..., xn termina o meno. Il predicato STP invece ci permette di capire se il programma y con i parametri x1, x2, ..., xn termina in al più t passi. Mentre il programma universale ci permette di simulare il programma y con i parametri x1, x2, ..., xn.

Il programma S seguente, è la funzione somma che calcola la somma di X1 e X2 e la mette in Y. La identifichiamo come f(x1, x2).

```
1      Y ← X1
2      Z ← X2
3  [A] IF Z ≠ 0 GOTO B
4      GOTO E
5  [B] Z ← Z - 1
6      Y ← Y + 1
7      GOTO A
```

Codice 3.3: Funzione somma

Come visto nelle sezioni precedenti, per poter usare funzioni come STP e la funzione universale dobbiamo codificare l'S-Programma in un numero, ricaviamo la nostra y.

Il programma $\mathcal{P}$ viene codificato tramite la funzione di Gödel. Definiamo la quaterna di codifica per ogni istruzione I_k come $\#(I_k) = \langle a, b, c, d \rangle$. Utilizzando la mappatura dei registri $R_1 = Y, R_2 = Z, R_3 = X_1, R_4 = X_2$, otteniamo:

- $n_1 = \#(Y \leftarrow X_1) = \langle 0, 0, 1, 3 \rangle$
- $n_2 = \#(Z \leftarrow X_2) = \langle 0, 0, 2, 4 \rangle$
- $n_3 = \#([A] \text{ IF } Z \neq 0 \text{ GOTO } B) = \langle 1, 4, 2, 2 \rangle$
- $n_4 = \#(\text{GOTO } E) = \langle 0, 2, 0, 0 \rangle$
- $n_5 = \#([B] Z \leftarrow Z - 1) = \langle 2, 2, 2, 0 \rangle$
- $n_6 = \#(Y \leftarrow Y + 1) = \langle 0, 1, 1, 0 \rangle$
- $n_7 = \#(\text{GOTO } A) = \langle 0, 3, 0, 1 \rangle$

Il parametro z da utilizzare nei nostri programmi sarà (per semplicità non calcoliamo il numero di Gödel completo ma lasciamo la quaterna di codifica di ogni istruzione):

$$z = \langle n_1, n_2, n_3, n_4, n_5, n_6, n_7 \rangle$$

Ok ora possiamo scrivere un programma S che valuta se il programma di somma termina in al più t passi (nel programma i passi t sono Z0) e, in tal caso, ne calcola il risultato ad esempio.

```
1    Z1 ← 0 # Inizializziamo il contatore dei passi
2    [A] Z2 ← STP(X1, X2, z, t) # Il parametro z è ciò che abbiamo trovato nei passi precedenti
3
4    IF Z2 ≠ 0 GOTO B
5    Z1 ← Z1 + 1 # Se il programma non è terminato in Z1 passi allora incrementiamo Z1 e ricominciamo
6    GOTO A
7
8    [B] Y ← f(X1, X2) # Sappiamo che f termina in al più Z1 passi, quindi calcoliamo il risultato
```

Codice 3.4: Funzione somma

La funzione universale si usa in modo simile, ma invece di verificare se il programma termina in al più t passi, lo esegue direttamente. Quindi il programma S diventa:

```
1
2    Y ←Φ (X1, X2, z) # Il parametro z è ciò che abbiamo trovato nei passi precedenti
```

Codice 3.5: Funzione somma

Per inciso, in questo specifico esempio era inutile usare la funzione STP che cercava di capire se in al più t passi il programma terminasse poiché sappiamo già che la funzione f(x1, x2) termina sempre. Ma ai fini didattici è utile per capire come usare queste funzioni.

3.8 Esercizi dalle prove d'esame su STP, HALT e Funzione Universale

3.8.1 Esercizio 1

Sia f una funzione binaria totale tale che $f(x,y) > 0 \iff \exists z : \mathrm{STP}(x,y,z)$. Dimostrare che f non può essere calcolabile.

Soluzione: Qui non c'è bisogno di creare un programma ma solo usare qualche definizione e proprietà viste nel capitolo.

Supponiamo per assurdo che f sia calcolabile. Allora $\mathrm{STP}x,y,z$ sarebbe calcolabile ma STP è scrivibile anche come $\mathrm{HALT}(x,y)$. Se ciò fosse vero allora avremmo risolto il problema della fermata, che sappiamo essere indecidibile. Quindi la nostra ipotesi iniziale è falsa e f non può essere calcolabile.

3.8.2 Esercizio 2

Sia h la funzione binaria così definita:

$$h(x_1,x_2) = \begin{cases} (x_1,x_2) & \text{se } \exists z : \mathrm{STP}(x_1,x_2,z) \\ \uparrow & \text{altrimenti.} \end{cases}$$

Tale funzione è parzialmente calcolabile? Dimostrare o confutare.

Soluzione: Qui attenzione a non confondere i parmaetri di STP. Sappiamo che STP accetta: parametri, programma, passi. Quindi in questo caso x_1 è il parametro, x_2 il programma (numero che ci viene già fornito quindi non dobbiamo calcolarlo noi) e z i passi. Fatta questa doverosa precisazione, possiamo scrivere l'S-programma che calcola la funzione h:

```
Z1 ← 0 # Inizializziamo il contatore dei passi

# Qui stiamo cercando di capire se il programma X2 con parametri X1 termina in al più Z1 passi.
# Dovesse essere così, allora potremmo usare liberamente il programma universale per calcolare il
    risultato.
[A] Z2 ← STP(X1, X2, Z1)

IF Z2 ≠ 0 GOTO B
Z1 ← Z1 + 1 # Se il programma non è terminato in Z1 passi allora incrementiamo Z1 e ricominciamo
GOTO A

[B] Y ← Φ^(2)(x1,x2) # Sappiamo già che il programma X2 con parametri X1 termina in al più Z1 passi (
    grazie ad STP), quindi calcoliamo il risultato
```

Codice 3.6: Funzione h

3.9 Esercizi e approfondimenti

Esercizio:

Dimostrare che la funzione di Fibonacci $F(x)$ è ricorsiva primitiva.
Consiglio, usare funzione pairing.

Consideriamo la funzione g definita come:

$$\begin{cases} g(0) = \langle 0,1 \rangle \\ g(n+1) = \langle r(g(n)), l(g(n)) + r(g(n)) \rangle \end{cases} \tag{3.37}$$

La funzione g è ricorsiva primitiva in quanto composizione di funzioni ricorsive primitive.

Possiamo ora facilmente notare che $F(x) = l(g(x))$ è ricorsiva primitiva in quanto composizione di funzioni ricorsive primitive.

Esercizio

Calcolare $\#\mathcal{P}$ per i seguenti programmi.

```
    IF X ≠ 0 GOTO A
[A] X ← X + 1
```

```
    IF X ≠ 0 GOTO A
    Y ← Y + 1
```

Codice 3.7: Esercizio Codifica Programmi 1

Iniziamo dalla codifica delle singole istruzioni. Avremo:

1. $\langle 0, \langle 3, 1\rangle\rangle = \left\langle 0, 2^3(2 \cdot 1 + 1) - 1\right\rangle = \langle 0, 23\rangle = 2^0(2 \cdot 23 + 1) - 1 = 46$;
2. $\langle 1, \langle 1, 1\rangle\rangle = \left\langle 1, 2^1(2 \cdot 1 + 1) - 1\right\rangle = \langle 1, 5\rangle = 2^1(2 \cdot 5 + 1) - 1 = 21$;
3. $\langle 0, \langle 3, 1\rangle\rangle = 46$;
4. $\langle 0, \langle 1, 0\rangle\rangle = \left\langle 0, 2^1(2 \cdot 0 + 1) - 1\right\rangle = \langle 0, 1\rangle = 2^0(2 \cdot 1 + 1) - 1 = 2$;

Utilizziamo ora i Numeri di Gödel per trovare $\#(\mathcal{P})$. Essendo il programma composto da 4 istruzioni avremo:

$$\#(\mathcal{P}) = P_1^{\#(I_1)} + P_2^{\#(I_2)} + P_3^{\#(I_3)} + P_4^{\#(I_4)} - 1 = 2^{46} \cdot 3^{21} \cdot 5^{46} \cdot 7^2 - 1 \approx 5.125573068 \times 10^{57} \tag{3.38}$$

```
[B] IF X ≠ 0 GOTO A
    Z ← Z + 1
    IF Z ≠ 0 GOTO B
[A] X ← X
```

Codice 3.8: Esercizio Codifica Programmi 1

Iniziamo dalla codifica delle singole istruzioni. Avremo:

1. $\langle 2, \langle 3, 1\rangle\rangle = \left\langle 2, 2^3(2 \cdot 1 + 1) - 1\right\rangle = \langle 2, 23\rangle = 2^2(2 \cdot 23 + 1) - 1 = 187$;
2. $\langle 0, \langle 1, 2\rangle\rangle = \left\langle 0, 2^1(2 \cdot 2 + 1) - 1\right\rangle = \langle 0, 9\rangle = 2^0(2 \cdot 9 + 1) - 1 = 18$;
3. $\langle 0, \langle 4, 2\rangle\rangle = \left\langle 0, 2^4(2 \cdot 2 + 1) - 1\right\rangle = \langle 0, 79\rangle = 2^0(2 \cdot 79 + 1) - 1 = 158$;
4. $\langle 1, \langle 0, 1\rangle\rangle = \left\langle 1, 2^0(2 \cdot 1 + 1) - 1\right\rangle = \langle 1, 2\rangle = 2^1(2 \cdot 2 + 1) - 1 = 9$;

Utilizziamo ora i Numeri di Gödel per trovare $\#(\mathcal{P})$. Essendo il programma composto da 4 istruzioni avremo:

$$\#(\mathcal{P}) = P_1^{\#(I_1)} + P_2^{\#(I_2)} + P_3^{\#(I_3)} + P_4^{\#(I_4)} - 1 = 2^{187} \cdot 3^{18} \cdot 5^{158} \cdot 7^9 - 1 \approx 8.39334006442862300469657 5 \times 10^{182} \tag{3.39}$$

Esercizio:

Determinare il programma $\mathcal{P}$ di numero $\#\mathcal{P} = 575$.

Iniziamo scomponendo in fattori primi il numero dato +1:

$$576 = 5^2 \cdot 23 = 2^6 \cdot 3^2 \tag{3.40}$$

Abbiamo dunque che $Lt(576) = 2$. Decodifichiamo adesso gli esponenti nelle singole istruzioni:

1. $$\begin{aligned} &6 = \langle a_1, \langle b_1, c_1\rangle\rangle \implies 2^{a_1}(2\langle b_1, c_1\rangle + 1) - 1 = 6 \implies 2^{a_1}(2\langle b_1, c_1\rangle + 1) = 7 \implies a_1 = 0 \wedge \langle b_1, c_1\rangle = 3 \\ &a_1 = 0, 2^{b_1}(2c_1 + 1) = 4 \implies a_1 = 0, b_1 = 2, c_1 = 0 \end{aligned} \tag{3.41}$$

   Abbiamo dunque l'istruzione (0,2,0) ovvero l'istruzione `Y ← Y - 1`;
2. $2 = \langle a_1, \langle b_1, c_1\rangle\rangle \implies 2^{a_1}(2\langle b_1, c_1\rangle + 1) - 1 = 2 \implies 2^{a_1}(2\langle b_1, c_1\rangle + 1) = 3 \implies a_1 = 0 \wedge \langle b_1, c_1\rangle = 1 \implies a_1 = 0, 2^{b_1}(2c_1 + 1) = 2 \implies a_1 = 0, b_1 = 1, c_1 = 0$. Abbiamo dunque l'istruzione (0,1,0) ovvero l'istruzione `Y ← Y + 1`;

Abbiamo dunque che il programma $\mathcal{P}$ di numero $\#\mathcal{P} = 575$ è:

```
    Y ← Y - 1
    Y ← Y + 1
```

Codice 3.9: Esercizio Codifica Programmi 3

Esercizio:

Sia $\overline{HALT}(x,y) \iff \Psi^{(1)}_{\mathcal{P}_y}(x)\uparrow$. *Mostrare che* $\overline{HALT}(x,y)$ *non è calcolabile.*

Dimostrazione per Assurdo

Supponiamo che $\overline{HALT}(x,y)$ sia calcolabile. Allora è calcolabile anche il predicato $\neg\overline{HALT}(x,y)$ dal teorema 2.3.1. Ma si ha che:

$$\neg\overline{HALT}(x,y) \iff HALT(x,y) \tag{3.42}$$

Ma dal teorema 3.4 sappiamo che $HALT(x,y)$ non è calcolabile. Assurdo.

Esercizio:

Dimostrare o Confutare: "Se f *è una funzione* n*-aria totale per la quale esiste* $k \in \mathbb{N}$ *tale che* $f(x_1,\cdots,x_n) \leq k$ *per ogni* n*-upla* $(x_1,\cdots,x_n)$*, allora* f *è calcolabil".*

Confutazione per controesempio.

È sufficiente notare che il **predicato** $HALT^{(n-1)}$ rispetta i requisiti della traccia ma non è calcolabile.

Infatti da definizione di **predicato**, $HALT^{(n-1)}$ è una **funzione totale** con codominio $\{0,1\}$. Dunque:

$$\forall(x_1,x_2,\cdots,x_{n-1},y) \in \mathbb{N}^n, HALT^{(n-1)}(x_1,x_2,\cdots,x_{n-1},y) \leq k = 1 \tag{3.43}$$

Esercizio. *Mostrare come calcolabile la funzione:*

$$H_1(x) = \begin{cases} 1, & \Phi(x,x)\downarrow \\ \uparrow, & \text{altrimenti} \end{cases} \tag{3.44}$$

Per dimostrare la calcolabilità di questa funzione è sufficiente considerare il seguente programma:

```
Z ← Φ(X,X)
Y ← Y + 1
```

Codice 3.10: Programma H1

Essendo $\Phi(x,x)$ calcolabile può essere infatti usata come macro in altri programmi. Nel caso $\Phi(x,x)$ termini, il programma terminerà con output 1, altrimenti terminerà in loop.

Alternativamente è possibile considerare la funzione data come:

$$H_1(x) = \alpha(\Phi(x,x)) + \alpha(\alpha(\Phi(x,x))) \tag{3.45}$$

Si avrà infatti che:

$$\begin{aligned} \Phi(x,x)\downarrow \wedge \Phi(x,x) = 0 &\implies \alpha(\Phi(x,x)) = 1 \wedge \alpha(\alpha(\Phi(x,x))) = 0 \implies H_1(x) = 1 \\ \Phi(x,x)\downarrow \wedge \Phi(x,x) \neq 0 &\implies \alpha(\Phi(x,x)) = 0 \wedge \alpha(\alpha(\Phi(x,x))) = 1 \implies H_1(x) = 1 \\ \Phi(x,x)\uparrow &\implies H_1(x)\uparrow \end{aligned} \tag{3.46}$$

Esercizio. *Trovare un programma* $\mathcal{P}$ *tale che il seguente predicato sia non calcolabile:*

$$H_{\mathcal{P}}(x_1,x_2) \iff \text{il programma } \mathcal{P} \text{ termina su input } x_1,x_2 \tag{3.47}$$

Basta considerare il programma universale $\mathcal{U}^{(1)}$ che calcola $\Phi^{(1)}(x_1,x_2)$, allora:

$$H_{\mathcal{U}^{(1)}}(x_1,x_2) \iff \Phi^{(1)}(x_1,x_2)\downarrow \iff HALT(x_1,x_2) \tag{3.48}$$

Insiemi Ricorsivi e Ricorsivamente Enumerabili

4.1 Da funzioni a insiemi

Conclusa l'analisi relativa alla calcolabilità delle funzioni possiamo ora spostarci a quella relativa agli insiemi.
Chiaramente, avendo considerato esclusivamente funzioni con dominio in $\mathbb{N}^n$, ci occuperemo solo di sottoinsiemi di tale dominio.
Per collegare le due analisi, consideriamo la funzione caratteristica di un insieme $S \subseteq \mathbb{N}^n$:

Funzione Caratteristica

Dato $S \subseteq \mathbb{N}^n$, definiamo come **funzione caratteristica** il predicato:

$$f_S(a_1, \cdots, a_n) = \begin{cases} 1, & (a_1, \cdots, a_n) \in S \\ 0, & \text{altrimenti} \end{cases} \tag{4.1}$$

È facile intuire[1] che esiste una corrispondenza biunivoca tra l'insieme delle funzioni caratteristiche e l'insieme delle parti di $\mathbb{N}^n$, ovvero che esista esattamente una funzione per ogni sottoinsieme S considerato.

Diremo dunque, con un leggero abuso di nomenclatura, che S *appartiene* a una certa **classe PRC** $\mathcal{C}$ se vi appartiene f_s.

Proprietà della funzione caratteristica Diremo dunque che S è *ricorsivo* $\iff$ f_s è *ricorsivo* ovvero *calcolabile*, mentre S è *ricorsivo primitivo* $\iff$ f_s è *ricorsivo primitivo*.

4.1.1 Analisi degli Insiemi Ricorsivi

Iniziamo l'analisi della *ricorsività* degli insiemi semplificandone la trattazione. Possiamo vedere infatti che, partendo da un sottoinsieme di $\mathbb{N}^n$ con n arbitrario, è sempre possibile ricondurci allo studio di un sottoinsieme di $\mathbb{N}$ analogo.

[1] Questa dimostrazione non rientra nelle finalità di questo corso, quindi verrà tralasciata. È però trattata nel dettaglio nel corso di Algebra.

Riduzione a sottoinsiemi di $\mathbb{N}$

Enunciato. $S \subseteq \mathbb{N}^n$ appartiene alla classe PRC $\mathcal{C}$ *se e solo se* a $\mathcal{C}$ appartiene anche:

$$S' = \{[x_1, \cdots, x_n] | (x_1, \cdots, x_n) \in S\} \subseteq \mathbb{N} \tag{4.2}$$

Dimostrazione. Verifichiamo la doppia implicazione:

$\Longrightarrow$ Sia $f_S \in \mathcal{C}$. Allora:

$$f_{S'}(x) \iff Lt(x) \leq n \wedge x \neq 0 \wedge f_S((x)_1, \cdots, (x)_n) \tag{4.3}$$

Dunque $f_{S'} \in \mathcal{C}$ poiché composto di predicati ricorsivi primitivi o appartenenti a $\mathcal{C}$.

$\Longleftarrow$ Sia $f_{S'} \in \mathcal{C}$. Allora:

$$f_S(x_1, \cdots, x_n) \iff f_{S'}([x_1, \cdots, x_n,]) \tag{4.4}$$

Dunque $f_S \in \mathcal{C}$ poiché composto di predicati ricorsivi primitivi o appartenenti a $\mathcal{C}$.

Andiamo adesso, analogamente a quanto fatto per le funzioni, ad analizzare una **proprietà di chiusura** degli **insiemi ricorsivi**.

Chiusura per operazioni insiemistiche su insiemi ricorsivi

Enunciato. Siano $S, T \subseteq \mathbb{N}^n$ appartenenti a $\mathcal{C}$. Allora anche $S \cup T$, $S \cap T$ e $\overline{S} = \mathbb{N}^n \setminus S$ appartengono a $\mathcal{C}$.

Dimostrazione. Basta osservare che:

$$\begin{aligned} f_{S\cup T}(x_1, \cdots, x_n) &= f_S(x_1, \cdots, x_n) \vee f_T(x_1, \cdots, x_n) \\ f_{S\cap T}(x_1, \cdots, x_n) &= f_S(x_1, \cdots, x_n) \wedge f_T(x_1, \cdots, x_n) \\ f_{\overline{S}}(x_1, \cdots, x_n) &= \neg f_S(x_1, \cdots, x_n) \end{aligned} \tag{4.5}$$

Abbiamo dunque che questi predicati sono composti di predicati ricorsivi primitivi o appartenenti a $\mathcal{C}$.

È possibile notare come da questa dimostrazione, è possibile ricavare tale proprietà di chiusura anche per altre operazioni insiemistiche come:

$$\begin{aligned} S \setminus T &= S \cap \overline{T} \\ S \triangle T &= (S \setminus T) \cup (T \setminus S) \end{aligned} \tag{4.6}$$

Esempi

1. Qualsiasi $S \subseteq \mathbb{N}$ finito è ricorsivo primitivo. Infatti se $S = \{s_1, \cdots, s_k\}$, allora $f_S(x) \iff x = s_1 \vee \cdots \vee x = s_k$
2. L'insieme $\mathbf{P}$ dei numeri primi è ricorsivo primitivo poiché $f_{\mathbf{P}} = Primo$.
3. L'insieme $H = \left\{(x_1, x_2) \in \mathbb{N}^2 | HALT(x_1, x_2)\right\}$ **non è ricorsivo**, dato che $f_H = HALT$. Abbiamo inoltre che $H' = \{[x_1, x_2] \in \mathbb{N} | (x_1, x_2) \in H \iff HALT(x_1, x_2)\}$ **non è ricorsivo**, dal teorema precedente.

4.1.2 Insiemi Ricorsivamente Enumerabili

Abbiamo visto come partendo dai predicati, sottoinsieme delle funzioni totali, è possibile definire il concetto di insieme ricorsivo. Vediamo ora come sia possibile definire un concetto analogo per le funzioni parziali.

Insiemi Ricorsivamente Enumerabili

$S \subseteq \mathbb{N}$ si dice **ricorsivamente enumerabile** se esiste una funzione g **unaria** e **parzialmente calcolabile** tale che:

$$S = \{x \in \mathbb{N} | g(x) \downarrow\} \tag{4.7}$$

Possiamo dunque dire che un insieme è **ricorsivamente enumerabile** se esiste per esso una **Procedure di Semi-Decisione**, ovvero una **procedura** che termina *se e solo se* l'elemento appartiene all'insieme. Nel nostro caso specifico, se esiste un **S-Programma** che termina solo per gli input appartenenti a tale insieme.

Errore molto comune: Nel S-Programma che usiamo per descrivere la funzione g NON possiamo usare f_S per dimostrare che l'elemento x è nell'insieme S poiché sarebbe un **ragionamento circolare**. Dobbiamo semplicemente scrivere un programma-S per rappresentare g che termina SE E SOLO SE rispetta le condizioni di appartenenza a S.

Un insieme è invece **ricorsivo** *se e solo se* esiste per esso una **Procedura di Decisione**, cioè un programma che termina **sempre** e dà output diversi, come 1 o 0, a seconda che l'input appartenga all'insieme o meno. Un esempio di **Procedura di Decisione** per l'insieme S è il programma che calcola f_S.

È fondamentale sottolineare le differenze tra questi due tipi di **procedure**. Le **Procedura di Decisione** sono per definizione **funzioni totali**, poiché per ogni input dato devono valutare l'appartenenza o meno all'insieme. A differenza di quest'ultime le **Procedure di Semi-Decisione**, essendo **funzioni parziali**, mi garantiscono esclusivamente l'*appartenenza* di un input a un dato insieme, ma non la *non appartenenza*, poiché la procedura non terminerebbe.

Sottolineata questa analogia tra **funzioni totali e parziali** e **insiemi ricorsivi e ricorsivamente enumerabili** è facile intuire che gli **insiemi ricorsivi** siano un sottoinsieme di quelli **ricorsivamente enumerabili**.

Insiemi ricorsivi contenuti nei ricorsivamente enumerabili

Enunciato. Se $S \subseteq \mathbb{N}$ è ricorsivo, allora è anche ricorsivamente enumerabile.

Dimostrazione. È sufficiente considerare il programma:

```
[A] IF ¬f_S(X) GOTO A
```

Questo programma termina *se e solo se* $X \in S$.

Il legame tra insiemi ricorsivi e ricorsivamente enumerabili è però molto più stretto di quanto appena visto.

Relazione tra ricorsività e ricorsivo enumerabilità

Enunciato. $S \subseteq \mathbb{N}$ è ricorsivo *se e solo se* S e $\overline{S}$ sono ricorsivamente enumerabili.

Dimostrazione. Verifichiamo la doppia implicazione:

$\Longrightarrow$ Ovvia dai teoremi 4.1.1 e 4.1.2.

$\Longleftarrow$ Siano g, h funzioni parzialmente calcolabili tali che $S = \{x \in \mathbb{N} | g(x) \downarrow\}$ e $\overline{S} = \{x \in \mathbb{N} | h(x) \downarrow\}$. Siano inoltre $p, q \in \mathbb{N}$ tali che $g = \Psi^{(1)}_{\mathcal{P}_p}$ e $h = \Psi^{(1)}_{\mathcal{P}_q}$. Allora f_s è calcolata dal programma:

```
[A] IF STP(X,p,Z) GOTO B
    IF STP(X,q,Z) GOTO E
    Z ← Z + 1
    GOTO A
[B] Y ← Y + 1
```

È importante notare che non sarebbe stato sufficiente utilizzare h e g come macro in un programma, poiché essendo funzioni parziali, la prima delle due incontrata su input che non la soddisfa avrebbe impedito la terminazione.
È necessario dunque procedere col predicato STP aumentando gradualmente il numero di *step* concessi. In questo modo il programma non andrà mai in loop poiché $S \cup \overline{S} = \mathbb{N}$, ovvero prima o poi il programma dovrà terminare.

Caratterizzati a modo questa classe di insiemi, possiamo valutare la **proprietà di chiusura** per operazioni insiemistiche anche su di loro.

Teorema

Enunciato. Siano S e S' insiemi ricorsivamente enumerabili. Allora lo sono anche $S \cap S'$ e $S \cup S'$.

Dimostrazione.

$\cap$: Siano g, h funzioni unarie parzialmente calcolabili tali che $S = \{x \in \mathbb{N} | g(x) \downarrow\}$ e $S' = \{x \in \mathbb{N} | h(x) \downarrow\}$. Consideriamo allora il programma:

```
Z ← g(X)
Z ← h(X)
```

Questo programma termina *se e solo se* $g(x) \downarrow \wedge h(x) \downarrow$, ovvero *se e solo se* $x \in S \cap S'$.

$\cup$: Siano g, h funzioni unarie 1parzialmente calcolabili tali che $S = \{x \in \mathbb{N} | g(x) \downarrow\}$ e $S' = \{x \in \mathbb{N} | h(x) \downarrow\}$. Siano inoltre $p, q \in \mathbb{N}$ tali che $g = \Psi^{(1)}_{\mathcal{P}_p}$ e $h = \Psi^{(1)}_{\mathcal{P}_q}$. Consideriamo allora il programma:

```
[A] IF STP(X,p,Z) GOTO E
    IF STP(X,q,Z) GOTO E
    Z ← Z + 1
    GOTO A
```

Analogamente al teorema 4.1.2 è fondamentale l'utilizzo progressivo del predicato STP per valutare entrambe le funzioni. Avremo dunque che questo programma termina *se e solo se* $g(x) \downarrow \vee h(x) \downarrow$, ovvero *se e solo se* $x \in S \cup S'$.

Per quanto riguarda l'**operatore di complemento** non è possibile dimostrare la proprietà di chiusura, e infatti in seguito vedremo un controesempio[2].

Possiamo quindi definire un nuovo tipo di insieme:

$$W_n = \{x \in \mathbb{N} | \Phi(x, n) \downarrow\} \tag{4.8}$$

Grazie a questa nuova definizione possiamo osservare che:

$$S \subseteq \mathbb{N} \text{ ricorsivamente enumerabile} \iff \exists n \in \mathbb{N} : S = W_n \tag{4.9}$$

Infatti avremo che S ricorsivamente enumerabile $\iff \exists g$ parzialmente calcolabile $: S = \{x | g(x) \downarrow\} \iff S = \left\{x | \Psi^{(1)}_{\mathcal{P}_n}(x) \downarrow\right\}$ per qualche $n \in \mathbb{N}$

Con la definizione di W_n abbiamo un nuovo modo per indicare che un programma termina su un dato input, infatti avremo che:

$$HALT(x, y) \iff \Psi^{(1)}_{\mathcal{P}_y}(x) \downarrow \iff \Phi(x, y) \downarrow \iff \exists t \in \mathbb{N} : STP(x, y, t) \iff x \in W_y \tag{4.10}$$

[2] Esercizi consigliati: 4.13

Insieme Diagonale

Definiamo quindi l'**insieme diagonale** come:

$$K = \{x \in \mathbb{N} | x \in W_x\} = \{x \in \mathbb{N} | \Phi(x,x) \downarrow\} \subseteq \mathbb{N}$$

Avremo che il nuovo insieme K appena definito, conterrà tutti i numeri naturali $n \in \mathbb{N}$ tali che, presi come input, facciano terminare il programma numero n-esimo.
Di conseguenza possiamo definire il **complemento** di K come l'**insieme anti-diagonale** $\overline{K}$.

Questo insieme ha questo nome poiché, nella matrice vista nel teorema 3.4, considera esattamente gli elementi sulla sua diagonale.

Questo insieme è particolarmente rilevante, poiché è esempio di insieme **ricorsivamente enumerabile** il cui complemento non è **ricorsivamente enumerabile**.

Teorema di Enumerazione

Enunciato. K è ricorsivamente enumerabile ma non ricorsivo.

Dimostrazione. Dalla sua definizione sappiamo che, K è ricorsivamente enumerabile.
Vediamo ora due dimostrazioni per la sua non ricorsività:

1. Dal teorema 4.1.2 sappiamo che K ricorsivamente enumerabile è **ricorsivo** *se e solo se* anche $\overline{K}$ è **ricorsivamente enumerabile**. **Per assurdo**, supponiamo $\overline{K}$ ricorsivamente enumerabile.
   Abbiamo allora da quanto visto prima che $\exists i \in \mathbb{N} : \overline{K} = W_i = \{x \in \mathbb{N} | \Phi(x,i) \downarrow\}$.
   Si ha dunque che $\forall x \in \mathbb{N} : x \in \overline{K} \iff x \in W_i \iff \Phi(x,i) \downarrow$.
   In particolare per $x = i$ abbiamo che $i \in \overline{K} \iff i \in W_i \iff i \in K$. Questo porta chiaramente ad **assurdo** poiché un numero non può appartenere sia a un insieme che al suo complemento.
   Questa dimostrazione oltre a mostrare che K non è **ricorsivo** ci mostra anche che $\overline{K}$ non è **ricorsivamente enumerabile**.
2. K è ricorsivo *se e solo se* f_k è calcolabile. Ma abbiamo che $f_k(n) \iff n \in K \iff n \in W_n \iff HALT(n,n)$
   E come visto dalla dimostrazione di non calcolabilità di $HALT$ abbiamo che $HALT(x,x)$ non è calcolabile.

Gli insiemi ricorsivamente enumerabili possono essere caratterizzati anche in un altro modo, ovvero come **codominio** di funzioni piuttosto che **dominio**.

Prima di vedere ciò però, è necessario dimostrare prima un risultato intermedio che ci aiuterà nella dimostrazione, ovvero che per ogni insieme ricorsivamente enumerabile esiste un predicato binario ricorsivamente enumerabile che lo caratterizza.

Insiemi ricorsivamente enumerabili e predicati binari ricorsivi primitivi

Enunciato. Se $S \subseteq \mathbb{N}$ è ricorsivamente enumerabile allora esiste R predicato binario ricorsivo primitivo tale che $x \in S \iff \exists t \in \mathbb{N} : R(x,t)$, ovvero $S = \{x \in \mathbb{N} | \exists t \in \mathbb{N} : R(x,t)\}$.

Dimostrazione. Sia n tale che $S = W_n$. Allora $S = \{x \in \mathbb{N} | \Phi(x,n) \downarrow\} = \{x \in \mathbb{N} | \exists t \in \mathbb{N} : STP_n(x,t) \downarrow\}$, dove $STP_n(x,t)$ è un predicato binario definito come $STP_n(x,t) \iff STP(x,n,t)$. Questo è chiaramente ricorsivo primitivo poiché definibile per casi partendo da STP. Abbiamo dunque che l'enunciato del teorema è vero per $R(x,t) = STP_n(x,t)$.

Ottenuto questo risultato possiamo dunque procedere con la dimostrazione di una prima versione del teorema accennato in precedenza, relativo alle funzioni **ricorsive primitive**.

Insiemi ricorsivamente enumerabili e funzioni ricorsive primitive

Enunciato. Sia $S \subseteq \mathbb{N}$ non vuoto ricorsivamente enumerabile. Allora esiste h funzione ricorsiva primitiva tale che $S = \{h(x) | x \in \mathbb{N}\}$.

Dimostrazione. Sia R predicato binario ricorsivo primitivo tale che $S = \{x \in \mathbb{N} | \exists t \in \mathbb{N} : R(x,t)\}$. Essendo $S \neq \varnothing$ è possibile fissare un $x_0 \in S$. Definiamo dunque h come:

$$h(x) = \begin{cases} l(x), & R(l(x), r(x)) \\ x_0, & \text{altrimenti} \end{cases} \tag{4.11}$$

Essendo definita per casi da funzioni e predicati ricorsivi primitivi h è chiaramente ricorsiva primitiva. Verifichiamo ora che $S = H := \{h(x) | x \in \mathbb{N}\}$. Per vedere che $S = H$ verifichiamo la doppia inclusione:

- $\subseteq$ $y \in H \implies y = x_0 \in S \vee \exists x \in \mathbb{N} : (y = l(x) \wedge R(l(x), r(x)))$. Abbiamo dunque in questo secondo caso $R(y, r(x))$.
  Abbiamo dunque che $y \in S$ dal teorema precedente poiché $\exists t \in \mathbb{N} : R(y,t)$, ovvero $t = r(x)$.
- $\supseteq$ Viceversa $x \in S \implies \exists t : R(x,t)$. Allora $x = h(\langle x,t \rangle)$ poiché $R(l(\langle x,t \rangle), r(\langle x,t \rangle))$ e dunque $x \in H$.

Vediamo ora un teorema più generale, che ci permette di caratterizzare gli insiemi ricorsivamente enumerabili come **codominio** di funzioni parzialmente calcolabili.

Teorema

Enunciato. Sia $S = \{f(n) | f(n) \downarrow\}$, con f parzialmente calcolabile. Allora S è ricorsivamente enumerabile.

Dimostrazione. Sia p il numero di programma che calcola f. Consideriamo dunque il programma:

```
[A] IF ¬STP(Z1, p, Z2) GOTO B
    Z3 ← f(Z1)
    IF X = Z3 GOTO E
[B] Z1 ← Z1 + 1
    IF Z1 ≤ Z2 GOTO A
    Z1 ← 0
    Z2 ← Z2 + 1
    GOTO A
```

Questo programma termina solo su input x tali che $\exists n \in \mathbb{N} : f(n) = x$. Abbiamo dunque che S è ricorsivamente enumerabile.
Nel programma viene eseguito un doppio controllo su i potenziali n e il numero di passi concessi per la computazione. Questo è necessario per assicurarsi che tutti i numeri vengano controllati, infatti per ogni valore del contapassi Z_2 vengono controllati tutti i valori di Z_1 compresi tra 0 e Z_2.
Senza questa accortezza il programma proverebbe a calcolare f sullo stesso numero all'infinito se questo non facesse terminare f.
Inoltre il programma può terminare solo effettuando il salto sulla terza istruzione, cioè se $X = Z_3$, ovvero se $f(Z_1) = X$.

Sfruttando quest'ultimi risultati possiamo dunque dimostrare un teorema di carattere generale che caratterizza gli insiemi ricorsivamente enumerabili a pieno[3].

Teorema di caratterizzazione degli insiemi ricorsivamente enumerabili

Enunciato. Sia $S \subseteq \mathbb{N}$ non vuoto. Allora sono equivalenti:
1. S è ricorsivamente enumerabile, cioè $\exists f$ parzialmente calcolabile tale che $S = \{n | f(n) \downarrow\}$.
2. Esiste f ricorsiva primitiva tale che $S = \{f(n) | n \in \mathbb{N}\}$.
3. Esiste f calcolabile tale che $S = \{f(n) | n \in \mathbb{N}\}$.
4. Esiste f parzialmente calcolabile tale che $S = \{f(n) | f(n) \downarrow\}$.

Dimostrazione. Dimostriamo le varie implicazioni.
Come abbiamo visto nei teoremi precedenti $1 \implies 2$ e $4 \implies 1$. Inoltre $2 \implies 3 \implies 4$ sono ovvie.

Esercizio d'esame

Dato un insieme $S \subseteq \mathbb{N}$, sia $2S = \{2x | x \in S\}$. Mostrare che se S è ricorsivamente enumerabile, lo è anche $2S$.
Soluzione. Se $S = \emptyset, 2S$ è banalmente ricorsivamente enumerabile. Altrimenti essendo S ricorsivamente enumerabile, esiste f ricorsiva primitiva tale che $S = \{f(n) | n \in \mathbb{N}\}$. Allora $2S = \{2f(n) | n \in \mathbb{N}\}$ è ricorsivamente enumerabile poiché $2f$ è ricorsiva primitiva.

4.2 Esercizi ricorsivamente enumerabili

4.2.1 Esercizio 1

Sia $B \subseteq \mathbb{N}$ un insieme ricorsivo ed f una funzione unaria calcolabile. Mostrare che l'insieme

$$\{x \in \mathbb{N} \mid x \notin B \vee \exists y : x = f(y)\}$$

è ricorsivamente enumerabile.

Soluzione

Ok quindi dobbiamo ricondurci alla definizione di insieme ricorsivamente enumerabile, ovvero dobbiamo CREARE una funzione g che termini SE E SOLO SE

$$x \notin B \vee \exists y : x = f(y)$$

.

[3] Con questo teorema è conclusa la prima metà del corso, che sarà argomento della prova intercorso del 7/11/2023

La nostra idea quindi è creare una funzione g unaria che prenda in input un numero x e che faccia:

$$g(x) = \begin{cases} x, & \text{se } x \notin B \vee \exists y : x = f(y) \\ \uparrow & \text{altrimenti} \end{cases} \tag{4.12}$$

Osservazioni: Sapendo che B è un insieme ricorsivo, dalla definizione della funzione caratteristica possiamo usare f_B per decidere se un elemento appartiene o meno a B. Ora che abbiamo un'idea di come deve essere fatta g, possiamo procedere a scrivere il programma S che la calcola:

```
# Capisco se x appartiene a B usando la funzione caratteristica di B.
# Non devo preoccuparmi di usare STP perché so che è calcolabile.
Z1 <-f_B(x)

# Se x non appartiene a B, allora abbiamo già finito. Sennò dobbiamo controllare la 2° condizione
IF Z1 !=0 GOTO B
GOTO A

[B] Z2 <-0

[C] Z3 <-f(Z_2)

    # Confrontiamo f(y) con x (contenuto in X1) usando la funzione di uguaglianza calcolabile
    # e ricorsiva primitiva già nota che chiameremo L.
    # Ovviamente non c'è bisogno di re-definirla poiché già trattata in precedenza.
    Z4 <-L(Z3, X1)

    IF Z4 !=0 GOTO A

    # Altrimenti incrementiamo Z2 e riproviamo
    Z2 <-Z2 + 1
    GOTO C

# La funzione g(x) deve restituire x se definita.
[A] Y <-X1
```

Codice 4.4: S-Programma che calcola g(x)

4.2.2 Esercizio 2

Dimostrare che l'insieme $\{[m, n, q] \mid m \in W_n \cup W_q\}$ è ricorsivamente enumerabile.

Soluzione

Come possiamo ben vedere stiamo codificando mediante il numero di Godel una tripla $[m, n, q]$, visto che vogliamo ricondurci alla definizione di insieme ricorsivamente enumerabile, dobbiamo creare una funzione g UNARIA. Ricordiamo che la funzione $[m, n, q]$ ci darà un unico numero che chiameremo x. Nella 2° parte del problema ci viene detto che dobbiamo capire se m appartiene a Wn OPPURE a Wq, come facciamo ad ottenere n, q e m se abbiamo codificato la tripla in un unico numero x? Semplice, usiamo le funzioni di estrazione della codifica di Godel, ovvero:

- $(x)_1$ per ottenere m
- $(x)_2$ per ottenere n
- $(x)_3$ per ottenere q

$$g(x) = \begin{cases} x & \text{se } (x)_1 \in W_{(x)_2} \cup W_{(x)_3} \\ \uparrow & \text{altrimenti} \end{cases}$$

Ricordiamo che $m \in W_n \iff \Phi_n(m) \downarrow \iff \exists t : \text{STP}(n, m, t)$. Poiché abbiamo un'unione ($\cup$), dobbiamo verificare se **almeno uno** dei due programmi (n o q) termina su input m.

Ecco l'S-programma derivato dagli appunti:

```
Z1 <-1

# Usando la definizione di prima, m appartenente a Wn significa che STP(n, m, t) termina per qualche t.
[A] Z2 <-STP((X)_2, (X)_1, Z1)

    IF Z2 !=0 GOTO C

    Z3 <-STP((X)_3, (X)_1, Z1)

    IF Z3 !=0 GOTO C

    Z1 <-Z1 + 1
    GOTO A

# Restituiamo l'input x come richiesto dalla funzione semi-caratteristica
[C] Y <-X
```

Codice 4.5: S-Programma con tecnica Dovetail

4.3 Esercizi ed approfondimenti

Esercizio. *Sia $B \subseteq \mathbb{N}^m, m > 1$. Diciamo che B è **ricorsivamente enumerabile** se $B = \{(x_1, \cdots, x_m) \in \mathbb{N}^m | g(x_1, \cdots, x_m) \downarrow\}$ per qualche funzione g parzialmente calcolabile. Mostrare che:*

$$B \text{ è ricorsivamente enumerabile} \iff B' = \{[x_1, \cdots, x_m] | (x_1, \cdots, x_m) \in B\} \text{ è ricorsivamente enumerabile} \tag{4.13}$$

Dimostriamo la doppia implicazione.

$\Longleftarrow$ Supponiamo che B' sia ricorsivamente enumerabile. Allora esiste una funzione g' parzialmente calcolabile tale che $B' = \{y \in \mathbb{N} | g'(y) \downarrow\}$. Sappiamo poi dalla definizione data in precedenza di B' che:

$$\forall y \in B' : y = [x_1, \cdots, x_m], (x_1, \cdots, x_m) \in B \tag{4.14}$$

Consideriamo dunque il seguente programma di m variabili:

```
        Z ← [X1, X2, \cdots, Xm]
        Z2 ← g'(Z)
```

Questo programma termina se e solo se $y = [x_1, \cdots, x_m] \in B'$, ovvero se e solo se $(x_1, \cdots, x_m) \in B$. Dunque B è ricorsivamente enumerabile.

$\Longrightarrow$ Sia $B \in \mathbb{N}^m$ ricorsivamente enumerabile. Allora fissato tale m possiamo considerare il seguente programma unario:

```
    [A] IF X = 0 ∨ Lt(x) > m GOTO A
        Z1 ← (X)_1
        Z2 ← (X)_2
        ⋮
        Zm ← (X)_m
        Zm+1 ← g(Z1, Z2, \cdots, Zm)
```

Questo programma innanzitutto verifica che l'input sia un numero di Gödel, ovvero sia maggiore di 0, e che la lunghezza della sua fattorizzazione sia m, ovvero sia numero di Gödel di una n-upla di $\mathbb{N}^m$, altrimenti non terminerà. Negli altri casi il programma termina se e solo se $((x)_1, \cdots, (x)_m) \in B$. Essendo che $\forall (y_1, \cdots, y_m) \in B \exists x \in B' : x = [y_1, \cdots, y_m]$ questo programma termina se e solo se $x \in B'$. Dunque B' è ricorsivamente enumerabile. È importante notare che questo singolo programma è sufficiente a dimostrare l'enunciato per un singolo m fissato. Per dimostrare l'enunciato per ogni m è necessario considerare un numero infinito di programmi, uno per ogni m.

Esercizio. *Dimostrare che $Z = \{\langle x, y \rangle | x \in W_y\}$ è ricorsivamente enumerabile*

Possiamo sfruttare la definizione dell'insieme W per dimostrare che $\{\langle x, y \rangle | x \in W_y\}$ è ricorsivamente enumerabile.
Sappiamo infatti che $W_y = \{x \in \mathbb{N} | \Phi(x, y) \downarrow\}$. Abbiamo dunque che $Z = \{z | \Phi(l(z), r(z)) \downarrow\}$. Essendo $\Phi(l(z), r(z))$ composta di funzioni parzialmente calcolabili è anch'essa parzialmente calcolabile. Dunque Z è ricorsivamente enumerabile.

Esercizio.

1. Sia $B \subseteq \mathbb{N}$ ricorsivamente enumerabile, e $C \subseteq B$. È sempre vero che C è ricorsivamente enumerabile?
   NO. Si consideri $B = N$, chiaramente ricorsivamente enumerabile. Sia ora per assurdo vero che $\forall C \subseteq \mathbb{N}, C$ è ricorsivamente enumerabile. Ma allora anche l'insieme $\overline{K}$ o l'insieme $H = \{x \in \mathbb{N} | HALT(x,x)\}$ dovrebbero essere ricorsivamente enumerabile. Avendo già dimostrato che questo è falso abbiamo l'assurdo.

2. Siano $B, C \in \mathbb{N}$ tali che $B \cup C$ è ricorsivamente enumerabile. È sempre vero che B e C sono ricorsivamente enumerabili?
   NO. Si consideri $B = K$ e $C = \overline{K}$. Avremo che $B \cup C = \mathbb{N}$ è ricorsivamente enumerabile. Ma C non è ricorsivamente enumerabile come già dimostrato.
   Si pensi alternativamente agli insiemi $B = \{x | HALT(x,x)\}$ e $C = \{x | \neg HALT(x,x)\} = \overline{B}$. Avremo che $B \cup C = \mathbb{N}$ è ricorsivamente enumerabile. Ma B e C non sono ricorsivamente enumerabile come già dimostrato.

Esercizio. *Dimostrare che non esiste una funzione calcolabile f tale che $f(x) = \Phi(x,x) + 1$ se $\Phi(x,x) \downarrow$*

Fissando un $x_0 \in \mathbb{N}$ qualsiasi possiamo esprimere una tale funzione f qualsiasi come:

$$f(x) = \begin{cases} \Phi(x,x) + 1, & \Phi(x,x) \downarrow \\ x_0, & \text{altrimenti} \end{cases} = \begin{cases} \Psi^{(1)}_{\mathcal{P}_x}(x) + 1, & \Psi^{(1)}_{\mathcal{P}_x}(x) \downarrow \\ x_0, & \text{altrimenti} \end{cases} \tag{4.15}$$

Questo funzione è dunque analoga alla funzione definita nel teorema 3.4 e dunque non è calcolabile.

Macchine di Turing e Automi a Stati Finiti

Per la valutazione della calcolabilità fino a ora ci siamo limitati a un unico modello di calcolo, ovvero il linguaggio S. Questo modello è stato scelto per la sua semplicità e per la sua capacità di calcolare tutte le funzioni calcolabili.

Nonostante ciò esistono altri modelli di calcolo che sono equivalenti a S, ovvero che calcolano le stesse funzioni calcolabili essendo però ancora più minimali.

Il linguaggio S infatti, nonostante possa sembrare già minimale, mette in realtà a disposizione operazioni non esattamente banali. Infatti una macchina che fa un incremento deve sapere ad esempio che al numero 99 segue 100, cosa che rappresentando i numeri come stringhe di caratteri da 0 a 9 non è banale.

Ciò nonostante quanto detto in precedenza rimane vero, ovvero che ogni funzione calcolabile è calcolabile da S e il linguaggio S è minimale nelle istruzioni.

La banalità delle istruzioni è raggiunta considerando i numeri in base 1, ovvero non più come stringhe di cifre da 0 a 9 ma come stringhe di 1. $(x \mapsto \underbrace{1, \cdots, 1}_{x})$

Interpretando in questo modo i numeri, le operazioni diventano banali, ad esempio l'incremento diventa aggiungere un 1 alla fine della stringa e il decremento rimuoverne uno.

La base 1 però ha molte limitazioni, ad esempio per quanto riguarda la rappresentazione, ogni numero avrebbe infatti bisogno di spazio n per essere rappresentato.

Per ovviare a questo problema è possibile definire linguaggi S_n **equivalenti a S** adatti a lavorare in base n arbitraria.

Questi linguaggi però condividono con il linguaggio S il difetto di non essere pienamente generali, poiché presuppongono di avere come input e output numeri naturali che, per quanto possano codificare ogni insieme con cardinalità minore di $\mathbb{R}$, limitano la generalità del modello.

5.1 Macchine di Turing

Uno dei primi modelli di calcolo fu stato proposto da Alan Turing nel 1936. Questo modello, equivalente anch'esso al linguaggio S, è detto **Macchina di Turing**.

Una macchina di Turing può essere immaginata come una macchina con un nastro infinito, su cui sono scritti dei simboli. La macchina può a ogni istante in base alle istruzioni fornite cambiare cosa c'è scritto sul nastro e spostarsi a destra o a sinistra o cambiare stato, altra cosa che caratterizza queste macchine.

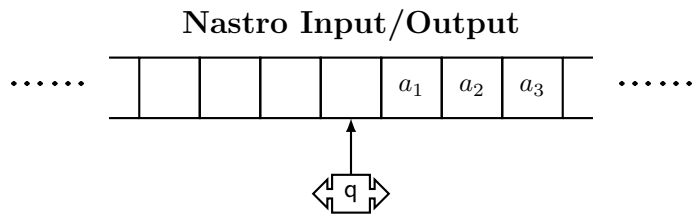


Figura 5.1: Rappresentazione grafica del nastro di una macchina di Turing

Macchina di Turing

Una macchina di Turing è una quadrupla $(Q, A, I, q_1 \in Q)$ dove:

- Q è un insieme finito (di stati);
- A è un insieme finito (Alfabeto), che non contenga i simboli $\{\sqcup, L, R\}$;
- $I \subseteq Q \times (A \cup \{\sqcup\}) \times (A \cup \{\sqcup, L, R\}) \times Q$, è l'insieme delle istruzioni. Queste sono quadruple in cui i primi due valori ci danno stato e valore scritto da verificare e i secondi due ci danno il valore da scrivere e il nuovo stato;
- q_1 è lo stato iniziale.

Come è già chiaro dalla definizione delle macchine di Turing queste sono più generali del linguaggio S, non presupponendo un dominio specifico per l'input e l'output, e definendo inoltre operazioni banali a prescindere dalla rappresentazione dei dati.

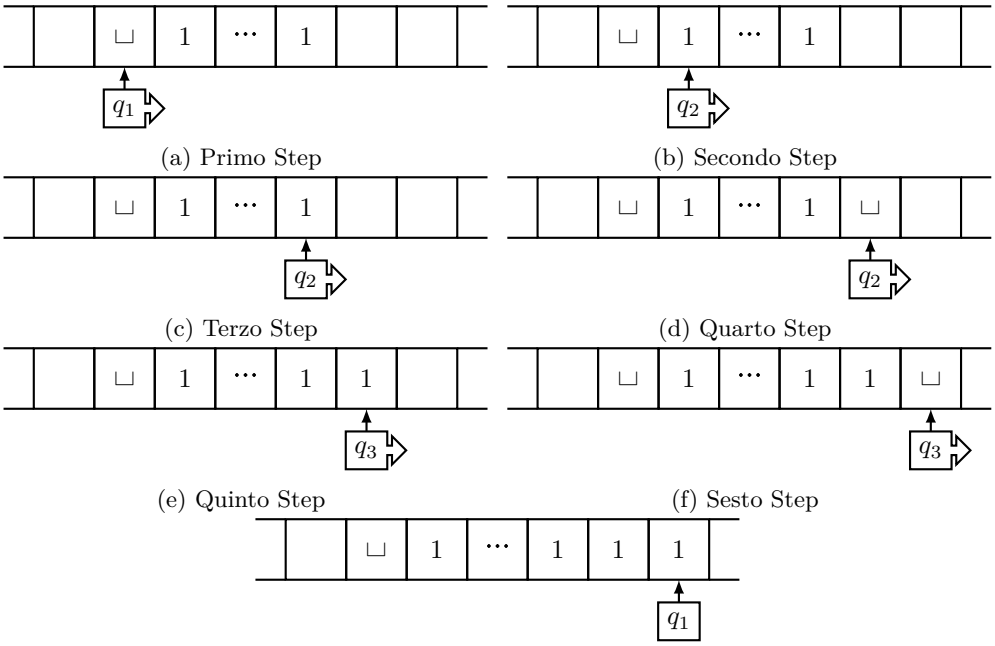
Per ottenere l'output di una macchina di Turing $\mathcal{M}$ rispetto a un input esprimibile come stringa su A, facciamo sì che la macchina parta dallo stato q_1 su un "nastro" vuoto tranne che nelle posizioni immediatamente a destra di quella di $\mathcal{M}$, occupate dai caratteri della stringa di input.

L'esecuzione continuerà finché in I saranno presenti istruzioni con stato e carattere di controllo corrispondenti alla situazione attuale (stato e simbolo correnti). L'output esiste se l'esecuzione termina (e in tal caso consisterà in ciò che è rimasto scritto sul nastro).

Essendo questo processo come detto generale e del tutto indipendente dalla tipologia di dominio di input e output, questo modo di procedere è perfettamente valido per le funzioni in $\mathbb{N}$.

Consideriamo una macchina di Turing $\mathcal{M}$ unaria con dominio $\mathbb{N}$ in base 1, ovvero si comporti come il linguaggio S.

$$\mathcal{M} = (Q, A, I, q_n) : Q = \{q_1, q_2, q_3\}, A = \{1\}, I = \{(q_1, \sqcup, R, q_2), (q_2, 1, R, q_2), (q_2, \sqcup, 1, q_3), (q_3, 1, R, q_3), (q_3, \sqcup, 1, q_1)\}$$



(a) Primo Step (b) Secondo Step (c) Terzo Step (d) Quarto Step (e) Quinto Step (f) Sesto Step (g) Settimo Step

Questa macchina di Turing calcola la funzione $f(n) = n + 2$, poiché su un input generico aggiungerà sempre due volte '1'.

Le macchine di Turing possono calcolare anche funzioni non unarie, separando gli input con degli spazi vuoti.

Questo modello di calcolo ci suggerisce che il dominio delle stringe renda il calcolo ancora più elementare e astratto essendo ogni numero rappresentabile come una stringa di caratteri.

In questo passaggio di astrazione andremo a chiamare:

- A^* l'insieme delle stringhe su A, compresa ε (stringa vuota);
- $L \subseteq A^*$ un linguaggio su A.

Diremo che un linguaggio L è ricorsivamente enumerabile se esiste una macchina di Turing $\mathcal{M}$ che termina se e solo se l'input appartiene a L. Essendo i linguaggi generalizzazione dei sottoinsiemi di $\mathbb{N}$, questa definizione comprende quella data per i sottoinsiemi di $\mathbb{N}$. Diremo che un linguaggio L è ricorsivo se L e $\overline{L} = A^* \setminus L$ sono ricorsivamente enumerabili.

Consideriamo ora la versione più limitata possibile di macchina di Turing, ovvero che possa solo scorrere in una direzione o cambiare stato. Queste macchine sono dette **automi a stati finiti**.

Un Automa a Stati Finiti è una macchina che legge un input e può solo cambiare stato di conseguenza.

Questo tipo di macchine non producono output, e possono dunque esclusivamente *accettare*, ovvero terminare, o meno una parola. Per questo motivo vengono dunque detti **accettori** e hanno solo due configurazioni finali, quella di **accettazione** e quella di **rifiuto**.

Andiamo dunque a definire formalmente un primo tipo di **accettore**, gli **Automi Finiti Deterministici**.

5.2.1 Automi Finiti Deterministici

Automi Finiti Deterministici

Un automa finito deterministico (DFA) è una quintupla (Q, A, δ, q_1, F) dove:
- Q è un insieme finito (di stati);
- A è un insieme finito (Alfabeto);
- $\delta : Q \times A \to Q$ è una funzione detta di transizione;
- $q_1 \in Q$ è lo stato iniziale;
- $F \subseteq Q$ è l'insieme degli stati terminali o di accettazione.

Questi automi sono detti deterministici poiché, grazie alla funzione δ, non esistono transizioni ambigue, ovvero per ogni stato e simbolo di input è definito un unico stato successivo. In oltre, dalle proprietà delle applicazioni, sappiamo che per ogni stato dev'essere associata un istruzione per simbolo di input. Definiamo dunque per ricorrenza la funzione di transizione iterata come:

$$\delta^* : Q \times A^* \to Q$$
$$\delta^*(q, w) = \begin{cases} \delta^*(q, \varepsilon) = q, \forall q \in Q \\ \delta^*(q, wa) = \delta(\delta^*(q, w), a). \end{cases} \tag{5.1}$$

Diremo quindi che un linguaggio L è accettato da una DFA $\mathcal{M}$ *se e solo se* $L = L(\mathcal{M}) = \{w \in A^* | \delta^*(q_1, w) \in F\}$, dove per $L(\mathcal{M})$ si intende l'insieme delle stringhe accettate da $\mathcal{M}$.

Esempio: DFA

Sia $\mathcal{M} = (Q, A, \delta, q_1, F)$ con $Q = \{q_1, q_2, q_3, q_4\}$, $A = \{a, b\}$, $F = \{q_4\}$ e δ definita dalla seguente tabella:

δ	a	b
q_1	q_2	q_3
q_2	q_2	q_4
q_3	q_3	q_3
q_4	q_3	q_4

Possiamo subito notare che q_3 non è uno stato terminale ma una volta raggiunto non è possibile uscirne, questo tipo di stato è detto **stato pozzo**.
Da questo possiamo vedere che le parole accettate **devono necessariamente iniziare** per a poiché altrimenti entreremo nel pozzo immediatamente. Inoltre essendo q_1 **non terminale** non possiamo accettare la stringa vuota.
Dovendo necessariamente avere una lettera a come iniziale, arriveremo necessariamente nello stato q_2. Da qui, sia a che b saranno accettate. Infatti per a rimarremo in q_2 e per b andremo in q_4 che è uno stato terminale, in cui potremo rimanere finché non arriverà una a che porta nel **pozzo**.
Seguendo quindi le transizioni possiamo notare che le parole accettate da $\mathcal{M}$ sono quelle formate da una serie di a seguite da una serie di b. Ovvero $L(\mathcal{M}) = \{a^n b^m | n, m > 0\}$.

Per denotare la concatenazione successiva di parole identiche, utilizzeremo la notazione di elevamento a potenza. La notazione varia rispetto al testo di riferimento, il quale utilizza parentesi quadre aggiuntive per indicare l'esponente.

In generale se $w \in A^*$, avremo che:

$$\begin{cases} w^0 = \varepsilon \\ w^{n+1} = ww^n \end{cases} \tag{5.2}$$

Diagramma di Transizione di stato di un DFA

Esaminiamo adesso un approccio alternativo per la rappresentazione di tali automi. Prendiamo in considerazione l'esempio precedentemente menzionato 5.2.1. Un metodo per rappresentare le transizioni di stato consiste nel disegnare un grafo orientato in cui ogni stato è un nodo e ogni transizione è un arco. Ad esempio avremo:

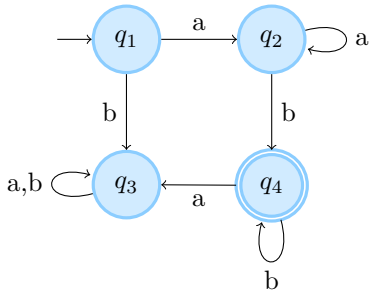


Figura 5.3: Diagramma di Transizione di Stato di un DFA

In questa rappresentazione un qualsiasi $q \in Q$ avrà il doppio cerchio *se e solo se* $q \in F$. Mentre lo stato iniziale è indicato con una freccia entrante nello stato senza stato di partenza.

Ora che sappiamo come rappresentare graficamente un DFA, possiamo caratterizzare i linguaggi sono accettati da tali DFA, chiamati **linguaggi regolari**.

5.2.2 Linguaggi Regolari

Linguaggi Regolari

Un linguaggio $L \subseteq A^*$ è regolare se esiste una DFA $\mathcal{M}$ su A tale che $L = L(\mathcal{M})$.

Dalla definizione appena data, i **linguaggi regolari** sembrano essere strettamente legati all'alfabeto su cui sono definiti. Tuttavia, come vedremo, questo non è il caso, poiché la regolarità di un linguaggio è mantenuta in tutti gli alfabeti che contengono quello in cui è definito.

Indipendenza dei Linguaggi Regolari dall'alfabeto di supporto

Enunciato. Sia $A \subseteq A'$ e $L \subseteq A^*$. Allora

$$\exists \mathcal{M} = (Q, A, \delta, q_1, F) : L = L(\mathcal{M}) \iff \exists \mathcal{M}' = (Q', A', \delta', q_1', F') : L = L(\mathcal{M}') \tag{5.3}$$

Dimostrazione. Dimostriamo la doppia implicazione:

"$\Rightarrow$" Se $L = L(\mathcal{M})$, definiamo $Q' = Q \cup \{q_p\}$, dove $q_p \notin Q$ è uno stato pozzo, $q_1' = q_1, F' = F$ e δ' come:

$$\delta'(q, a) = \begin{cases} \delta(q, a), & q \in Q \wedge a \in A \\ q_p, \text{altrimenti} & \end{cases} \tag{5.4}$$

Avremo che $\mathcal{M}'$ si comporterà esattamente come $\mathcal{M}$ per tutte le parole di A'^* che appartengono anche a A^*, mentre per tutte le altre non arriverà mai a terminazione.

"$\Leftarrow$" Se $L = L(\mathcal{M}')$, definiamo $Q = Q', F = F', q_1 = q_1'$ e δ come la restrizione di δ' su A, ovvero:

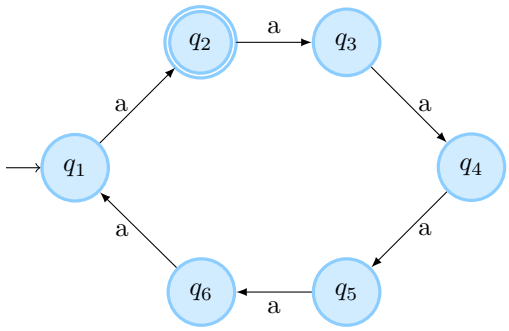
$$\delta(q, a) = \delta'(q, a), \forall q \in Q, a \in A \tag{5.5}$$

Avremo che $\mathcal{M}$ accetterà tutte le parole accettate da $\mathcal{M}'$ composte unicamente da simboli in A. Essendo però che $L \subseteq A^*$, avremo che $\mathcal{M}$ accetterà tutte le parole di L.

Consideriamo ora un esempio di linguaggio regolare per meglio comprendere la definizione appena data.

Esempio: Linguaggio regolare

Si consideri il linguaggio su $A = \{a\}$ definito da $L = \left\{a^{6k+1} | k \in \mathbb{N}\right\}$. Questo corrisponde a tutti i numeri che hanno resto 1 nella divisione per 6. Per dimostrare che questo linguaggio è regolare, possiamo dunque costruire un DFA che lo accetti come segue:



Avremo che infatti una qualsiasi parola di A^* sarà accettata *se e solo se* farà esattamente un numero qualsiasi di "giri completi" del diagramma e termini con un "passo aggiuntivo" per raggiungere q_2, in poche parole se la parola avrà lunghezza $l \equiv_6 1$.

Proprietà e caratterizzazione dei linguaggi regolari

Definiti bene i **linguaggi regolari** possiamo andare a descriverne alcune proprietà.

In primo luogo, possiamo vedere come ogni linguaggio $L \subseteq A^*$ regolare è anche un insieme **ricorsivamente enumerabile**[1].

Linguaggi regolari e ricorsivo enumerabilità

Enunciato. $L \subseteq A^*$ **regolare** $\implies$ L **ricorsivamente enumerabile**.

Dimostrazione. L è ricorsivamente enumerabile *se e solo se* esiste una macchina di Turing $\mathcal{M}'$ che termina solo su input in L[a].
Sia $\mathcal{M} = (Q, A, \delta, q_1, F)$ una DFA tale che $L = L(\mathcal{M})$. Allora definiamo $Q' = Q \cup \{q_0\}$, con $q_0 \notin Q$ che funga da stato iniziale della macchina di Turing[b] e poniamo

$$I = \{(q_0, \sqcup, R, q_1)\} \cup \{(q_1, a, R, \delta(q, a)) | q \in Q, a \in A\} \cup \{(q, \sqcup, R, q_0) | q \in Q \setminus F\} \tag{5.6}$$

Dove $(q_0, \sqcup, R, q_1)$ serve a far iniziare la computazione alla macchina, $\{(q_1, a, R, \delta(q, a)) | q \in Q, a \in A\}$ serve a farla continuare seguendo i percorsi di $\mathcal{M}$ e $\{(q, \sqcup, R, q_0) | q \in Q \setminus F\}$ serve a non far terminare la computazione in caso la stringa sia terminata ma non si sia raggiunto uno stato terminale.

[a] Importante prestare attenzione al fatto che si parli di un implicazione singola, ergo non vale necessariamente l'opposta

[b] Si ricorda che per definizione una macchina di Turing presuppone che di iniziare da una cella vuota con l'input che inizia immediatamente alla sua destra.

Possiamo però caratterizzare i linguaggi regolari in maniera più profonda. Iniziamo però prima da dimostrare una loro proprietà che ci aiuterà nella caratterizzazione.

Regolarità del Complemento

Enunciato. $L \subseteq A^*$ regolare $\iff$ $\overline{L} = A^* \setminus L$ è regolare.

Dimostrazione. Se $L = L(\mathcal{M})$ con $\mathcal{M} = (Q, A, \delta, q_1, F)$ DFA, allora $\overline{L} = L(\mathcal{M}')$ con $\mathcal{M}' = (Q, A, \delta, q_1, Q \setminus F)$. L'implicazione inversa è analoga.

Sfruttando questa proprietà, il teorema dimostrato subito prima e la caratterizzazione degli insiemi ricorsivi4.1.2, possiamo dedurre che L è regolare $\implies$ L è ricorsivo.

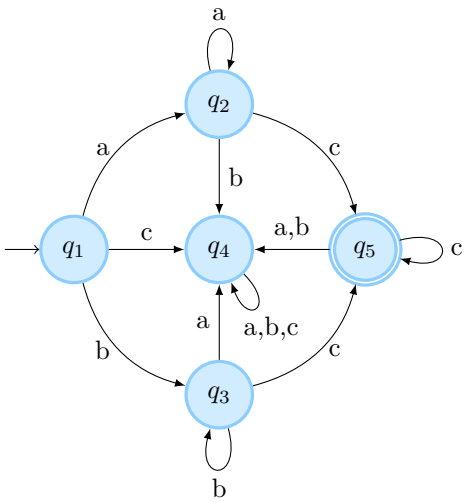
[1] Essendo $L \subseteq A^*$ non possiamo utilizzare la nozione di ricorsivo enumerabilità relativa ai sottoinsiemi di $\mathbb{N}$ che sfrutta il Linguaggio S, pensi quella più generale ma equivalente fornitaci dalle macchine di Turing

Esempio: Sintesi del Linguaggio accettato da un Automa

Fino a ora abbiamo analizzato il caso in cui, dato un linguaggio abbiamo *sintetizzato* il diagramma di un automa che lo accetti. È però possibile anche fare il processo opposto. Vediamone un esempio.
Sia $\mathcal{M}$ il DFA su $A = \{a, b, c\}$ con $Q = \{q_1, q_2, q_3, q_4, q_5\}, F = \{q_5\}$ e δ dato da:

δ	a	b	c
q_1	q_2	q_3	q_4
q_2	q_2	q_4	q_5
q_3	q_4	q_3	q_5
q_4	q_4	q_4	q_4
q_5	q_4	q_4	q_5

Andiamo dunque a disegnare il diagramma di questo automa:



Per determinare il linguaggio accettato da questo automa ci basterà ricostruire tutti i percorsi che portano da q_1 a q_5.
In particolare avremo che se iniziamo con una a, per arrivare in q_5 possiamo avere un altro numero arbitrario di a dal cappio in q_2, almeno una c per raggiungere q_5 e poi un altro numero arbitrario di c dal cappio in q_5.
Alternativamente partendo con una b avremo un numero arbitrario di b dal cappio in q_3, almeno una c per raggiungere q_5 e poi un altro numero arbitrario di c dal cappio in q_5.
Tutti gli altri percorsi non sono percorribili poiché passano per q_4 che è uno stato pozzo.
Avremo dunque che il linguaggio accettato da $\mathcal{M}$ sarà $L(\mathcal{M}) = \{a^n c^m | n, m > 0\} \cup \{b^n c^m | n, m > 0\}$.

5.2.3 Automi Finiti Non Deterministici

Automi Finiti Non Deterministici

Un automa finito non deterministico (NDFA o NFA) è una quintupla $\mathcal{M} = (Q, A, \delta, q_1, F)$ dove:
- Q e A sono insiemi finiti;
- $q_1 \in Q$
- $F \subseteq Q$
- $Q \times A \mapsto \mathbb{P}(Q)$ funzione di transizione (totale).

Avremo dunque che, partendo da un qualsiasi stato con la parola vuota assoceremo il singleton di quello stato, mentre con una parola generica wa andremo ad associare l'unione di tutti gli stati raggiungibili con a partendo da quelli raggiunti con w da q.

Per dare un senso a questa definizione però è necessario identificare quali linguaggi vengono accettati da una NFA. Analogamente a come fatto per i DFA utilizziamo la funzione δ iterata.

$$\begin{cases} \delta^*(q, \varepsilon) = \{q\}, & \forall q \in Q \\ \delta^*(q, wa) = \bigcup\limits_{q' \in \delta^*(q,w)} \delta(q', a) \end{cases} \tag{5.7}$$

Possiamo infine definire $L \subseteq A^*$ come accettato da una NFA *se e solo se* $L = L(\mathcal{M}) = \left\{w \in A^* \middle| \delta^*(q_1, w) \cap F \neq \varnothing\right\}$.

Diagramma di Transizione di stato di un NFA

Come con i DFA è possibile rappresentare i NFA utilizzando dei diagrammi di passaggi di stato. La differenza principale è che in un NFA è possibile avere più di una transizione per stato e simbolo di input, come non averne affatto.

Vediamo un esempio di NFA. Consideriamo il linguaggio $L = \{w \in \{a,b\}^* | w \text{ contiene } aa \text{ oppure } bb\}$.

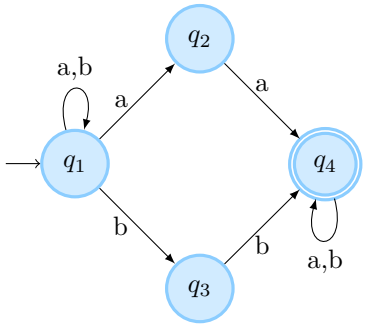


Figura 5.4: Diagramma di Transizione di Stato di un NFA

5.2.4 Equivalenza tra DFA e NFA

Nonostante i DFA e i NFA siano due modelli di automi molto diversi, e si potrebbe pensare che i linguaggi accettati da questi due modelli siano diversi e, in particolare, essendo le NFA meno stringenti e più facili da elaborare, che queste accettino più linguaggi delle DFA.

Questo non è il caso però, infatti, come vedremo, ogni linguaggio accettato da una NFA è accettato anche da una DFA e viceversa. Questo ci permette di affermare che i due modelli sono equivalenti, ovvero che accettano esattamente gli stessi linguaggi.

Corrispondenza tra DFA e NFA

Enunciato. $L \subseteq A^*$ regolare $\iff$ $L = L(\mathcal{M})$ per qualche NDFA $\mathcal{M}$.

Dimostrazione. Dimostriamo la doppia implicazione:

"$\Rightarrow$" Sia L regolare. Allora $\exists \mathcal{M}' = (Q, A, \delta', q_1, F)$ DFA : $L = L(\mathcal{M}')$. Allora $L = L(\mathcal{M})$ dove $\mathcal{M} = (Q, A, \delta, q_1, F)$ è l'NFA definita da $\delta(q,a) = \{\delta'(q,a)\}$.

"$\Leftarrow$" Sia $L = L(\mathcal{M})$ con $\mathcal{M}(Q, A, \delta, q_1, F)$ NFA. Definiamo allora $\mathcal{M}' = (\mathbb{P}(Q), A, \delta', \{q_1\}, \mathbb{F})$ dove

$$\begin{aligned} \mathbb{F} &= \{Q' \subseteq Q | Q' \cap F \neq \varnothing\} \\ \delta'(Q', a) &= \bigcup_{q \in Q'} \delta(q,a) \forall Q' \subseteq Q, a \in A \end{aligned} \tag{5.8}$$

Si può dimostrare, anche se non lo vedremo allora che $\delta'^*(Q', w) = \bigcup_{q \in Q'} \delta^*(q, w)$

Allora $L(\mathcal{M}) = \{w \in A^* | \delta'^*(\{q_1\}, w) \in \mathbb{F}\} = \{w \in A^* | \delta^*(\{q_1\}, w) \in \mathbb{F}\} = \{w \in A^* | \delta^*(q_1, w) \cap F \neq \varnothing\} = L(\mathcal{M})$.

Questa equivalenza è fondamentale poiché ci permette di trattare ogni NFA, più semplice da concepire e rappresentare, come un DFA, più semplice da implementare con una vera macchina non dovendo fare scelte arbitrarie, e viceversa.

5.3 Esercizi ed approfondimenti

Esercizio. *Consideriamo l'alfabeto $A = \{a, b\}$ dimostrare che sono regolari i linguaggi:*

1. Stringhe che terminano con *bbab*

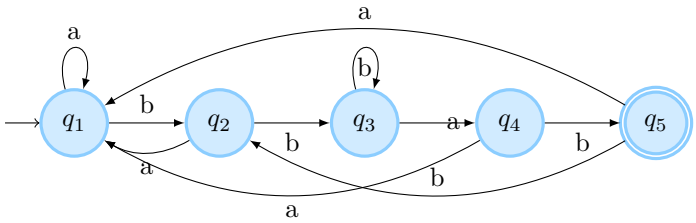
   Per dimostrare che questo linguaggio è regolare possiamo costruire un automa a stati finiti che lo riconosca.

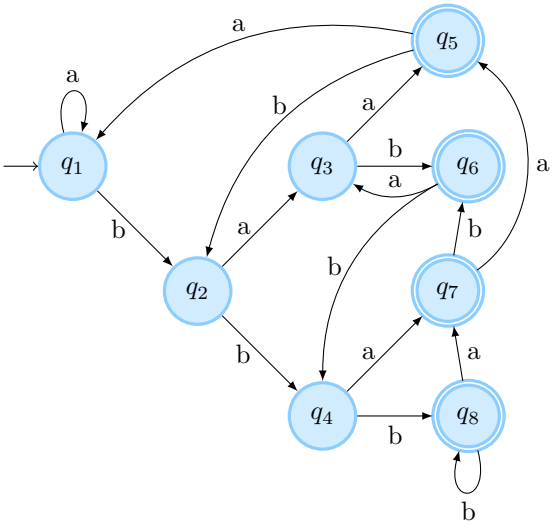
2. Parole $w = s_1 \cdots s_n$ tali che $s_{n-2} = b$

   Per dimostrare che questo linguaggio è regolare possiamo costruire un automa a stati finiti che lo riconosca.

3. Parole che **non** contengono *aaa*

   Per dimostrare che questo linguaggio è regolare possiamo costruire un automa a stati finiti che lo riconosca.





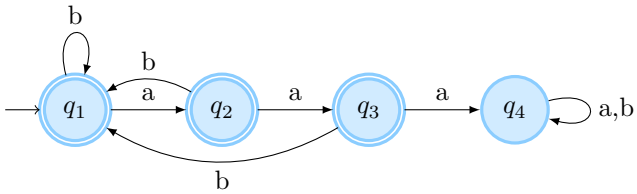
Esercizio. *Considerando l'esempio 5.4, ricavare la tabella di transizione dell'automa non deterministica e disegnare il diagramma usando un DFA*

Esercizio. *Prendendo in considerazione l'esempio 5.2.2, ricavare il linguaggio accettato dall'automa con* $F' = \{q_4\}$

Per ottenere il linguaggio accettato dall'automa sarà sufficiente seguire tutti i percorsi che arrivano allo stato q_4 essendo che questo è l'unico stato d'accettazione e che ogni lettera successiva all'arrivo in q_4 ci farà rimanere in q_4.

Si avrà che, considerando $A = a, b, c$ il linguaggio accettato dall'automa è:

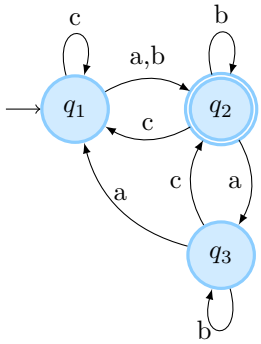
$$\begin{aligned} L(\mathcal{M}) = &\{cw | w \in A^*\} \cup \{a^n bw | n > 0, w \in A^*\} \cup \{b^n aw | n > 0, w \in A^*\} \cup \\ &\{a^n c^m xw | n, m > 0, w \in A^*, x \in \{a, b\}\} \cup \{b^n c^m xw | n, m > 0, w \in A^*, x \in \{a, b\}\} \end{aligned} \tag{5.9}$$



Esercizio. *Ricavare il linguaggio accettato dal DFA rappresentato dalla seguente tabella avente* $F = \{q_2\}$

δ	a	b	c
q_1	q_2	q_2	q_1
q_2	q_3	q_2	q_1
q_3	q_1	q_3	q_2

Innanzitutto procediamo a disegnare il diagramma dell'automa.



Possiamo ora ricavare il linguaggio accettato dall'automa. Seguendo i percorsi che arrivano allo stato q_2 avremo che il linguaggio accettato dall'automa conterrà solo parole che iniziano con un numero positivo di c e poi una a o una b.

Avremo dunque che $L(\mathcal{M}) = \{c^n xw | n \in \mathbb{N}, x \in \{a,b\}, w \in L'\}$, dove L' è l'insieme delle parole che permettono nell'automa di tornare a q_2 partendo da q_2 stesso.

Quest'ultimo sarà dunque una qualsiasi combinazione delle parole che permettono di compiere un ciclo singolo da q_2 a q_2, ovvero $L' = \{u_1 u_2 \cdots u_n | n \in \mathcal{N}, \forall i \in \{1, \cdots, n\}, u_i \in L''\}$, dove L'' è l'insieme delle parole che permettono di compiere un ciclo semplice da q_2 a q_2.

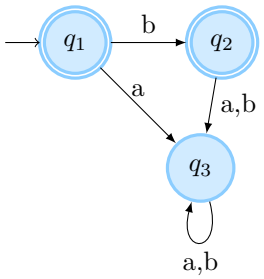
Nello specifico avremo che $L'' = \{b\} \cup \{ab^n c | n \in \mathbb{N}\} \cup \{c^m x | m > 0, x \in \{a,b\}\} \cup \{ab^n ac^m x | n, m \in \mathcal{N}, x \in \{a,b\}\}$.

Riassumendo si avrà che:

$$\begin{aligned} L(\mathcal{M}) =& \{c^n xw | n \in \mathbb{N}, x \in \{a,b\}, w \in L'\} \\ \text{dove } L' =& \{u_1 u_2 \cdots u_n | n \in \mathcal{N}, \forall i \in \{1, \cdots, n\}, u_i \in L''\} \\ \text{dove } L'' =& \{b\} \cup \{ab^n c | n \in \mathbb{N}\} \cup \{c^m x | m > 0, x \in \{a,b\}\} \cup \{ab^n ac^m x | n, m \in \mathcal{N}, x \in \{a,b\}\} \end{aligned} \tag{5.10}$$

Esercizio. *Mostrare che esiste un linguaggio regolare non accettato da una DFA con uno stato terminale*

Si consideri il linguaggio $L = \{\varepsilon, b\}$. Questo linguaggio è regolare poichè è accettato dall'automa a stati finiti:



Per questo linguaggio non esiste un DFA con uno stato terminale che lo accetti. Infatti per accettare ε è necessario che lo stato iniziale sia anche terminale. D'altro canto per accettare b c'è bisogno di una transizione che valuti b dallo stato iniziale ad

uno stato terminale. Se ci fosse un unico stato terminale questo sarebbe quello iniziale e dunque la transizione in questione un cappio, che però renderebbe accettate tutte le parole della forma b^n con $n \in \mathcal{N}$.

Operazioni sui Linguaggi regolari

Per descrivere completamente la classe dei linguaggi regolari, oltre alla definizione di automa, è necessario introdurre alcune operazioni che permettono di costruire nuovi linguaggi regolari a partire da altri.

In particolare vedremo come i linguaggi regolari siano chiusi rispetto alle operazioni di unione, concatenazione e iterazione.

Per dimostrare ciò più agilmente però, ci sarà necessaria una nuova classe di automi, equivalente ai DFA, ovvero i **DFA non-restarting**.

6.1 DFA Non-Restarting

DFA Non-restarting

Un DFA $\mathcal{M}$ è **Non-Restarting** se $\mathcal{M} = (Q, A, \delta, q_1, F)$ con $q_1 \notin \delta(QxA)$, ovvero non è possibile tornare allo stato iniziale dopo aver letto un simbolo di input.

Una definizione analoga può essere data per gli NFA, sincerandosi che $q_1 \notin \bigcup_{Q' \in \delta(Q \times A)} Q'$.

Come anticipato, vediamo ora come questa classe di automi sia equivalente a quella dei DFA

DFA Non-Restarting

Enunciato. $L \subseteq A$ regolare $\implies$ esiste un DFA **non-restarting** $\mathcal{M}'$ tale che $L = L(\mathcal{M}')$.

Dimostrazione. Sia $\mathcal{M} = (Q, A, \delta, q_1, F)$ un DFA tale che $L = L(\mathcal{M})$ e sia $q_0 \notin Q$.
Definiamo $\mathcal{M}' = (Q \cup \{q_0\}, A, \delta', q_0, F')$ dove δ' e F' sono rispettivamente:

$$\delta'(q, a) = \begin{cases} \delta(q, a), & q \in Q \wedge a \in A \\ \delta(q_1, a), & \text{altrimenti} \end{cases} \tag{6.1}$$

Che è quindi identica a δ tranne per q_0 che emula l'uscita da q_1.

$$F' = \begin{cases} F, & q_1 \notin F \\ F \cup \{q_0\}, & q_1 \in F \end{cases} \tag{6.2}$$

Evidentemente $\mathcal{M}'$ è un DFA **non-restarting** e $L(\mathcal{M}') = L(\mathcal{M})$.

6.2 Chiusura dei Linguaggi Regolari

6.2.1 Unione

Grazie a questa nuova classe di automi possiamo dimostrare le prime proprietà di chiusura dei linguaggi regolari. Iniziamo vedendo come la classe dei linguaggi regolari sia chiusa rispetto all'operazione di **unione**.

Regolarità dell'Unione

Enunciato. Siano $L, L' \subseteq A^*$ regolari. Allora $L \cup L'$ è regolare.

Dimostrazione. Siano $\mathcal{M} = (Q, A, \delta, q_1, F)$ e $\mathcal{M}' = (Q', A, \delta', q_1', F')$ DFA **non-restarting** tali che $L = L(\mathcal{M})$ e $L' = L(\mathcal{M}')$ con $Q \cap Q' = \varnothing$.
Definiamo un NFA $\widehat{\mathcal{M}} = (Q \cup Q' \setminus \{q_1, q_1'\} \cup \{q_0\}, A, \widehat{\delta}, q_0, \widehat{F})$ con

$$\widehat{F} = \begin{cases} F \cup F', & q_1 \notin F \wedge q_1' \notin F' \\ F \cup F' \setminus \{q_1, q_1'\} \cup \{q_0\}, & q_1 \in F \vee q_1' \in F' \end{cases}$$
$$\forall a \in A : \widehat{\delta}(q, a) = \begin{cases} \{\delta(q, a)\}, & q \in Q \\ \{\delta'(q, a)\}, & q \in Q' \\ \{\delta(q_1, a), \delta'(q_1', a)\}, & q = q_0 \end{cases} \tag{6.3}$$

È evidente che il linguaggio accettato da questo NFA è $L(\widehat{\mathcal{M}}) = L(\mathcal{M}) \cup L(\mathcal{M}') = L \cup L'$.

È possibile ora facilmente vedere come i linguaggi regolari siano chiusi anche rispetto all'intersezione.

Corollario: Regolarità dell'Intersezione

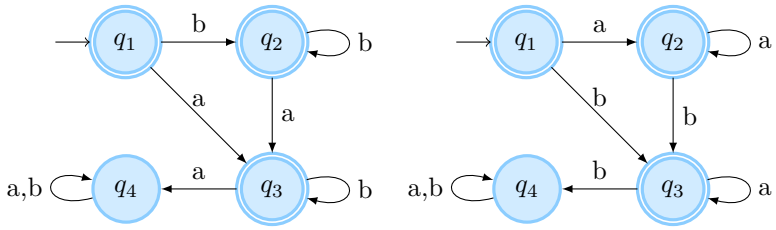
Enunciato. Siano $L, L' \subseteq A^*$ regolari. Allora $L \cap L'$ è regolare.

Dimostrazione. Si ha $L \cap L' = \overline{\left(\overline{L} \cup \overline{L'}\right)}$. Dai teoremi 5.2.2 e 6.2.1 abbiamo la tesi.

Vediamo ora un applicazione pratica di questi due teoremi.

Si consideri $L_1 = \{w \in \{a, b\} * | w$ contiene almeno 2 a$\}$ e $L_1 = \{w \in \{a, b\} * | w$ contiene almeno 2 b$\}$. Si consideri gli automi ottenuti nell'esercizio 6.5.

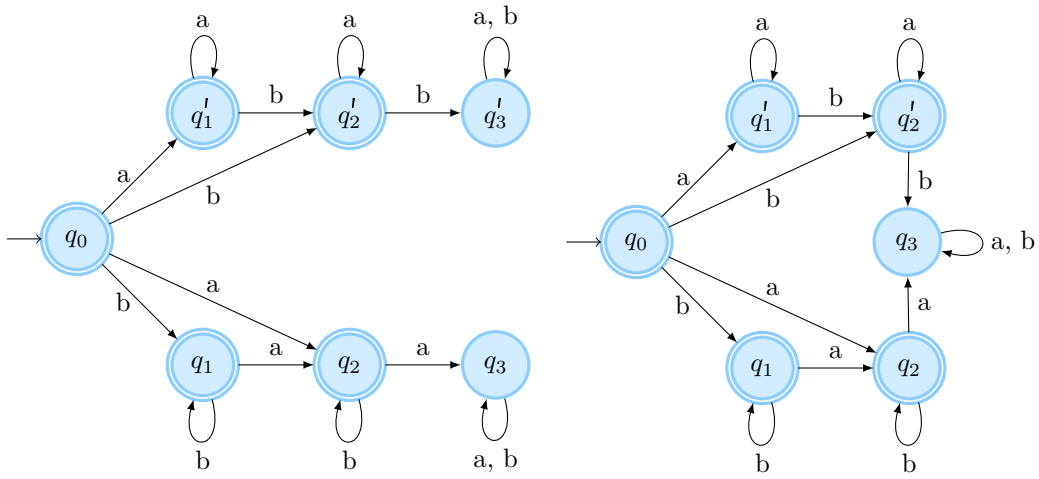
Avremo che $\overline{L_1} = L(\mathcal{M}_1)$ e $\overline{L_2} = L(\mathcal{M}_2)$ dove $\mathcal{M}_1$ e $\mathcal{M}_2$ sono rispettivamente:



(a) Diagramma di $\mathcal{M}_1$ (b) Diagramma di $\mathcal{M}_2$

Avremo che $\overline{L_1} \cup \overline{L_2} = L(\widehat{\mathcal{M}})$ dove $\widehat{\mathcal{M}}$ sarà:

A questo punto usando la tecnica usata nel teorema 5.2.4 possiamo ottenere un DFA da complementare.



(a) Diagramma di $\widehat{\mathcal{M}}$ (b) Diagramma di $\widehat{\mathcal{M}}$ semplificato

Andiamo ora a elencare alcuni risultati derivanti dalla chiusura dei linguaggi regolari rispetto alle operazioni appena viste.

Proposizione: Regolarità dell'insieme vuoto

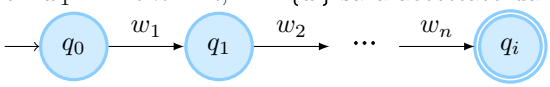
Enunciato. $\varnothing \subseteq A^*$ è regolare.

Dimostrazione. Accettato da qualsiasi DFA con $F = \varnothing$.

Proposizione: Regolarità dei singleton

Enunciato. $\forall w \in A^*, \{w\}$ è regolare.

Dimostrazione. Se $w = w_1 \cdots w_n$, con $w_1 \in A$ e $n \in \mathbb{N}$, $L = \{w\}$ sarà accettato dall'NFA con diagramma:



Corollario: Regolarità dei linguaggi finiti

Enunciato. L finito $\implies$ L regolare.

Dimostrazione. Tutti i linguaggi finiti sono unione finita di singleton. Dalla proposizione appena vista e dalla chiusura per unione abbiamo la tesi.

6.2.2 Prodotto per Concatenazione

Prodotto per Concatenazione

Se $L, L' \subseteq A^*$, definiamo:

$$LL' = \{ww' \in A^* | w \in L \wedge w' \in L'\} \tag{6.4}$$

È importante notare che questo linguaggio conterrà parole di uno dei due linguaggi originali *se e solo se* l'altro contiene ε.

Chiusura dei Linguaggi regolari per Prodotto

Enunciato. Siano $L, L' \subseteq A^*$ regolari. Allora LL' è regolare.

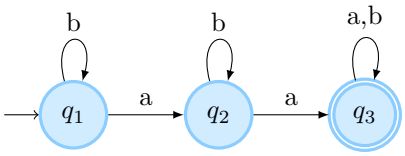
Dimostrazione. Siano $\mathcal{M} = (Q, A, \delta, q_1, F)$ e $\mathcal{M}' = (Q', A, \delta', q_1', F')$ DFA **non-restarting** tali che $L = L(\mathcal{M})$ e $L' = L(\mathcal{M}')$ con $Q \cap Q' = \varnothing$.
Definiamo un NFA $\widehat{\mathcal{M}} = (Q \cup Q', A, \hat{\delta}, q_1, \widehat{F})$ con

$$\widehat{F} = \begin{cases} F \cup F', & q_1' \in F' \\ F', & q_1' \notin F' \end{cases}$$
$$\forall a \in A : \hat{\delta}(q, a) = \begin{cases} \{\delta(q, a)\}, & q \in Q \setminus F \\ \{\delta'(q, a)\}, & q \in Q' \\ \{\delta(q, a), \delta'(q_1', a)\}, & q \in F \end{cases} \tag{6.5}$$

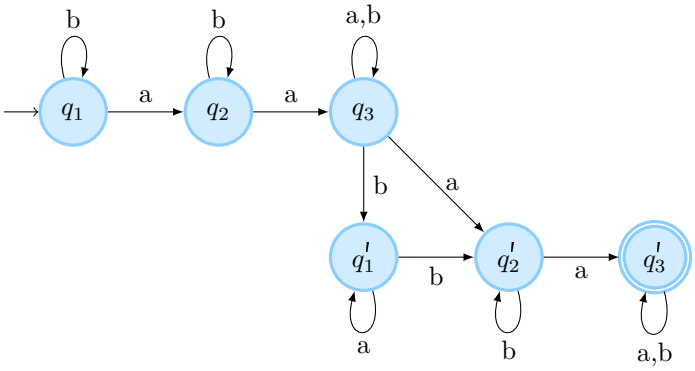
È evidente che il linguaggio accettato da questo NFA è $L(\widehat{\mathcal{M}}) = L(\mathcal{M})L(\mathcal{M}') = LL'$.

Esempio: Concatenazione di Linguaggi Regolari

Consideriamo ad esempio il linguaggio L_1L_1 con L_1 definito come in precedenza. Consideriamo due automi $\mathcal{M}$ e $\mathcal{M}'$ che accettano L_1, i quali avranno entrambi la forma:



Si avrà che il prodotto per concatenazione L_1L_1 sarà accettato dall'automa $\widehat{\mathcal{M}}$:



In particolare questo prodotto è un esempio di prodotto per concatenazione di un linguaggio per se stesso. Possiamo in generale definire dal prodotto l'operazione potenza di un linguaggio come:

$$\begin{cases} L^0 = \{\varepsilon\} \\ L^{n+1} = LL^n \end{cases} \tag{6.6}$$

6.2.3 Iterazione

Iterazione (Star)

Se $L \subseteq A^*$, definiamo:

$$L^* = \{w_1 \cdots w_n | n \in \mathbb{N}, w_i \in L\} = \bigcup_{n \in \mathbb{N}}^{\infty} L^n \tag{6.7}$$

Essendo l'operazione Star definita tramite uso di un unione infinita di linguaggi regolari, non possiamo ricavarla direttamente come applicazione finita di operazioni di unione e prodotto. Per questo è necessario definire separatamente questa operazione.

Chiusura dei Linguaggi regolari per Iterazione

Enunciato. Sia $L \subseteq A^*$ regolare. Allora L^* è regolare.

Dimostrazione. Sia $\mathcal{M} = (Q, A, \delta, q_1, F)$ un DFA **non-restarting** tale che $L = L(\mathcal{M})$.
Definiamo un NFA $\widehat{\mathcal{M}} = (Q, A, \widehat{\delta}, q_1, \{q_1\})$ con:

$$\forall a \in A : \widehat{\delta}(q, a) = \begin{cases} \{\delta(q, a)\}, & \delta(q, a) \notin F \\ \{\delta(q, a), q_1\}, & \text{altrimenti} \end{cases} \tag{6.8}$$

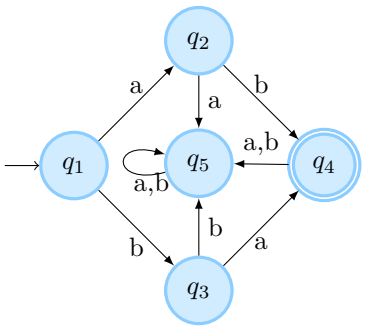
Per vedere come $L^* = L(\mathcal{M})$ verifichiamo la doppia inclusione.

"$\subseteq$" Verifichiamo per induzione che $L^* \subseteq L(\widehat{\mathcal{M}})$. Come base induttiva possiamo facilmente notare che $\varepsilon \in L(\widehat{\mathcal{M}})$. Per quanto riguarda il passo induttivo invece supponiamo $L^{n-1} \subseteq L(\widehat{\mathcal{M}})$ e mostriamo che $L^n \subseteq L(\widehat{\mathcal{M}})$.
Siano $u \in L^{n-1}, v \in L$ tali che $w = uv$. Si ha $q_1 \in \widehat{\delta}^*(q_1, u)$ per ipotesi induttiva. Ma allora essendo $v \in L$ partendo da q_1 si tornerà in q_1 per costruzione di $\widehat{\mathcal{M}}$. Si avrà dunque che $q_1 \in \widehat{\delta}^*(q_1, uv) \implies L^n \subseteq L(\widehat{\mathcal{M}})$

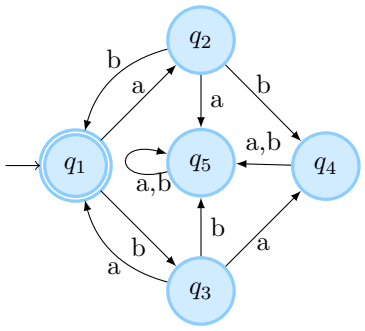
"$\supseteq$" Si ha che $L(\widehat{\mathcal{M}}) \subseteq L^*$ poiché, per costruzione di $\widehat{\mathcal{M}}, w \in L(\mathcal{M})$ è composizione di parole di L, dovendo per terminare raggiungere q_1, ovvero l'unico stato terminale di $\mathcal{M}$. Essendo che L era **non-restarting** l'unico modo per raggiungere q_1 in $\widehat{\mathcal{M}}$ è raggiungere uno stato terminale di L.

Esempio: Iterazione di Linguaggi Regolari

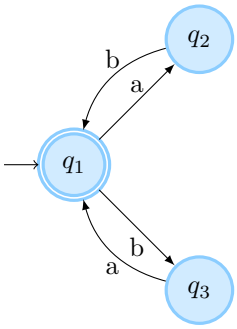
Consideriamo ad esempio il linguaggio $L = \{ab, ba\}$, accettato dall'automa $\mathcal{M}$:



Si avrà che l'iterazione L^* sarà accettata dall'automa $\widehat{\mathcal{M}}$:



È evidente però che questo automa non sia il più semplice possibile che accetti L^*. Infatti, $\widehat{\mathcal{M}}$ può essere semplificato in:



Enunciato. Un linguaggio $L \subseteq A^*$ è regolare *se e solo se* è finito oppure è ottenuto da linguaggi finiti mediante un numero finito di operazioni di unione, concatenazione e iterazione.

Dimostrazione. Dimostriamo la doppia implicazione.
"$\Rightarrow$" Ovvio dai teoremi 6.2.1 6.2.1, 6.2.1 e 6.2.3.
"$\Leftarrow$" Sia $\mathcal{M} = (Q, A, \delta, q_1, F)$ un DFA con $Q = \{q_1, \cdots, q_n\}$. Definiamo dunque una famiglia di linguaggi della forma $R_{i,j}^{(k)}$, dove:

$$\begin{aligned} &\forall 1 \leq i, j \leq n \wedge 0 \leq k \leq n : \\ &\quad R_{i,j}^{(k)} = \left\{ w \in A^* \,|\, \delta^*(q_i, w) = q_j \wedge \delta^*(q_i, w') \in \{q_1, \cdots, q_k\} \forall w' \neq \varepsilon \text{ prefisso proprio di } w \right\} \end{aligned} \tag{6.9}$$

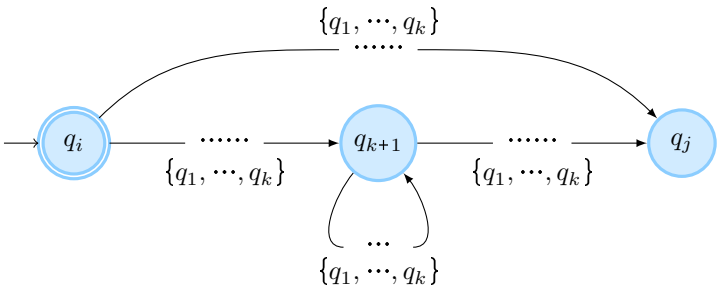
Ovvero fissati i, j e k, il linguaggio $R_{i,j}^{(k)}$ conterrà le parole che mi permettono di raggiungere lo stato q_j da q_i passando solo per stati $q \in \{q_1, \cdots, q_k\}$.
Nella definizione formale data, la seconda condizione richiede sostanzialmente che ogni sotto stringa di w ci porti **esclusivamente** in stati da q_1 a q_k.
È per questo fondamentale specificare che w' debba essere un prefisso **proprio** di w, ovvero diverso da w stesso, poiché altrimenti ogni insieme con $k < j$ sarebbe vuoto, poiché le parole al suo intero dovrebbero raggiungere lo stato q_j dovendo passare esse stesse, non solo le loro sotto stringhe, solo per stati da q_1 a q_k, ovvero impossibilitando l'arrivo in q_j.
Analogamente è fondamentale specificare che w' non sia la parola vuota, per evitare una contraddizione simile a quella appena vista ma nel caso $i > k$. Si avrà in fatti in questo caso che per ogni parola, si dovrà avere che:

$$\{q_1, \cdots, q_k\} \ni \delta^*(q_i, \varepsilon) = q_i \notin \{q_1, \cdots, q_k\} \tag{6.10}$$

Definiti dunque questi insiemi possiamo osservare che:

$$\begin{aligned} R_{i,i}^{(0)} &= \{\varepsilon\} \cup \left\{ a \in A^* \,|\, \delta(q_i, a) = q_i \right\} \text{ è un insieme finito} \\ \text{Con } i \neq j : R_{i,j}^{(0)} &= \left\{ a \in A \,|\, \delta(q_i, a) = q_j \right\}, \text{ è un insieme finito} \\ R_{i,j}^{(k+1)} &= R_{i,j}^{(k)} \cup R_{i,k+1}^{(k)} \cdot \left(R_{k+1,k+1}^{(k)}\right)^* \cdot R_{k+1,j}^{(k)} \end{aligned} \tag{6.11}$$

In particolare l'ultima osservazione ci dice che tutte le parole che ci portano da q_i a q_j attraversando esclusivamente stati da q_1 a q_{k+1} possono essere di due tipi, ovvero quelle che raggiungono q_j passando soltanto per stati da q_1 a q_k, ovvero ignorando lo stato q_{k+1}, e quelle che attraversano almeno una volta q_{k+1}.
In particolare queste raggiungeranno q_{k+1} una prima volta, passando dunque solo per stati da q_1 a q_k, torneranno in q_{k+1} un numero arbitrario di volte, ancora passando solo per stati da q_1 a q_k, e infine lasceranno un'ultima volta q_{k+1} per raggiungere q_j passando ancora solo da stati tra q_1 e q_k. Questa divisione si può evincere più chiaramente dal diagramma seguente:



Con queste tre osservazioni possiamo dimostriamo dunque per induzione su k che **ogni linguaggio** $R_{i,j}^{(k)}$ può essere ottenuto da linguaggi finiti mediante un numero finito di operazioni di unione, concatenazione e iterazione.
Possiamo ora facilmente vedere però che per ogni automa a stati finiti $\mathcal{M}$ di n stati, in particolare quello considerato all'inizio della dimostrazione, vale che $L(\mathcal{M}) = \bigcup_{q_j \in F} R_{1,j}^{(n)}$.
Ovvero il linguaggio accettato dall'automa è l'unione dei linguaggi che portano da 1 a un qualsiasi stato terminale fissato passando potenzialmente per tutti e n gli stati, cioè dunque un qualsiasi linguaggio regolare è sempre della forma $R_{i,j}^{(k)}$ per qualche i, j, k definiti come sopra, la quale è sempre come da tesi.

6.3 Espressioni Regolari

Avendo analizzato le operazioni che permettono di ottenere linguaggi regolari, possiamo ora definire un modo per descrivere questi linguaggi in modo più compatto e semplice, ovvero delle stringhe che seguano specifiche regole sintattiche e che, abbinate a una semantica ben definita, permettano di descrivere tutti i linguaggi regolari.

Espressioni regolari

Dato A alfabeto finito, definisco:

$$\hat{A} = \{\underline{a} | a \in A\} \cup \left\{\underline{\cup}, \underline{\cdot}, \underline{*}, \underline{(}, \underline{)}, \underline{\varepsilon}, \underline{\varnothing}\right\} \tag{6.12}$$

Chiameremo **espressione regolare** su A una parola su $\hat{A}^*$ ottenuta dalle seguenti regole:

1. $\forall a \in A : \underline{a}$ è un'espressione regolare, così come $\underline{\varepsilon}$ e $\underline{\varnothing}$.
2. Se α e β sono espressioni regolari, allora anche $\underline{(}\alpha\underline{\cup}\beta\underline{)}$ lo è.
3. Se α e β sono espressioni regolari, allora anche $\underline{(}\alpha\underline{\cdot}\beta\underline{)}$ lo è.
4. Se α è un'espressione regolare, allora anche $\underline{(}\alpha\underline{)}\underline{*}$ lo è.
5. Non ci sono altre regole.

6.3.1 Semantica delle Espressioni Regolari

Oltre ad aver definito la sintassi delle espressioni regolari, per far si che queste possano essere utilizzate per descrivere linguaggi regolari è necessario definire una semantica per queste espressioni, ovvero un modo per interpretare le espressioni regolari come linguaggi regolari.

Se γ è un'espressione regolare su A, il linguaggio associato a γ: $\langle\gamma\rangle \subseteq A^*$ è definito dalle regole semantiche:

1. $\forall a \in A : \langle\underline{a}\rangle = \{a\}$, inoltre $\langle\underline{\varepsilon}\rangle = \{\varepsilon\}$ e $\langle\underline{\varnothing}\rangle = \varnothing$.
2. Se $\gamma = \underline{(}\alpha\underline{\cup}\beta\underline{)}$, allora $\langle\gamma\rangle = \langle\alpha\rangle \cup \langle\beta\rangle$.
3. Se $\gamma = \underline{(}\alpha\underline{\cdot}\beta\underline{)}$, allora $\langle\gamma\rangle = \langle\alpha\rangle \cdot \langle\beta\rangle$.
4. Se $\gamma = \underline{(}\alpha\underline{)}\underline{*}$, allora $\langle\gamma\rangle = (\langle\alpha\rangle)^*$.

Il teorema di Kleene 6.2.3 può essere dunque reinterpretato come:

$$L \subseteq A^* \text{ regolare} \iff \exists\alpha \text{ espressione regolare su } A \text{ tale che } L = \langle\alpha\rangle \tag{6.13}$$

Esempio: Espressione Regolare

Consideriamo il linguaggio dell'esercizio 6.4 sull'alfabeto $\{a, b\}$.
Si avrà che l'espressione regolare che lo definisce è:

$$\alpha = ((a \cup b) \cdot ((a \cup b) \cup (a \cdot ((a \cup b)^* \cdot a)) \cup (b \cdot ((a \cup b)^* \cdot b)))) \tag{6.14}$$

Per comodità di scrittura, possiamo definire delle regole di precedenza tra gli operatori in modo da evitare di dover usare le parentesi e per sottintendere il simbolo $\cdot$. In particolare, definiamo:

$$* > \cdot > \cup \tag{6.15}$$

Avremo quindi che l'espressione regolare dell'ultimo esempio può essere scritta come:

$$\alpha = (a \cup b)(a \cup b \cup a(a \cup b)^* a \cup b(a \cup b)^* b) \tag{6.16}$$

6.4 Pumping Lemma per Linguaggi Regolari

Fino ad ora abbiamo analizzato e descritto esclusivamente linguaggi regolari e come operare su di essi.

Vogliamo ora analizzare il problema opposto, ovvero come dimostrare che un linguaggio non è regolare.

Per fare ciò, introduciamo un risultato teorico che dimostra una condizione necessaria per la regolarità di un linguaggio, che se quindi negata ci permette di dimostrare che un linguaggio non è regolare.

Questo risultato è il **Pumping Lemma per Linguaggi Regolari**, e sfrutta un principio combinatorio detto **principio della piccionaia**.

Il **principio della piccionaia** afferma che se n piccioni vengono messi in m piccionaie, con $n > m$, allora almeno una piccionaia conterrà almeno due piccioni.

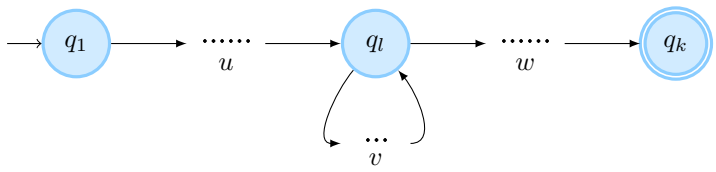
Generalizzando questo concetto, potremmo dire che se ho n oggetti da distribuire su m insiemi, con $n > m$, non potrò fare a meno di associare a più oggetti lo stesso insieme.

Pumping lemma per linguaggi regolari (lemma uwu)

Enunciato. Sia $\mathcal{M}$ un DFA con n stati, e $x \in L(\mathcal{M})$ tale che $|x| \geq n$. Allora esistono $u, v, w \in A^*$ tali che:

1. $x = uvw$
2. $v \neq \varepsilon$
3. $\forall i \in \mathbb{N} : uv^iw \in L(\mathcal{M})$, ovvero $\{u\}\{v\}^*\{w\} \subseteq L(\mathcal{M})$

Dimostrazione. Per il principio della piccionaia, nel percorso di accettazione di x (da q_1 a $q_k \in F$), che passa necessariamente per almeno $n+1$ stati dovendo includere anche lo stato iniziale, deve esistere un ciclo, ovvero si passerà per un certo stato q_l almeno due volte.
Si ha allora $x = uvw$, dove $\delta^*(q_1, u) = q_l$, ovvero u è la parola che porta da q_1 a q_l, $\delta^*(q_l, v) = q_l$, ovvero v è la parola descitta dal ciclo partente da q_l, e $\delta^*(q_l, w) = q_k$, ovvero w è la parola che porta dalla fine del ciclo fino allo stato terminale q_k.
Avremo dunque chiaramente che $v \neq \varepsilon$, dovendo passare per q_l almeno due volte distinte, ma anche che $\forall i \in \mathbb{N} : \delta^*(q_1, uv^iw) = q_k$, potendo percorrere il ciclo un numero arbitrario di volte, da cui la tesi.
La suddivisione di x in u, v, w come da dimostrazione è più facilmente intuibile dal seguente diagramma:



Visto questo importante risultato, possiamo dunque dimostrare che un linguaggio non è regolare dimostrando che non rispetta il **Pumping Lemma**.

Oltre a fare ciò però è possibile derivare dal **Pumping Lemma** altri risultati teorici minori utili.

Corollario 1

Enunciato. Sia $\mathcal{M}$ un DFA con n stati, allora $L(\mathcal{M}) \neq \varnothing \implies \exists x \in L(\mathcal{M}) : |x| < n$.

Dimostrazione. Per assurdo, supponiamo che $x \in L(\mathcal{M})$ sia la parola di lunghezza minima con $|x| \geq n$.
Per il pumping lemma, esistono $u, v, w \in A^*$ tali che $x = uvw$, con $v \neq \varepsilon$ e $\forall i \in \mathbb{N} : uv^iw \in L(\mathcal{M})$. Ma allora $uv^0w = uw \in L(\mathcal{M})$, contraddicendo la minimalità di x.

Questo corollario risulta particolarmente utile per determinare se, dati due DFA $\mathcal{M}_1, \mathcal{M}_2$, si ha che $L(\mathcal{M}_1) \subseteq L(\mathcal{M})_2$.

Si ha infatti che, poiché $L(\mathcal{M}_1) \setminus L(\mathcal{M}_2) = L(\mathcal{M}_1) \cap \overline{L(\mathcal{M}_2)}$ è regolare, possiamo disegnare un DFA $\mathcal{M}$ che accetti $L(\mathcal{M}_1) \setminus L(\mathcal{M}_2)$, e verificare se $L(\mathcal{M}) = \varnothing$.

Dal corollario appena dimostrato però, per dimostrare che la differenza sia vuota è sufficiente che

$$\forall x \in A^* |x| < n \implies x \notin L(\mathcal{M}) \tag{6.17}$$

Ovvero che non ci siano parole di lunghezza minore del numero di stati dell'automa accettate dal linguaggio.

Corollario 2

Enunciato. Sia $\mathcal{M}$ un DFA con n stati, allora $L(\mathcal{M})$ è infinito $\iff \exists x \in L(\mathcal{M}) : n \leq |x| < 2n$.

Dimostrazione. Dimostriamo la doppia implicazione.

"$\Rightarrow$" Dal **Pumping Lemma**, $x = uvw$ con $v \neq \varepsilon$ e $\forall i \in \mathbb{N} : uv^iw \in L(\mathcal{M})$. Avremo dunque che L accetta un numero infinito di parole, al variare di i in $\mathbb{N}$.

"$\Leftarrow$" Essendo $L(\mathcal{M})$ infinito, la lunghezza delle parole non può essere superiormente limitata, altrimenti sarebbe possibile contare il numero massimo di parole accettabili.
Sia x dunque la più corta parola in $L(\mathcal{M})$ tale che $|x| \geq 2n$. Possiamo dunque scrivere $x = yz$ con $|y| = n$ e $|z| \geq n$.
Per il principio della piccionaia, avremo $y = uvw$ con $v \neq \varepsilon$ e $\forall i \in \mathbb{N} : \delta^*(q_1, uv^iw) = \delta^*(q_1, y)$, ovvero al variare di i raggiungerò lo stesso stato raggiunto con y, da cui $\forall i \in \mathbb{N} : uv^iwz \in L(\mathcal{M})$.
Allora anche $uwz \in L(\mathcal{M})$ ma $|uwz| < |yx| = |x|$, da cui $|uwz| < 2n$ dalla minimalità di x. Ma essendo $|z| \geq n$ per costruzione, si ha che $|uwz| \geq n$, da cui la tesi.

Vediamo ora finalmente degli esempi di **Linguaggi non Regolari**, e di come dimostrare ciò utilizzando il **Pumping Lemma**.

Esempi: Linguaggi non regolari

1. Il linguaggio $L = \{a^nb^n | n \in \mathbb{N}\}$, ovvero un linguaggio che accetti solo parole con un numero di a consecutive seguito da un ugual numero di b consecutive, **non** è regolare.

   Infatti sia per assurdo $\mathcal{M}$ un DFA con p stati, tale che $L(\mathcal{M}) = L$.

   Presa $x = a^pb^p$, se $x = uvw$ con $v \neq \varepsilon$ sono possibili tre casi:
   (a) $v = a^j, j > 0 \implies uw = a^{p-j}b^p \notin L$
   (b) $v = b^j, j > 0 \implies uw = a^pb^{p-j} \notin L$
   (c) $v = a^jb^k, j, k > 0 \implies uv^2w = a^{p-j}(a^jb^k)^2b^{p-k} = a^pb^ka^jb^p \notin L$

   In questo esempio specifico è possibile dimostrare la tesi anche con altri valori di i, come ad esempio $i = 0$ nell'ultimo caso, ma in generale questo non è possibile.

   Abbiamo quindi che per qualsisi scelta di v il **Pumping Lemma** non è rispettato, dimostrando la tesi.

2. Il linguaggio $L' = \{a^mb^n | m \geq n > 0\}$, non è regolare.

   Infatti sia per assurdo $\mathcal{M}$ un DFA con p stati, tale che $L(\mathcal{M}) = L'$.

   Scelgo $x = a^pb^p \in L, |x| = 2p \geq p$. Possiamo dividere i casi possibili in maniera analoga all'esempio precedente, con l'unica differenza che nel primo caso non potremo scegliere un'arbitraria i per la dimostrazione, poichè $\forall i \geq 1$ la parola sarebbe ancora accettata dall'automa, dunque siamo costretti a scegliere $i = 0 \implies uv^iw = uw$ per dimostrare la tesi.

3. Considerando invece il linguaggio $L'' = \{a^mb^n | m > n > 0\}$, avremo che per la dimostrazione non potremo più usare la parola $a^pb^p \notin L''$. Possiamo però considerare la parola $x = a^pb^{p-1} \in L'', |x| = 2p - 1 \geq p$, che si suddivide nei medesimi tre casi degli esempi precedenti e si dimostra con la stessa scelta di i del secondo esempio.

4. **Esercizio d'esame.** Mostrare che $L = \{a^nb^n | n \geq 0\} \cup \langle b^* \cdot a^* \rangle$ non è regolare.

   Soluzione. Per quanto il **Pumping Lemma** sia uno strumento utile per dimostrare la non regolarità di un linguaggio, non è sempre l'approccio corretto da seguire. In questo caso andremo ad utilizzare le proprietà di chiusura dei linguaggi regolari.

   Per assurdo, sia L regolare. Allora dovrebbe esserlo anche $L \cap \langle a^*b^* \rangle$, ma $L \cap \langle a^*b^* \rangle = \{a^nb^n | n \geq 0\}$, che per quanto visto in precedenza non è regolare, da cui l'assurdo.

Come già intuito dall'ultimo esempio, e come è evincibile dall'enunciato stesso, il **Pumping Lemma** non fornisce una condizione sufficiente per la regolarità di un linguaggio, bensì una condizione necessaria.

Possono esistere dunque linguaggi non regolari che rispettano il **Pumping Lemma**, e per questo motivo non è possibile dimostrare la regolarità di un linguaggio utilizzando il **Pumping Lemma**.

Utilizzando però l'enunciato di **Pumping Lemma** dato possiamo trovare dei linguaggi non regolari, per i quali anche la non regolarità non è dimostrabile utilizzandolo.

Vediamo dunque una versione più forte del **Pumping Lemma**, derivabile in maniera diretta dalla dimostrazione precedente, per poi vedere un esempio di linguaggio non regolare per il quale non è possibile dimostrare la non regolarità utilizzando il **Pumping Lemma** precedente.

Pumping Lemma per linguaggi regolari rafforzato

Enunciato. Sia $\mathcal{M}$ un DFA con n stati, e $x \in L(\mathcal{M})$ tale che $|x| \geq n$. Allora esistono $u, v, w \in A^*$ tali che:

1. $x = uvw$
2. $v \neq \varepsilon$
3. $\forall i \in \mathbb{N} : uv^i w \in L$
4. $|uv| \leq n$

Dimostrazione. Per dimostrare questa versione del **Pumping Lemma** consideriamo la parola $x_n \subseteq x$ formata dalle prime n lettere.
Per il principio della piccionaia sappiamo che già in questa parola deve essere presente almeno uno stato ripetuto.
Potrò allora scegliere come u la sottoparola di x fino a questo primo stato ripetuto, con dunque $u \subseteq x_n$, come v la parola del ciclo contenuto in x_n, e come w la parola formata dalle restanti lettere di x.
Avendo scelto $u, v \subseteq x_n$ si ha che $|uv| \leq n$, da cui la tesi.

Esempio: Non regolarità del linguaggio dei palindromi

Vediamo dunque un esempio di linguaggio non regolare, che però non può essere dimostrato con il precedente **Pumping Lemma**.

Sia $L = \{w \in \{a,b\}^* | w = \tilde{w}\}$, dove se $w = w_1 w_2 \cdots w_n, w_1 w_2 \cdots w_n \in \{a,b\}^*$ allora $\tilde{w} = w_n w_{n-1} \cdots w_1$. Questo linguaggio non è regolare.

Proviamo inizialmente a dimostrare col **Pumping Lemma** originario che L non è regolare.

Sia p il numero di stati di un DFA che accetta L. Avremo che, per ogni parola di L, si potrà sempre scegliere v come la sotto parola centrale e ottenere una parola in L. Infatti, se $x = uvw = \tilde{x}$ con $v = \tilde{v}$, allora $w = \tilde{u}$ e dunque $\forall i \in \mathbb{N} : uv^i w \in L$.

Utilizzando il **Pumping Lemma** rafforzato però, sappiamo che $|uv| \leq p$, ovvero v dev'essere contenuta nelle prime p lettere di x. Ma allora v non può essere la sotto parola centrale di x.
Consideriamo la parola $x = a^p b a^p = uvw$, con $v \neq \varepsilon$ e $|uv| \leq p$. Allora avremo necessariamente che:

$$v = a^k, p \geq k > 0 \implies uw = a^{p-k} b a^p \notin L \tag{6.18}$$

da cui la tesi.

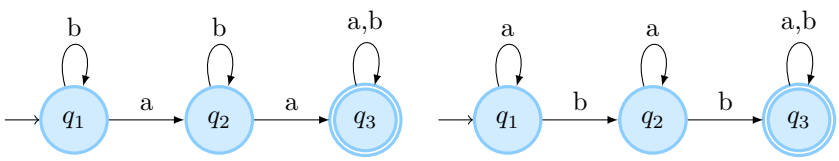
6.5 Esercizi ed approfondimenti

Esercizio. *Sia $A = \{a,b\}$ e siano $L_1 = \{w \in A^* | w$ contiene almeno due occorrenze di $a\}$ e*

$L_2 = \{w \in A^* | w$ contiene almeno due occorrenze di $b\}$. *Per ognuno dei seguenti L, esibire un NFA che accetti L.*

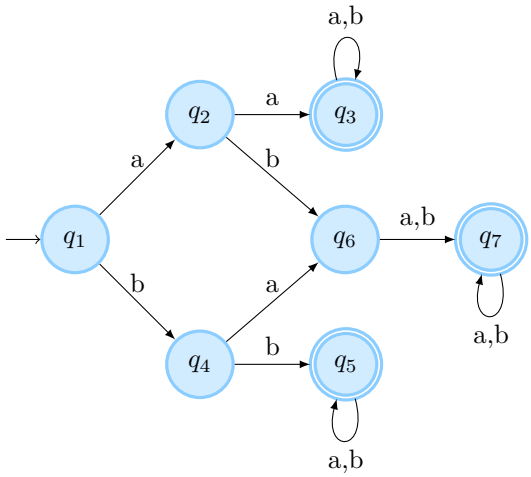
1. $L = L_1 \cup L_2$
2. $L = A^* \setminus L_1$
3. $L = A^* \setminus L_2$
4. $L = L_1 \cap L_2$

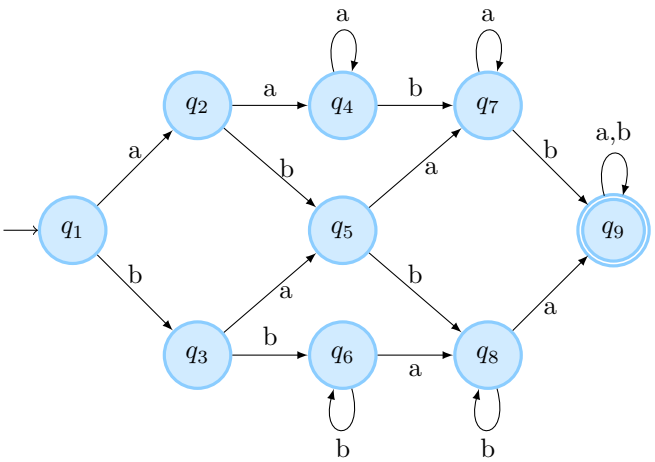
Innanzitutto andiamo a verificare che esista un NFA che accetti L_1 e una che accetti L_2.



(a) NFA che accetta L_1 (b) NFA che accetta L_2

1. $L = L_1 \cup L_2$ Per l'unione sarà sufficiente considerare l'unione dei due NFA
2. $L = A^* \setminus L_1 = \overline{L_1}$. Dai teoremi 5.2.2 e 5.2.4 sarà sufficiente prendere l'automa 6.3a e invertire tra loro stati terminali e non.
3. $L = A^* \setminus L_2$ Ragionamento analogo al punto precedente.
4. $L = L_1 \cap L_2$





Esercizio. *Siano L, L' regolari. Mostrare che $L \setminus L'$ è regolare.*

$L \setminus L' = L \cap \overline{L'}$. Per i teoremi 6.2.1 e 5.2.2 sappiamo che $L \cap \overline{L'}$ è regolare.

Esercizio. *Siano L, L' tali che L è regolare, $L \cup L$ è regolare e $L \cap L' = \varnothing$. Mostrare che L' è regolare.*

$L \cap L' = \varnothing \implies (L \cup L') \setminus L = L'$. Dal teorema 6.2.1 e dall'esercizio precedente sappiamo che L' è regolare.

Esercizio. *Mostrare che il linguaggio $L = \{w = a_1 a_2 \cdots a_n | n \in \mathbb{N}, a_i \in A, a_2 = a_n\}$ è regolare*

Iniziamo mostrando che $\forall a \in A : L_a = \left\{xaw \in A^* | w \in A^*, x \in A\right\}$ è regolare.

Per questo è sufficiente considerare l'automa a stati finiti:

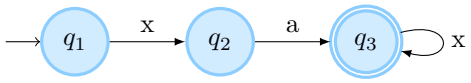


Figura 6.4: Automa a stati finiti per L_a

Si avrà per chiusura rispetto al prodotto che $\forall a \in A : L_a\{a\}$ è regolare, essendo tutti i singleton regolari in quanto finiti.

Otteniamo infine che $L = \bigcup_{a \in A} L_a\{a\}$ è regolare per chiusura rispetto all'unione. Si sottolinea come questa operazione non sia una unione infinita in quanto per definizione di **macchina di Turing** la cardinalità di A è finita.

Esercizio. *Per ogni linguaggio L descritto nell'esercizio 6.5, ricavare un'espressiore regolare α tale che $L = \langle\alpha\rangle$.*

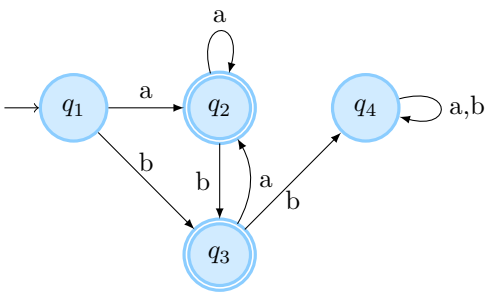
1. $L = L_1 \cup L_2 = \langle\alpha\rangle$ con $\alpha = b^* a b^* a b^* \cup a b^* a b^* a b^*$
2. $L = A^* \setminus L_1 = \langle\alpha\rangle$ con $\alpha = b^* a b^*$
3. $L = A^* \setminus L_2 = \langle\alpha\rangle$ con $\alpha = a^* b a^*$
4. $L = L_1 \cap L_2 = \langle\alpha\rangle$ con $\alpha = a(a(a \cup b)^* \cup b(a \cup b)(a \cup b)^*) \cup b(b(a \cup b)^* \cup a(a \cup b)(a \cup b)^*)$

Esercizio. *Sia $L = \left\{x \in \{a, b\}^* | x \neq \varepsilon \wedge x \text{ non contiene } bb\right\}$*

a. Construire un DFA che accetti L;

b. Scrivere un'espressione regolare α tale che $L = \langle \alpha \rangle$.

a. DFA:



b. $\alpha = (a \cup ba)^* b \cup (a \cup ba)^* (a \cup ba) = (a \cup ba)^* (b \cup a \cup ba)$

Esercizio *Sia $L = \{a^n b^m c^r | n + m \leq r \wedge n, m, r \in \mathbb{N}\}$. Mostrare che L non è regolare.*

Sia per assurdo $\mathcal{M}$ un DFA di p stati che accetti L. Avremo dunque che $x = a^{\frac{p}{2}} b^{\frac{p}{2}} c^p$ è accettata da $\mathcal{M}$ e $|x| = 2p \geq p$.

Dal **Pumping Lemma rafforzato** sappiamo che $\exists u, v, w \in A^*$ tali che $x = uvw \wedge v \neq \varepsilon \wedge |uv| \leq p \wedge \forall i \in \mathbb{N} : uv^i w \in L$.

Avremo dunque che la scelta per v è limitata a tre possibilità:

1. $v = a^k, \frac{p}{2} \geq k > 0 \implies uv^2 w = a^{\frac{p}{2}+k} b^{\frac{p}{2}} c^p \notin L$, essendo che $\frac{p}{2} + k + \frac{p}{2} > p$
2. $v = b^k, \frac{p}{2} \geq k > 0 \implies uv^2 w = a^{\frac{p}{2}} b^{\frac{p}{2}+k} c^p \notin L$, essendo che $\frac{p}{2} + k + \frac{p}{2} > p$
3. $v = a^j b^k, \frac{p}{2} \geq j, k > 0 \implies uv^2 w = a^{\frac{p}{2}} b^k a^j b^{\frac{p}{2}} c^p \notin L$, avendo una struttura errata.

Grammatiche context-free e Automi Pushdown

Dopo aver esplorato a fondo il concetto di automa, analizziamo ora un altro modello di calcolo, le grammatiche context-free.

Il concetto di **grammatica context-free** nasce in ambito linguistico per descrivere i linguaggi naturali, e solo successivamente venne formalizzato per essere utilizzato con i linguaggi formali.

Le **grammatiche context-free** forniscono descrizioni del linguaggio più ad alto livello sulla struttura delle stringhe rispetto a quanto visto coi linguaggi regolari, che, ricordando l'origine di questo concetto, è assimilabile a un analisi logica di un periodo.

7.1 Grammatiche context-free

Innanzitutto per utilizzare le **grammatica context-free** con linguaggi formali, dobbiamo darne una definizione formale.

Grammatiche context-free

Una **grammatica context-free** è una quadrupla $G = (V, \Sigma, R, S)$ dove:
- V è un insieme finito di simboli detto **alfabeto delle variabili**;
- Σ è un insieme finito di simboli detti **alfabeto dei terminali**;
- $S \in V$ è detta **variabile iniziale** o **assioma**;
- $R \subseteq V \times (V \cup \Sigma)^*$ è un insieme finito di **regole** o **produzioni**

Le coppie $(X, u) \in R$ si indicano con $X \to u$ e si leggono X *produce* u.

Similmente a quanto visto per i DFA con la funzione di transizione e la funzione di transizione iterata, per definire i linguaggi generati da una **grammatica context-free** dobbiamo definire la relazione di derivazione elementare e la relazione di derivazione.

Data la definizione formale di **grammatica context-free**, dobbiamo definire il concetto di derivazione.

Derivazione e derivazione elementare

Date $w, w' \in (V \cup \Sigma)^*$ diremo che w' si ottiene da w per **derivazione elementare** $\left(w \underset{G}{\Longrightarrow} w'\right)$ se:

$$w = v_1 X v_2 \wedge w' = v_1 u v_2, \text{ con } v_1, v_2 \in (V \cup \Sigma)^*, X \to u \in R \tag{7.1}$$

Diremo inoltre che w' si ottiene da w per **derivazione** $\left(w \overset{*}{\underset{G}{\Longrightarrow}} w'\right)$ se:

$$\exists w_1, \ldots, w_n = w' \in (V \cup \Sigma)^* : w \underset{G}{\Longrightarrow} w_1 \underset{G}{\Longrightarrow} \ldots \underset{G}{\Longrightarrow} w_n = w', \text{ con } n \geq 0 \tag{7.2}$$

Possiamo finalmente definire i linguaggi generati da una **grammatica context-free** come $L(G) = \left\{w \in \Sigma^* \,|\, S \overset{*}{\underset{G}{\Longrightarrow}} w\right\}$, ovvero stringhe di soli **terminali** derivabili dall'**assioma**.

Esempi: Grammatiche context-free

Un primo esempio di **grammatica context-free** è la seguente:

$$\begin{aligned} S &\to \varepsilon \\ S &\to aSb \end{aligned} \tag{7.3}$$

Per leggibilità, nella scrittura delle regole di una grammatica context-free, si utilizzano alcune convenzioni, come ad esempio scrivere per prime le regole che sostituiscono l'assioma, o raggruppare tutte le regole che sostituiscono la stessa variabile separando le possibili sostituzioni col simbolo $|$.
Possiamo quindi riscrivere le regole della grammatica precedente come:

$$S \to \varepsilon \mid aSb \tag{7.4}$$

Per questa grammatica, possiamo dire che $L(G) = \{a^n b^n | n \geq 0\}$. Infatti, a ogni derivazione da S avremo o la terminazione della sequenza di derivazioni o l'aggiunta di un'a e di una b alla stringa, che quindi avrà sempre la forma $a^n b^n$.

Si avrà infatti $S \underset{G}{\Longrightarrow} aSb \underset{G}{\Longrightarrow} aaSbb \underset{G}{\Longrightarrow} \dots \underset{G}{\Longrightarrow} a^n b^n \underset{G}{\Longrightarrow}$ utilizzando la regola $S \to \varepsilon$ come ultima.

Avendo dimostrato nel capitolo precedente che questo linguaggio non è regolare, possiamo affermare che esistono linguaggi che non sono regolari ma sono generabili da una **grammatica context-free**.

Un altro esempio di linguaggio non regolare che abbiamo visto nello scorso capitolo è $L = \{a^m b^n | m \geq n > 0\}$. Anche in questo caso il linguaggio è generabile da una **grammatica context-free**.

Innanzitutto possiamo scomporre la generica parola del linguaggio come $a^m b^n = a^{m-n} a^n b^n$.
Avremo dunque che per ottenere una parola di L basta concatenare una parola di a^{m-n} con una parola di $a^n b^n$.

Possiamo quindi scrivere la seguente grammatica:

$$\begin{aligned} S &\to AX \\ A &\to aA \mid \varepsilon \\ X &\to aXb \mid ab \end{aligned} \tag{7.5}$$

La prima regola di questa grammatica divide la parola generata sempre in due parti, che saranno poi competenza di altre due variabili. La seconda regola genera la parte di a^{m-n} e la terza regola genera la parte di $a^n b^n$.

Vediamo infine un esempio di grammatica context-free che permette di descrivere una sintassi per le espressioni aritmetiche, ovvero $G = (V, \Sigma, R, E), V = \{E, T, F\}, \Sigma = \{a, +, \cdot, (,)\}$ con R così definito:

$$\begin{aligned} E &\to E + T \mid T \\ T &\to T \cdot F \mid F \\ F &\to (E) \mid a \end{aligned} \tag{7.6}$$

Si può facilmente vedere infatti che $)((a\cdot \notin L(G)$ mentre $a + a \cdot a \in L(G)$, infatti:

$$E \underset{G}{\Longrightarrow} E + T \underset{G}{\Longrightarrow} T + T \underset{G}{\Longrightarrow} F + T \underset{G}{\Longrightarrow} a + T \underset{G}{\Longrightarrow} a + T \cdot F \underset{G}{\Longrightarrow} a + F \cdot F \underset{G}{\Longrightarrow} a + a \cdot F \underset{G}{\Longrightarrow} a + a \cdot a \tag{7.7}$$

Possiamo tra l'altro notare come questa grammatica segua le regole di precedenza delle operazioni aritmetiche, ovvero che la moltiplicazione ha precedenza sulla somma.
Infatti la stringa $a \cdot a + a \notin L$

7.1.1 Albero di derivazione e ambiguità

Definite le **grammatiche context-free** e i linguaggi da esse generate, andiamo ora ad analizzare il funzionamento di queste grammatiche.

Un primo aspetto interessante da affrontare è il concetto di ambiguità.

Con una rapida analisi degli esempi appena visti, possiamo notare che una stessa stringa può essere derivata in più modi diversi.

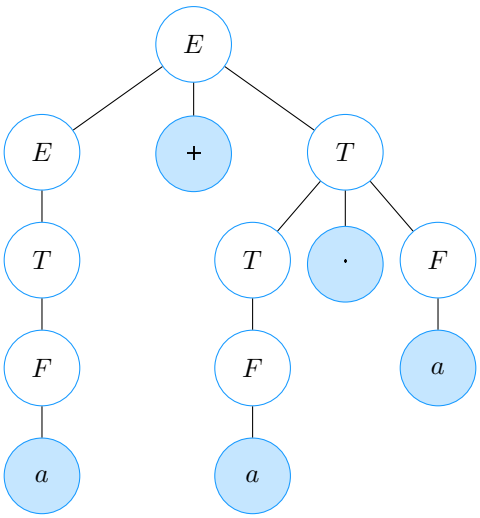
Nel caso di $a + a \cdot a$ ad esempio, si è scelta sempre la variabile più a sinistra da sostituire, ovvero si è fatta una **derivazione estrema a sinistra**, ma si poteva anche scegliere di sostituire prima la variabile più a destra.

Queste due derivazioni però non utilizzano regole diverse, ma semplicemente le applicano in ordine diverso.

Per rendere questa equivalenza è possibile costruire un particolare albero, detto **albero di derivazione** o **parse tree**, ovvero un albero che segue le seguenti regole:

- La radice dell'albero è l'assioma della grammatica
- Ogni foglia è un terminale
- Ogni nodo interno è una variabile
- Ogni nodo ha per figli la sotto stringa derivata da quel nodo da destra a sinistra
- Le foglie, lette da sinistra a destra, formano la stringa derivata

Ad esempio, l'**albero di derivazione** per la stringa $a + a \cdot a$ è il seguente:



Consideriamo ora la stringa $a + a \cdot a$ **derivata estremamente a destra**. Si avrà:

$$E \underset{G}{\Longrightarrow} E + T \underset{G}{\Longrightarrow} E + T \cdot F \underset{G}{\Longrightarrow} E + T \cdot a \underset{G}{\Longrightarrow} E + F \cdot a \underset{G}{\Longrightarrow} E + a \cdot a \underset{G}{\Longrightarrow} T + a \cdot a \underset{G}{\Longrightarrow} F + a \cdot a \underset{G}{\Longrightarrow} a + a \cdot a \tag{7.8}$$

Possiamo facilmente vedere che anche questa derivazione porta allo stesso **albero di derivazione**.

Diremo dunque che due derivazioni sono **equivalenti** se portano allo stesso **albero di derivazione**.

Possiamo infine definire formalmente il concetto di ambiguità e non ambiguità.

Ambiguità e non ambiguità

Una **grammatica context-free** è **non ambigua** se per ogni stringa generata esiste un unico albero di derivazione distinto.
Una **grammatica context-free** è **ambigua** se esiste almeno una stringa generata da più alberi di derivazione distinti.

Avendo visto che derivazioni equivalenti portano a un unico **albero di derivazione**, possiamo anche riformulare la definizione di ambiguità e non ambiguità in termini di **derivazioni estreme a sinistra**.

Diremo infatti che una **grammatica context-free** è **non ambigua** se per ogni stringa è generata da un unica derivazione estrema a sinistra.

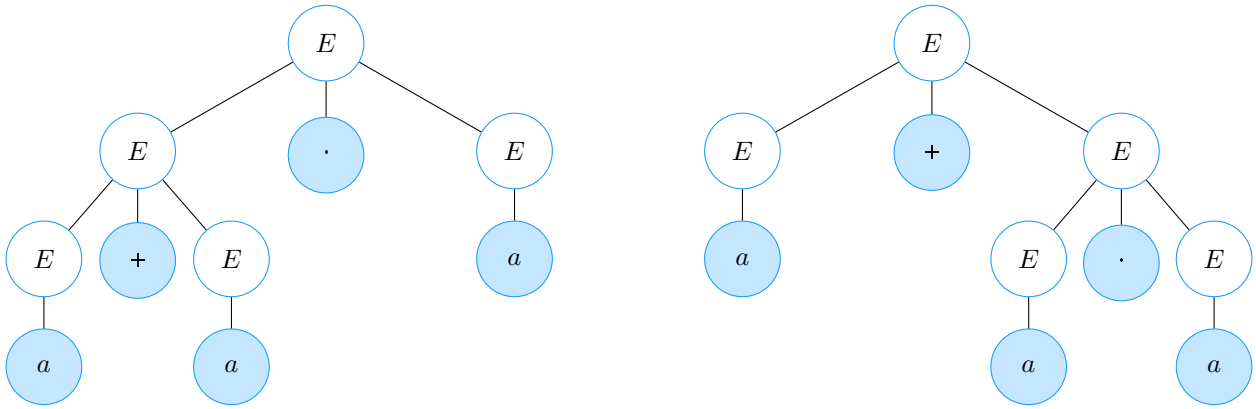
Per vedere un esempio di grammatica ambigua, è sufficiente considerare un insieme di regole che generano lo stesso linguaggio analizzato fino a ora, nello specifico $E \to (E) \mid E + E \mid E \cdot E \mid a$.

Considerando ora la solita stringa $a + a \cdot a$, possiamo derivarla in più modi diversi non equivalenti, senza *rispettare l'ordine aritmetico delle operazioni*:

1. $E \underset{G}{\Longrightarrow} E \cdot E \underset{G}{\Longrightarrow} E + E \cdot E \underset{G}{\Longrightarrow} a + E \cdot E \underset{G}{\Longrightarrow} a + a \cdot E \underset{G}{\Longrightarrow} a + a \cdot a$
2. $E \underset{G}{\Longrightarrow} E + E \underset{G}{\Longrightarrow} E + E \cdot E \underset{G}{\Longrightarrow} a + E \cdot E \underset{G}{\Longrightarrow} a + a \cdot E \underset{G}{\Longrightarrow} a + a \cdot a$

Possiamo facilmente notare che queste due derivazioni sono diverse poiché entrambe estreme a sinistra, ma che portano entrambe alla stessa stringa.

Per maggiore chiarezza possiamo costruire i due alberi di derivazione:



Per questo esempio, possiamo dire che la grammatica è ambigua poiché esistono due alberi di derivazione distinti per la stringa $a + a \cdot a$, ma ciò dipende esclusivamente dalla grammatica utilizzata e non dal linguaggio, che infatti siamo stati in grado di derivare anche da una grammatica non ambigua.

Esistono però linguaggi che non possono essere generati da grammatiche non ambigue, detti **inerentemente ambigui**[1].

Un esempio di linguaggio **inerentemente ambiguo** è $L = \left\{a^i b^j c^k | i, j, k \geq 0 \wedge (i = j \vee i = k)\right\}$.

È facile vedere che questo linguaggio è generato necessariamente da una grammatica ambigua, poiché conterrà anche le parole della forma $a^i b^i c^i$ che possono essere derivate sia come parole della forma $a^i b^i c^k$ che della forma $a^i b^j c^i$.

7.2 Proprietà di chiusura dei linguaggi context-free

Come di consueto, una volta analizzato il funzionamento di un modello di calcolo, andiamo ora ad analizzare le proprietà di chiusura dei linguaggi generati da esso.

Andiamo innanzitutto a verificare che la classe dei linguaggi context-free sia chiusa per le operazioni regolari, ovvero unione, concatenazione e iterazione.

Chiusura per unione dei linguaggi context-free

Enunciato. Se L_1 e L_2 sono linguaggi context-free, allora lo è anche $L_1 \cup L_2$.

Dimostrazione. Sia $L_1 = L(G_1)$ con $G_1 = (V_1, \Sigma, R_1, S_1)$ e $L_2 = L(G_2)$ con $G_2 = (V_2, \Sigma, R_2, S_2)$[a].
Con $V_1 \cap V_2 = \varnothing$. Allora la grammatica $G = (V_1 \cup V_2 \cup \{S\}, \Sigma, R_1 \cup R_2 \cup \{S \to S_1 \mid S_2\}, S), S \notin V_1 \cup V_2$ genera $L_1 \cup L_2$.

[a] La dimostrazione è analoga per linguaggi definiti su insiemi di terminali diversi considerando come Σ l'unione degli insiemi dei terminali

Chiusura per concatenazione dei linguaggi context-free

Enunciato. Se L_1 e L_2 sono linguaggi context-free, allora lo è anche $L_1 L_2$.

Dimostrazione. Siano $L_1 = L(G_1)$ e $L_2 = L(G_2)$ con $G_1 = (V_1, \Sigma, R_1, S_1)$ e $G_2 = (V_2, \Sigma, R_2, S_2)$ e $V_1 \cap V_2 = \varnothing$.
Allora la grammatica $G' = (V_1 \cup V_2 \cup \{S\}, \Sigma, R_1 \cup R_2 \cup \{S \to S_1 S_2\}, S), S \notin V_1 \cup V_2$ genera $L_1 \cup L_2$.

Chiusura per iterazione dei linguaggi context-free

Enunciato. Se L_1 è un linguaggio context-free, allora lo è anche L_1^*.

Dimostrazione. Sia $L_1 = L(G_1)$ con $G_1 = (V_1, \Sigma, R_1, S_1)$.
Allora la grammatica $\widehat{G} = (V_1 \cup \{S\}, \Sigma, R_1 \cup \{S \to S S_1 \mid \varepsilon\}, S), S \notin V_1$ genera L_1^*.
Infatti possiamo osservare che se $L \subseteq \Sigma^*$ è un linguaggio qualsiasi, allora $L^* = L^* L \cup \{\varepsilon\}$. Infatti conterrà la stringa vuota, ovvero una concatenazione di zero parole di L unito alle concatenazioni di almeno una parola di L.
La tesi è facilmente evincibile dal fatto che le uniche due regole della grammatica da noi costruita si comportano in maniera analoga a questa osservazione.

[1] L'utilizzo della parola **inerentemente** è fatto in maniera desueta, nonostante tecnicamente corretto. Il termine infatti deriva dalla traduzione dall'inglese del termine **inherently**, presente anche nel testo consigliato di Sipser, che si traduce in maniera più puntuale in **intrinsecamente**. Si è scelto di utilizzare comunque il termine **inerentemente** poiché più diffuso nella letteratura riguardo questo argomento, e presente anche nella traduzione ufficiale del testo di Sipser, nonostante sia meno calzante. Per dubbi si veda la *definizione di inerente* e la *traduzione di inherently*.

ssendo dunque che la classe dei linguaggi context-free è chiusa per unione, concatenazione e iterazione, possiamo dire che è chiusa per tutte le operazioni regolari.[2]

Dal teorema di Kleene quindi, se dimostrassimo che tutti i linguaggi finiti sono context-free avremmo una prima dimostrazione che tutti i linguaggi regolari sono context-free.

Proposizione: Tutti i linguaggi finiti sono context-free

Enunciato. Se $L \subseteq \Sigma^*$ è finito, allora è context-free.

Dimostrazione. Se $L = \varnothing$ è generato dalla grammatica con $R = \{S \to S\}$.
Se $L = \{w_1, \dots, w_n | w_1, \dots w_n \in \Sigma^*\}$, allora $L = L(G)$ dove G ha come regole $R = \{S \to w_1 \mid \dots \mid w_n\}$.

Possiamo quindi concludere formalmente che tutti i linguaggi regolari sono context-free.

Teorema: Tutti i linguaggi regolari sono context-free

Enunciato. Se $L \subseteq \Sigma^*$ è regolare, allora è context-free.

Prima Dimostrazione Dal teorema di Kleene, L regolare $\implies$ L si ottiene da linguaggi finiti applicando $\cup, \cdot, *$. Dai teoremi precedenti si ha banalmente la tesi.

7.2.1 Grammatiche regolari

Per dare un ulteriore dimostrazione del fatto che tutti i linguaggi regolari sono context-free, possiamo definire un particolare tipo di grammatiche, dette **grammatiche regolari**, che generano esattamente i linguaggi regolari.

Grammatiche Regolari

Una grammatica context-free $G = (V, \Sigma, R, S)$ si dice **regolare** se $R \subseteq V \times \Sigma \cup V \times \{\varepsilon\} \cup V \times \Sigma V$.
Cioè se ogni regola ha una tra le seguenti forme: $X \to a$, $X \to \varepsilon$ o $X \to aY$.

Si può dimostrare, anche se non lo faremo, che un linguaggio accettato da una grammatica regolare è regolare.

Da questo possiamo ottenere un altra dimostrazione del teorema precedente

Teorema: Tutti i linguaggi regolari sono context-free

Enunciato. Se $L \subseteq \Sigma^*$ è regolare, allora è context-free.

Seconda Dimostrazione Sia $L = L(\mathcal{M})$ con $\mathcal{M} = (Q, \Sigma, \delta, q_1, F)$ DFA.
Consideriamo un insieme V equipotente a Q, cioè per ogni stato $q_i \in Q$ sia definita una variabile $X_i \in V$.
Definiamo allora una grammatica come segue:

- Se $\delta(q_i, a) = q_j$, con $q_i, q_j \in Q$ e $a \in \Sigma$, allora G avrà la regola $X_i \to aX_j$
- Se $q_i \in F$, allora G avrà la regola $X_i \to \varepsilon$

Si può facilmente vedere che la grammatica così definita, oltre a essere regolare e dunque context-free, se si sceglie come assioma X_1 genera $L(\mathcal{M})$, da cui la tesi.

[2] A titolo di notizia si può notare come queste proprietà di chiusura risultino quasi immediate dal momento che si considerano le regole di una grammatica context-free come operazioni fra linguaggi. Infatti, considerando il simbolo $|$ come $\cup$, il simbolo $\cdot$ come $\cdot$ e usando regole costruite in maniera analoga a quelle della chiusura per iterazione per simulare quest'ultima, abbiamo che ogni regola di una grammatica context-free si può interpretare come una operazione fra linguaggi e viceversa

Riprendiamo il concetto di automi per proporre un ulteriore modello di calcolo che, come vedremo, va di pari passo con **grammatiche e linguaggi context-free**.

Se abbiamo definito informalmente un DFA una particolare macchina di Turing con caratteristiche computazionali minime, ovvero una memoria finita e la capacità di funzionare esclusivamente come **accettori**, possiamo, sempre informalmente, definire un automa pushdown estendendo le caratteristiche computazionali di un DFA dandogli memoria non limitata.

In particolar modo la memoria di un automa pushdown si comporterà come una pila, ovvero seguirà il paradigma LIFO.

Definiamo quindi formalmente un automa pushdown.

Automi Pushdown

Un **automa pushdown** o PDA è una sestupla $\mathcal{M} = (Q, \Sigma, \Gamma, \delta, q_1, F)$ dove:
- Q è un insieme finito di stati;
- Σ è un insieme finito di simboli, detto alfabeto del **nastro**;
- Γ è un insieme finito di simboli, detti alfabeto della **pila**;
- δ è la funzione di transizione $\delta : Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \to \mathcal{P}(Q \times \Gamma_\varepsilon)$;
- $q_1 \in Q$ è lo stato iniziale;
- $F \subseteq Q$ è l'insieme degli stati terminali.

Dove $\Sigma_\varepsilon = \Sigma \cup \{\varepsilon\}$ e $\Gamma_\varepsilon = \Gamma \cup \{\varepsilon\}$.

È da notare che a differenza degli automi a stati finiti, in questo caso si è data direttamente una definizione non deterministica e che consente le transizioni spontanee[3]. Ciò è però poco rilevante poiché, le due definizioni sono equivalenti.

Se per Σ non troviamo nulla di nuovo rispetto ai precedenti A dei DFA o Σ delle grammatiche context-free, per Γ il discorso è un po' diverso.

Infatti, Γ non rappresenta la memoria in se, che vederemo in seguito da cosa sarà espressa matematicamente, bensì l'insieme di possibili simboli che possono essere inseriti nella memoria, il quale può come non può coincidere con Σ.

In particolare nel nostro caso, per praticità d'uso, considereremo $\Gamma = \Sigma \cup \{\$\}, \$ \notin \Sigma$, dove \$ è un simbolo speciale generico che ci aiuterà a rappresentare il fondo della pila in modo da poter correttamente contare gli elementi al suo interno.

Come di consueto per un nuovo modello di calcolo dobbiamo definire i linguaggi accettati da esso.

A differenza degli automi a stati finiti e delle grammatiche context-free, in cui è stato relativamente agevole descrivere la funzione di transizione iterata e la derivazione di una parola, per gli automi pushdown non è così immediato, dovendo tenere conto anche della coda.

Parole accettate da un PDA

Una parola $w = a_1 \ldots a_n$, $a_i \in \Sigma_\varepsilon \ \forall\ i \in \{1, \ldots, n\}$, è accettata da un PDA se:

$$\exists q_1, \ldots, q_{n+1} \in Q, q_{n+1} \in F, s_1 = \varepsilon, s_2, \ldots, s_{n+1} \in \Gamma^* : (q_{i+1}, \beta_i) \in \delta(q_i, a_i, \alpha_i) \forall i \in \{1, \ldots, n\} \tag{7.9}$$

Dove $s_1, \ldots s_{n+1}$ è una sequenza di parole formate da simboli di pila, che andranno a rappresentare l'evoluzione della nostra memoria, che dovrà partire vuota ($s_1 = \varepsilon$), e $\alpha_i, \beta_i \in \Gamma_\varepsilon$ sono due generici simboli di pila tali che $s_i = \alpha_i t, s_{i+1} = \beta_i t$ che rappresenteranno l'aggiunta in testa alla nostra memoria.
In definitiva una parola sarà accettata da un PDA se, dopo una sequenza finita di applicazioni della funzione δ, tenendo conto dell'aggiornamento della memoria, si potrà raggiungere uno stato terminale.

Diagramma di transizione

Come per gli automi a stati finiti, possiamo rappresentare un PDA con un diagramma di transizione. Andremo a rappresentare infatti la singola transizione che va da un $q_i \in Q$ a un $q_j \in Q$ con l'input a rimuovendo dalla testa della pila α e inserendoci β, ovvero una transizione del tipo $(q_j, \beta) \in \delta(q_i, a, \alpha)$, come

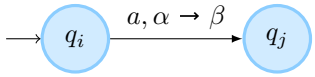
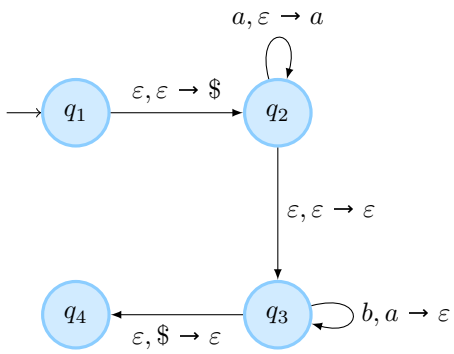


Figura 7.1: Diagramma di transizione di un PDA di uno stato

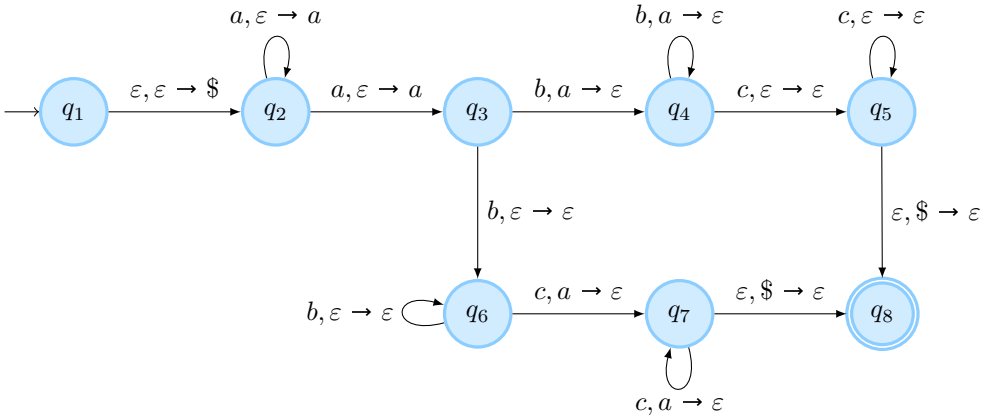
[3] Una transizione spontanea o **ε-transizione** è una transizione che avviene senza necessità di leggere nessun carattere.

Esempio: Diagramma di transizione di un PDA

Consideriamo il seguente PDA:



Avremo proprio che $L(\mathcal{M}) = \{a^n b^n | n \geq 0\}$.
Un altro linguaggio di cui possiamo scrivere un diagramma di un PDA è $L = \left\{a^m b^n c^k | m, n, k > 0 \wedge (m = n \vee n = k)\right\}$.

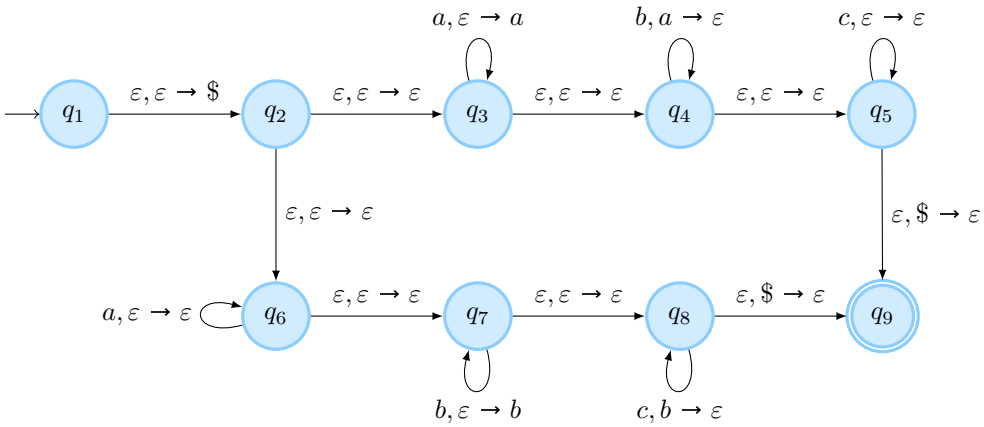


Consideriamo ora il linguaggio $L = \left\{a^i b^j c^k | i, j, k \geq 0 \wedge (i = j \vee j = k)\right\}$. Andiamo a determinare una grammatica e un PDA che accettino questo

1. **Grammatica**: Possiamo vedere il linguaggio come $L = \left\{a^i b^i c^k | i, k \geq 0\right\} \cup \left\{a^i b^k c^k | i, k \geq 0\right\}$. Definiamo dunque la grammatica con le seguenti regole:

$$\begin{aligned} S &\to X \mid Y \\ X &\to ZC \\ Y &\to AT \\ Z &\to aZb \mid \varepsilon \\ A &\to aA \mid \varepsilon \\ T &\to bTc \mid \varepsilon \\ C &\to cC \mid \varepsilon \end{aligned} \tag{7.10}$$

2. **PDA**: Possiamo definire il PDA come segue:



È importante notare che, essendo $i, j, k \geq 0$ tutte le transizioni che fanno cambiare stato devono essere spontanee, poiché $i = j = k = 0 \implies \epsilon \in L$.

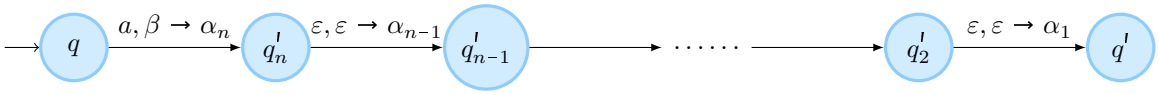
Per dimostrare il prossimo risultato importante, è necessario definire un analogo della funzione di transizione iterata per la pila.

In particolare dunque andremo a definire una funzione, e di conseguenza una notazione nei diagrammi, che dato uno stato q, un carattere del nastro a e un carattere della pila da leggere β, restituisca l'insieme delle coppie stato-parola (q_1', w) tali che q_1' è raggiungibile inserendo in pila la parola $w \in \Gamma^*$ alla rovescia, in modo da averla in ordine corretto in lettura.

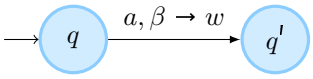
Formalmente dunque, dato $\mathcal{M} = (Q, \Sigma, \Gamma, \delta, q_1, F)$ PDA, $a \in \Sigma_\varepsilon, \beta \in \Gamma_\varepsilon, w = \alpha_1 \dots \alpha_n \in \Gamma^*, \alpha_i \in \Gamma_\varepsilon \forall i \in \{1, \dots, n\}$ diremo che

$$\left(q_1', w\right) \in \delta^*(q, a, \beta) \text{ se } \exists q_1', \dots, q_n' \in Q : (q_n, \alpha_n) \in \delta(q, a, \beta), (q_{n-1}', \alpha) \in \delta(q_n', \varepsilon), \dots (q_1', \alpha_1) \in \delta(q_2', \varepsilon, \varepsilon) \tag{7.11}$$

Diremo quindi che posso aggiungere la parola w in pila dallo stato q, leggendo a dal nastro e β dalla pila *se e solo se* nell'automa sono presenti degli stati del tipo:



Per praticità di rappresentazione, andremo a esprimere questa sequenza di stati con la seguente notazione:

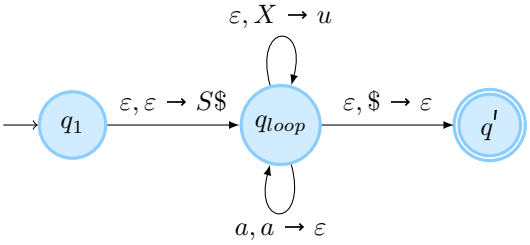


Introdotta questa nuova notazione, possiamo con molta più praticità andare a dimostrare la corrispondenza tra grammatiche context-free e PDA.

Teorema: PDA e Grammatiche context-free

Enunciato. $L \subseteq \Sigma^*$ context-free $\iff$ $\exists \mathcal{M}$ PDA tale che $L = L(\mathcal{M})$.

Dimostrazione. Dimostriamo per semplicità solo l'implicazione " $\implies$ ".
Sia $G = (V, \Sigma, R, S)$ una grammatica context-free tale che $L = L(G)$. Possiamo dunque costruire un PDA $\mathcal{M} = (Q, \Sigma, \Gamma, \delta, q_1, F)$ tale che $L = L(\mathcal{M})$ come segue:



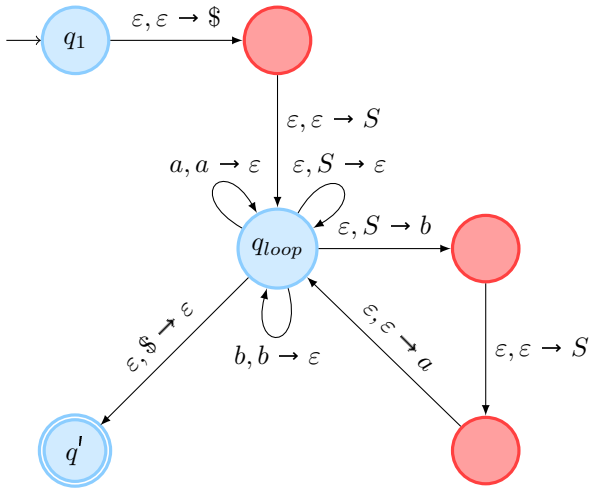
Dove con la transizione $\varepsilon, X \to u$ si denota una transizione **per ogni regola** $X \to u \in R, X \in V, u \in (V \cup \Sigma)^*$, ovvero per ogni regola della grammatica, mentre con la transizione $a, a \to \varepsilon$ si procede alla lettura dei simboli in pila rimuovendoli.
In particolare un automa così costruito inizializzerà la pila a $S\$$ per poi procedere a sostituire S con una regola della grammatica che coinvolga S e continuando ad applicare regole fino a che non venga inserita in pila come carattere in testa il primo carattere sul nastro. A quel punto il carattere verrà letto dall'altra transizione, rimuovendolo dalla pila, per poi ricominciare ad applicare regole o a leggere i caratteri.
Chiaramente una parola verrà accettata da questo PDA *se e solo se* è generata dalla grammatica, poiché l'automa non fa altro che simulare la derivazione di una parola.

Esempio: Derivazione PDA da grammatica context-free

Prendiamo per esempio la classica grammatica context-free:

$$S \to aSb \mid \varepsilon \tag{7.12}$$

Possiamo disegnare il seguente PDA:



In questo automa sono stati esplicitati gli stati intermedi dell'inserimento delle parole in pila visto precedentemente e denotati di rosso, è possibile dunque per una rappresentazione più compatta eliminare tali nodi e utilizzare la notazione vista poc'anzi.
Vediamo in ultimo l'esecuzione di questo PDA sulla parola $aabb \in L$, tenendo traccia del contenuto del nastro e della pila:

Nastro	Pila
ε	$S\$$
ε	$aSb\$$
a	$Sb\$$
a	$aSbb\$$
aa	$Sbb\$$

Nastro	Pila
aa	$bb\$$
aab	$b\$$
$aabb$	$\$$
$aabb$	ε
$aabb$	✓

Dal risultato dimostrato nel teorema 7.3 possiamo dare la terza e ultima dimostrazione del teorema 7.2.

Teorema: Tutti i linguaggi regolari sono context-free

Enunciato. Se $L \subseteq \Sigma^*$ è regolare, allora è context-free.

Terza Dimostrazione Se L regolare $\implies \exists \mathcal{M}$ DFA tale che $L = L(\mathcal{M})$. Essendo che i DFA sono un caso particolare di PDA, ovvero con $\Gamma = \{\varepsilon\}$, allora $\exists \mathcal{M}'$ PDA ottenuto da $\mathcal{M}$ tale che $L = L(\mathcal{M}')$, ovvero è context-free dal linguaggio precedente.

Forme Normali di Chomsky e Pumping Lemma

Similmente a come abbiamo fatto per i linguaggi regolari, è possibile, modificando opportunamente il **Pumping Lemma**, dimostrare una condizione necessaria per cui un linguaggio sia context-free.

Per fare ciò, però, è necessario introdurre prima un nuovo strumento di rappresentazione delle grammatiche context-free: le **forme normali di Chomsky**.

8.1 Forme normali di Chomsky

Grammatiche Context-Free in forma Normale di Chomsky

Una Grammatica Context-Free $G = (V, \Sigma, R, S)$ è in **forma normale di Chomsky** se ha solo regole del tipo:

$$\begin{aligned} &X \to YZ, && \forall X \in V, Y, Z \in V \setminus \{S\} \\ &X \to a, && \forall X \in V, a \in \Sigma \\ &S \to \varepsilon \end{aligned} \tag{8.1}$$

Nonostante questa forma normale possa sembrare molto restrittiva, è possibile dimostrare che ogni linguaggio context-free è accettato da una grammatica in forma normale di Chomsky, rendendo di fatto una forma normale esclusivamente una diversa rappresentazione di una grammatica. Vediamone un idea di dimostrazione[1].

Normalizzazione di una grammatica CF

Enunciato. Data una CFG G, esiste una CFG G' in forma normale di Chomsky tale che $L(G) = L(G')$.

Idea di Dimostrazione. Se G ha tra le regole:

$$X \to u_1 u_2 \dots u_k, u_i \in V \cup \Sigma, i \in \{1, \dots, k\} \tag{8.2}$$

Allora aggiungiamo $k - 2$ nuove variabili $X_1, \dots, X_{k-2}$ e sostituiamo la regola con:

$$\begin{aligned} X &\to u_1 X_1 \\ X_1 &\to u_2 X_2 \\ &\vdots \\ X_{k-3} &\to u_{k-2} X_{k-2} \\ X_{k-2} &\to u_{k-1} u_k \end{aligned} \tag{8.3}$$

Si avrà dunque che la derivazione sarà la stessa, ma con regole di lunghezza al più due.

[1] Durante i corsi questa dimostrazione è stata data solo negli sprazzi riportati, per mancanza di tempo. È possibile però per i più interessati recuperarla sui testi consigliati.

Per meglio comprendere il procedimento di normalizzazione, e mimando in maniera meno formale i passaggi evitati dalla dimostrazione, andiamo a normalizzare la prima grammatica context-free che abbiamo visto in questo corso.

Esempio: Normalizzazione di una grammatica CF

Per normalizzare la grammatica $G = G_0$ con regola $S \to aSb \mid \varepsilon$ passeremo per varie grammatiche intermedie non normali che ci avvicineranno sempre di più alla forma normale di Chomsky.

G_1 Iniziamo dal rimuovere la variabile iniziale S dalla parte destra delle regole, utilizzando una nuova variabile X:

$$\begin{aligned} S &\to X \\ X &\to aXb \mid \varepsilon \end{aligned} \tag{8.4}$$

G_2 Andiamo dunque a isolare i terminali in regole apposite, utilizzando due nuove variabili A, B.

$$\begin{aligned} S &\to X \\ X &\to AXB \mid \varepsilon \\ A &\to a \\ B &\to b \end{aligned} \tag{8.5}$$

G_3 Utilizzando il procedimento visto nella dimostrazione precedente, andiamo a ridurre tutte le regole di lunghezza maggiore di due, aggiungendo nuove variabili Y e Z.

$$\begin{aligned} S &\to X \\ X &\to AY \mid \varepsilon \\ Y &\to XB \\ A &\to a \\ B &\to b \end{aligned} \tag{8.6}$$

G_4 Essendo S l'unica variabile sostituibile con ε, dobbiamo rimuovere la regola $X \to \varepsilon$ per aggiungere $S \to \varepsilon$.
L'aggiunta non modifica la grammatica in quanto $S \implies \varepsilon \equiv S \implies X \implies \varepsilon$.
La rimozione però andrebbe a precludere tutte le sostituzioni di X con ε possibili nella grammatica originale.
Per ovviare a ciò, è sufficiente aggiungere per ogni regola contenente X a destra una regola identica con X rimosso da destra.

$$\begin{aligned} S &\to X \mid \varepsilon \\ X &\to AY \\ Y &\to XB \mid B \\ A &\to a \\ B &\to b \end{aligned} \tag{8.7}$$

G_5 In ultimo dobbiamo far si che le regole che sostituiscono variabili siano sempre di lunghezza due, per far ciò basta sostituire a $S \to X$ la regola $S \to AY$ arrivando a due, e alla regola $Y \to B$ la regola $Y \to b$ trasformando la regola da riguardante le variabili a riguardante i terminali, e dunque correttamente di lunghezza 1.

$$\begin{aligned} S &\to AY \mid \varepsilon \\ X &\to AY \\ Y &\to XB \mid b \\ A &\to a \\ B &\to b \end{aligned} \tag{8.8}$$

In definitiva, come abbiamo fatto per i linguaggi regolari, andiamo a dimostrare il **Pumping Lemma per i linguaggi context-free**, utilizzando la forma normale di Chomsky.

Pumping Lemma per i linguaggi context-free

Enunciato. $L \subseteq \Sigma^*$ context-free $\implies \exists p \geq 0 : x \in L \wedge |x| \geq p \implies \exists u, v, w, y, z \in \Sigma^* : x = uvwyz$ con:

1. $vy \neq \varepsilon$
2. $\forall i \geq 0, uv^iwy^iz \in L$
3. $|vwy| \leq p$

Dimostrazione. Sia $G = \{V, \Sigma, R, S\}$ una grammatica context-free in forma normale di Chomsky tale che $L = L(G)$, e sia $k = |V|$. Poniamo $p = 2^{k+1}$.
Se $x \in L$, ogni suo albero di derivazione in G è binario, ovvero ogni nodo ha al massimo due figli.
Per un albero binario di altezza h, il numero di foglie è al più 2^h, e dunque, essendo in un albero di derivazione le foglie i terminali della parola accettata, $2^h \geq |x| \geq p > 2^k$, ovvero l'altezza del suo albero di derivazione sarà almeno $k + 1$.
Quindi, dato un albero di derivazione per x con minimo numero di nodi, il percorso massimale dalla radice a una foglia incontrerà almeno $k + 1$ nodi interni, e dunque almeno $k + 1$ variabili. Per il **principio della piccionaia** visto nello scorso capitolo, $\exists T \in V$ che compare almeno 2 volte nel percorso.
Possiamo dunque suddividere la parola come segue:

- u sarà formata dalle prime lettere di x fino alla lettera nella foglia più a sinistra del sottoalbero radicato nella penultima occorrenza di T esclusa;
- v sarà formata dalle lettere dalla foglia più a sinistra del sottoalbero radicato nella penultima occorrenza di T inclusa fino alla lettera nella foglia più a sinistra del sottoalbero radicato nell'ultima occorrenza di T esclusa;
- w sarà formata dalle lettere dalla foglia più a sinistra del sottoalbero radicato nell'ultima occorrenza di T inclusa fino alla lettera nella foglia più a destra del sottoalbero radicato nell'ultima occorrenza di T esclusa;
- y sarà formata dalle lettere dalla foglia più a destra del sottoalbero radicato nell'ultima occorrenza di T inclusa fino alla lettera nella foglia più a destra del sottoalbero radicato nella penultima occorrenza di T esclusa;
- z sarà formata dalle ultime lettere di x a partire dalla foglia più a destra del sottoalbero radicato nella penultima occorrenza di T inclusa.

Per comprendere meglio questa suddivisione è possibile osservare la seguente figura:
Possiamo dunque dimostrare i tre punti del pumping lemma:

1. Se per assurdo $v = \varepsilon = y$, essendo che dalle regole della grammatica possiamo derivare da T sia la parola vwy che la parola w, si avrebbe che $vwy = w$, ovvero $x = uvwyz = uwz$, potendo dunque eliminare dall'albero di derivazione i rami che portano alle foglie di v e y, ottenendo un diverso albero di derivazione per x. Questo albero però avrà meno nodi di quello originario, violando l'ipotesi di minimalità dell'albero.
2. Per $i = 0$, sappiamo già dal punto precedente che $uv^0wy^0z = uwz \in L$, poiché abbiamo costruito un albero di derivazione su G che deriva uwz. Viceversa, è sufficiente notare che sostituendo il sottoalbero radicato all'ultima occorrenza di T con il sottoalbero radicato nella penultima occorrenza di T si otterrà un nuovo albero di derivazione su G per la parola uv^2wy^2z, che è quindi ancora una parola di L. Iterando questo ragionamento un numero arbitrario di volte otteniamo la tesi del punto 2.
3. Possiamo osservare che il principio della piccionaia si applica anche alle ultimi $k+1$ variabili incontrate nel percorso massimale dalla radice a una foglia. Dunque possiamo scegliere T in modo che la sua penultima occorrenza sia al più $k + 1$ nodi sopra una foglia, ovvero il sottoalbero in essa radicato abbia altezza $k + 1$, ovvero al più $2^{k+1} = p$ foglie, da cui $vwy \leq p$.

Come per i linguaggi regolari, essendo il **Pumping Lemma** una condizione sufficiente ma non necessaria, non è utile a dimostrare se un linguaggio è context-free, ma solo a dimostrare che non lo è. Vediamone in particolare un esempio.

Esempio: Linguaggi non context-free

Consideriamo il linguaggio $L = \{a^nb^nc^n \mid n \geq 0\}$. Dimostriamo che questo linguaggio non è context-free.

Supponiamo infatti per assurdo che lo sia, e sia p la lunghezza di pumping corrispondente. Scegliamo $x = a^pb^pc^p$.
Dovremmo dunque avere che $x = uvwyz$ con $vy \neq \varepsilon$. Potremmo avere allora i seguenti casi:

1. v e y contengono al più una lettera ciascuna, ovvero sono della forma $v = d^h, y = e^k$ per $d, e \in \{a, b, c\}$ e $h, k \geq 0$. Scegliendo $i = 0$ andrò necessariamente a cancellare occorrenze di al più due lettere diverse, e dunque avrò una parola con almeno una lettera con più occorrenze delle altre, di conseguenze non appartenente a L.
2. v (oppure y) contiene 2 lettere diverse tra a, b, c. Scegliendo $i = 2$ otterremo necessariamente una parola che non rispetta l'ordine delle occorrenze, poiché si avrà una parola in cui vi è l'alternarsi di lettere diverse, ovvero quelle di v, non appartenendo a L.
3. v (oppure y) non può contenere tutte e tre le lettere, poiché violerebbe il punto tre del pumping lemma.

Violando in ogni caso il pumping lemma, abbiamo dimostrato che L non è context-free.

Questo ultimo esempio ci evidenzia che, nonostante i linguaggi context-free godano delle proprietà di chiusura per unione, concatenazione e iterazione, non godono di tutte le proprietà di chiusura dei linguaggi regolari.

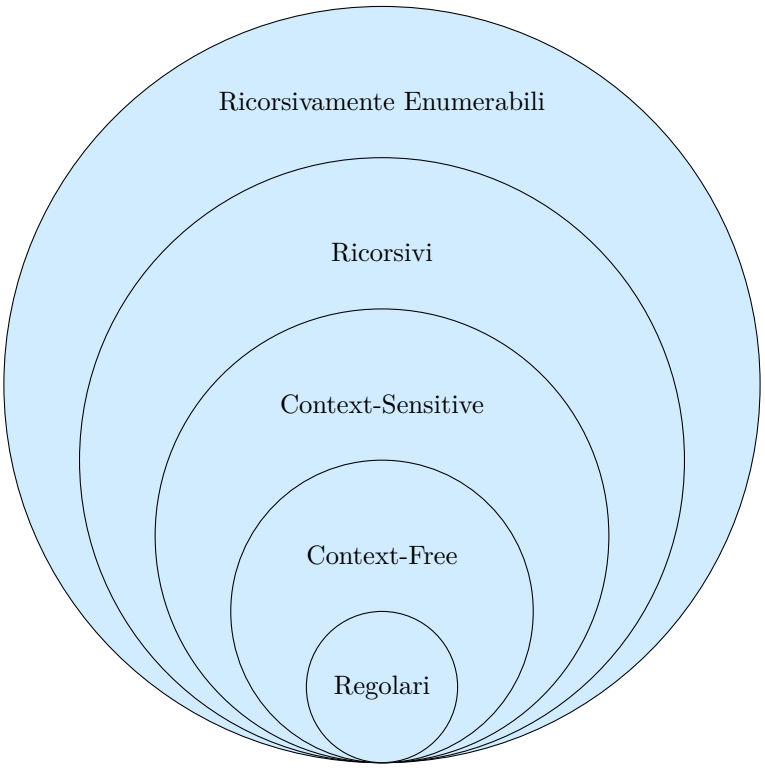
Infatti considerando i linguaggi $L = \{a^i b^i c^j | i, j \geq 0\}$ e $L' = \{a^i b^j c^j | i, j \geq 0\}$ già dimostrati come context-free, si ha che la loro intersezione non è context-free come appena dimostrato. Dunque l'intersezione di due linguaggi context-free non è necessariamente context-free.

La famiglia dei linguaggi context-free inoltre non è chiusa neanche rispetto al complemento, poiché altrimenti dalle leggi di De Morgan (dato che $L \cap L' = \overline{\overline{L} \cup \overline{L'}}$) si avrebbe che i linguaggi context-free sarebbero chiusi rispetto all'intersezione, che abbiamo appena dimostrato non essere vero.

8.3 Gerarchia di Chomsky-Schützenberger

Come visto per il precedente modello di calcolo analizzato nel capitolo **??**, possiamo infine riassumere i risultati ottenuti in un diagramma riassuntivo che evidenzi la gerarchia tra i linguaggi. A questa gerarchia proposta originariamente da Chomsky, sono stati aggiunti successivamente i linguaggi ricorsivi (o decidibili).

Andiamo a vedere questa schematizzazione



Linguaggi	Grammatiche	Automi
Regolari	Regolari • $X \to aY, \quad X, Y \in V$; • $X \to a, \quad a \in \Sigma$; • $X \to \varepsilon$;	DFA
Context-Free	Context-Free • $X \to w, \quad X \in V, w \in (V \cup \Sigma)^*$	PDA
Context-Sensitive	Context-Sensitive • $\alpha X \beta \to \alpha w \beta, \quad X \in V, \alpha, w, \beta \in (V \cup \Sigma)^*, w \neq \varepsilon$	Macchine di Turing limitate linearmente
Ricorsivi	*Non nota*	Macchine di Turing terminanti
Ricorsivamente Enumerabili	Senza restrizioni • $u \to w, \quad u, w \in (V \cup \Sigma)^*$	Macchine di Turing

Esercizi della 1° prova intercorso

In questo capitolo sono riportati alcuni esercizi d'esame, con le relative soluzioni, che possono essere utili per la preparazione dell'esame di Elementi di Informatica Teorica tenuto dal professore A. De Luca per l'annoA.A 2025–2026.

Il capitolo sarà suddiviso in due sezioni: la prima conterrà gli esercizi d'esame relativi alla prima parte del corso, argomento di intercorso, mentre la seconda conterrà gli esercizi d'esame relativi alla seconda parte del corso, argomento di fine corso.

9.1 2023

9.1.1 20 Gennaio 2023

Esercizio 4

Per $n \in \mathbb{N}$, sia $f(n)$ il prodotto dei numeri primi minori o uguali a n (con $f(0) = f(1) = 0$). Mostrare che f è ricorsiva primitiva.

Soluzione: Possiamo definire $f(n)$ come segue:

$$f(n) = \prod_{i=0}^{n} (i \cdot Primo(i) + (\neg Primo(i))) \tag{9.1}$$

Esercizio 5

Dimostrare che se $S \subseteq \mathbb{N}$ è un insieme finito, allora $\overline{K} \cup S$ non è ricorsivamente enumerabile.

Soluzione: Supponiamo per assurdo che $\overline{K} \cup S$ sia ricorsivamente enumerabile. Inoltre possiamo notare che $S \setminus K$ è un insieme finito, dunque è ricorsivo. Ma allora si ha che:

$$((\overline{K} \cup S) \cap \overline{S}) \cup (S \setminus K) = \overline{K} \tag{9.2}$$

Ciò porta a contraddizione.

Esercizio 6

Sia g la funzione unaria tale che

$$g(\langle x, y \rangle) = \begin{cases} m + 1, & \text{HALT}(x, y) \\ \uparrow, & \text{altrimenti} \end{cases} \tag{9.3}$$

per ogni $m, n \in \mathbb{N}$. Mostrare che g è parzialmente calcolabile.

Soluzione: Possiamo riscrivere g come:

$$g(z) = \begin{cases} l(z) + 1, & \Phi(l(z), r(z)) \downarrow \\ \uparrow, & \text{altrimenti} \end{cases} \tag{9.4}$$

Possiamo dunque vedere che il seguente programma calcola g:

```
Z ← Φ(l(X), r(X))
Y ← l(X) + 1
```

9.1.2 03 Marzo 2023

Esercizio 4

Sia $f(n)$ la somma di tutti i numeri naturali con la stessa parità di n fino a n incluso. Mostrare che la funzione f è ricorsiva primitiva.

Soluzione: Possiamo definire $f(n)$ come segue:

$$f(n) = \sum_{i=0}^{n} (i \cdot (n \bmod 2 = i \bmod 2)) \tag{9.5}$$

Esercizio 5

Sia $P(x)$ un predicato calcolabile. Dimostrare che l'insieme $S_p = \{x \in \mathbb{N} | \exists y : y > x \wedge P(y)\}$ è ricorsivamente enumerabile.

Soluzione: S_p è ricorsivamente enumerabile $\iff \exists g$ parzialmente calcolabile $: g(x) \downarrow \iff x \in S_p$.

Tale funzione è calcolata dal seguente programma:

```
[A] IF (P(Z) ∧ Z>X) GOTO E
    Z ← Z + 1
    GOTO A
```

Esercizio 6

Mostrare che non esiste una funzione calcolabile $g(x_1, x_2)$ tale che:

$$g(x_1, x_2) = 0 \iff \Phi(x_1, x_2) \downarrow \tag{9.6}$$

Soluzione:

$$g(x_1, x_2) = 0 \iff \Phi(x_1, x_2) \downarrow \iff HALT(x_1, x_2) \tag{9.7}$$

Ma allora $\alpha(g) = HALT$, dunque non calcolabile.

9.1.3 13 Giugno 2023

Esercizio 4

Per $n \geq 0$, sia $f(n)$ il più piccolo numero primo che non divide $n + 1$. Dimostrare che f è una funzione ricorsiva primitiva.

Soluzione: Possiamo definire $f(n)$ come segue:

$$f(n) = \min_{i \leq P_{(n+1)}} (Primo(i) \wedge \neg(i | (n+1))) \tag{9.8}$$

Esercizio 5

Per $m, n \in \mathbb{N}$, sia

$$g(m, n) = \begin{cases} \Phi(m, n), & m > n \wedge HALT(m, n) \\ \uparrow, & \text{altrimenti} \end{cases} \tag{9.9}$$

La funzione g è parzialmente calcolabile? Dimostrare o confutare.

Soluzione: Consideriamo il seguente programma:

```
    Y ← Φ(X1, X2)
[A] IF (X1 ⩽ X2) GOTO A
```

Questo programma calcola g.

Esercizio 6

Sia $C \subseteq \mathbb{N}$ un insieme ricorsivo, e sia $D = \{x \in \mathbb{N} | x \in W_x \setminus C\}$. Mostrare che D è ricorsivamente enumerabile.

9.2 2022

9.2.1 16 Marzo 2022

Esercizio 4

Sia $B \subseteq \mathbb{N}$ un insieme ricorsivo primitivo, e Q il predicato definito da

$$Q(x) \iff \text{tutti i divisori di } x+1 \text{ appartengono a } B \tag{9.10}$$

Mostrare che Q è ricorsivo primitivo.

Soluzione: Possiamo definire $Q(x)$ come segue:

$$Q(x) = \forall n \leq x+1(\neg(n|(x+1)) \vee f_B(n)) \tag{9.11}$$

Essendo Q composizione di **funzioni ricorsive primitive** con **l'operatore di quantificazione universale limitata** e di **congiunzione**, Q è **ricorsiva primitiva**.

Esercizio 5

La funzione f definita da

$$f(x) = \begin{cases} x+1, & \neg(\exists z : STP(x,x,z)) \\ \uparrow, & \text{altrimenti} \end{cases} \tag{9.12}$$

è parzilmente calcolabile? Dimostrare o confutare.

Soluzione: Possiamo riscrivere $\neg(\exists z : STP(x,x,z)) \iff \Phi(x,x)\uparrow$.

Sia per assurdo f parzialmente calcolabile. Sia p il numero del programma che la calcola. Allora abbiamo:

$$\begin{aligned} &\forall x \in \mathbb{N} : \\ &\quad \Psi^{(1)}_{\mathcal{P}_p}(x) = f(x)\uparrow \iff \Phi(x,x)\downarrow \iff \Psi^{(1)}_{\mathcal{P}_x}(x)\downarrow \end{aligned} \tag{9.13}$$

Per $x = p$ andiamo a contraddizione.

Esercizio 6

Mostrare che l'insieme $S = \{[i,j,k] | HALT(i,k) \wedge HALT(j,k)\}$ è ricorsivamente enumerabile.

Soluzione: Riscriviamo l'insieme come segue:

$$S = \{z \in \mathbb{N} | HALT((z)_1,(z)_3) \wedge HALT((z)_2,(z)_3)\} = \{z \in \mathbb{N} | \Phi((z)_1,(z)_3)\downarrow \wedge \Phi((z)_2,(z)_3)\downarrow\} \tag{9.14}$$

La funzione che descrive S non è automaticamente parzialmente calcolabile poichè le funzioni parzialmente calcolabili non sono chiuse rispetto a $\wedge$. Consideriamo dunque il programma seguente:

```
[A] IF (X == 0 ∨Lt(x) > 3) GOTO A
    Z1 ← Φ((X)_1,(X)_3)
    Z2 ← Φ((X)_2,(X)_3)
```

Questo programma termina se e solo se $x \in S$. Dunque S è ricorsivamente enumerabile.

9.2.2 07 Aprile 2022

Esercizio 4

Per $x, y \in \mathbb{N}$, sia $f(x,y)$ il più piccolo multiplo di $y+1$ che sia maggiore di x. Mostrare che la funzione f è ricorsiva primitiva.

Soluzione: Possiamo definire $f(x, y)$ come segue:

$$f(x, y) = \min_{n \leq x(y+1)} (((y+1)|n) \wedge n > x) \tag{9.15}$$

Essendo f composizione di **funzioni ricorsive primitive** con **l'operatore di quantificazione universale limitata** e di **congiunzione**, f è **ricorsiva primitiva**.

Esercizio 5

Dimostrare o confutare: la funzione g definita da

$$g(x_1, x_2) = \begin{cases} 0, HALT(x_1, x_2) \vee x_1 \in K \\ \uparrow, \text{altrimenti} \end{cases} \tag{9.16}$$

è parzialmente calcolabile?

Soluzione: Possiamo riscrivere $g(x_1, x_2)$ come segue:

$$g(x_1, x_2) = \begin{cases} 0, \Phi(x_1, x_2) \downarrow \vee \Phi(x_1, x_1) \downarrow \\ \uparrow, \text{altrimenti} \end{cases} \tag{9.17}$$

Allora g è calcolata dal seguente programma:

```
[A] IF STP(X1,X2,Z) GOTO E
    IF STP(X1,X1,Z) GOTO E
    Z ← Z + 1
    GOTO A
```

Esercizio 6

Sia $B \subseteq \mathbb{N}$ un insieme ricorsivo. Mostrare che l'insieme

$$C = \{\langle m, n \rangle | \Phi(m, n) \downarrow \wedge \Phi(m, n) \notin B\} \tag{9.18}$$

è ricorsivamente enumerabile?

Soluzione: $C = \{\langle m, n \rangle | \Phi(m, n) \downarrow \wedge \Phi(m, n) \notin B\} = \{z \in \mathbb{N} | \Phi(l(z), r(z)) \downarrow \wedge \Phi(l(z), r(z)) \notin B\}$.

Allora il seguente programma termina solo su input appartenti a C:

```
    Z ← Φ(l(X), r(X))
[A] IF f_B(Z) GOTO A
```

9.2.3 20 Luglio 2022

Esercizio 4

Per $x, y \in \mathbb{N}$, sia $P(x, y)$ vero se e solo se x è maggiore del più grande divisore primo di $y + 1$. Mostrare che P è ricorsivo primitivo.

Soluzione: Possiamo definire $P(x, y)$ come segue:

$$P(x, y) = \max_{n \leq y+1} (Primo(n) \wedge n | (y+1)) < x \tag{9.19}$$

oppure come:

$$P(x, y) = P_{Lt(y+1)} < x \tag{9.20}$$

Essendo P composizione di **funzioni ricorsive primitive** con **l'operatore di quantificazione universale limitata** e di **congiunzione**, P è **ricorsiva primitiva**.

Esercizio 5

Dimostrare o confutare: la funzione g definita da

$$g(x_1, x_2) = \begin{cases} 0, & x_2 \in K \vee \Phi(x_1, x_2) \downarrow \\ \uparrow, & \text{altrimenti} \end{cases} \tag{9.21}$$

è parzialmente calcolabile?

Soluzione: Possiamo riscrivere $g(x_1, x_2)$ come segue:

$$g(x_1, x_2) = \begin{cases} 0, & \Phi(x_2, x_2) \downarrow \vee \Phi(x_1, x_2) \downarrow \\ \uparrow, & \text{altrimenti} \end{cases} \tag{9.22}$$

Allora g è calcolata dal seguente programma:

```
[A] IF STP(X2,X2,Z) GOTO E
    IF STP(X1,X2,Z) GOTO E
    Z ← Z + 1
    GOTO A
```

Esercizio 6

Mostrare che l'insieme $C = \{\langle m, n\rangle | \Phi(m, n) \uparrow\}$ non è ricorsivamente enumerabile.

Soluzione: Supponiamo per assurdo che C sia ricorsivamente enumerabile.

Riscriviamo dunque C come segue:

$$C = \{\langle m, n\rangle | \Phi(m, n) \uparrow\} = \{z \in \mathbb{N} | \Phi(l(z), r(z)) \uparrow\} \tag{9.23}$$

Ma allora la funzione f che termina solo su input $z \in C$ termina se e solo se $\Phi(l(z), r(z)) \uparrow$. Sia p il numero del programma che calcola f. Allora abbiamo:

```
[A] IF STP(X,p,Z) GOTO E
    IF STP(l(X),r(X),Z) GOTO B
    Z ← Z + 1
    GOTO A
[B] Y ← Y + 1
```

questo programma calcola $HALT(l(x), r(x))$. Il che porta a contraddizione.

Soluzione 2: Sia C ricorsivamente enumerabile. Consideriamo l'isieme $B = \{z \in \mathbb{N} | l(z) = r(z)\}$. Questo insieme è chiaramente ricorsivo primitivo.

Ma allora $C \cap B$ è ricorsivamente enumerabile. Ma se vediamo che $C = \left\{z | l(z) \in \overline{W_{r(z)}}\right\}$ abbiamo che:

$$A = C \cap B = \left\{z | l(z) \in \overline{W_{r(z)}} \wedge l(z) = r(z)\right\} = \left\{z | l(z) \in \overline{K} \wedge r(z) \in \overline{K}\right\} \tag{9.24}$$

Se questo fosse ricorsicamente enumerabile, potrei trovare una funzione parziale che termina solo su input $x \in K$. Questo porta ad assurdo.

9.2.4 13 Settembre 2022

Esercizio 4

Sia $A = \{\langle x, y\rangle | Primo(x) \wedge Primo(y) \wedge |x - y| = 2\}$. Dimostrare che A è un insieme ricorsivo primitivo.

Soluzione: Se riscriviamo A come segue:

$$A = \{\langle x, y\rangle | Primo(x) \wedge Primo(y) \wedge |x - y| = 2\} = \{z | Primo(l(z)) \wedge Primo(r(z)) \wedge |l(z) - r(z)| = 2\} \tag{9.25}$$

Abbiamo che la funzione caratteristica di A è composta di funzioni ricorsive primitive, dunque A è ricorsivo primitivo.

Esercizio 5

Sia f una funzione ricorsiva primitiva, e $B = \{x | f(x) \in K\}$. Mostrare che B è ricorsivamente enumerabile.

Soluzione: $B = \{x | f(x) \in K\} = \{x | \Phi(f(x), f(x)) \downarrow\}$. Essendo composizione di parzialmente calcolabili, B è ricorsivamente enumerabile.

Esercizio 6

Sia

$$g(x) = \begin{cases} x + 1, & \exists z \in \mathbb{N} : STP(x, x, z) \\ 0, & \text{altrimenti} \end{cases} \tag{9.26}$$

g è una funzione calcolabile? Dimostrare o confutare.

Soluzione: Supponiamo per assurdo che g sia calcolabile. Allora il seguente programma calcola $HALT(x, x)$:

```
IF (g(x) ≠ x + 1) GOTO E
Y ← Y + 1
```

Alternativamente si può vedere come $HALT(x, x) = \alpha\alpha(g(x))$. Altra contraddizione.

9.3.1 22 Giugno 2021

Esercizio 4

Per ogni $n \in \mathbb{N}$ sia $f(n)$ il numero di coppie di numeri primi (p, q) tali che $p \leq q \wedge n = p + q$. Ad esempio $f(3) = 0$ e $f(10) = 2$ poichè $10 = 3 + 7 = 5 + 5$. Mostrare che f è ricorsiva primitiva.

Soluzione: Possiamo definire $f(n)$ come segue:

$$f(n) = \sum_{q=0}^{n} Primo(q) \cdot \exists p \leq q(Primo(p) \wedge n = p + q) \tag{9.27}$$

Essendo f composizione di **funzioni ricorsive primitive** con **l'operatore di sommatoria** e di **quantificazione esistenziale limitata**, f è **ricorsiva primitiva**.

Un modo alternativo per scrivere $f(n)$ è:

$$f(n) = \sum_{q=0}^{n} \left(Primo(q) \cdot \sum_{p=0}^{q} (Primo(p) \wedge n = p + q) \right) \tag{9.28}$$

Esercizio 5

Sia g la funzione parziale binaria definita come segue:

$$g(x_1, x_2) = \begin{cases} \langle x_1, x_2 \rangle, & \exists z \in \mathbb{N} : STP(x_1, x_2, z) \\ \uparrow, & \text{altrimenti} \end{cases} \tag{9.29}$$

La funzione g è parzialmente calcolabile? Dimostrare o confutare.

Soluzione: Possiamo riscrivere $\exists z \in \mathbb{N} : STP(x_1, x_2, z) \iff \Phi(x_1, x_2) \downarrow$. Quindi la funzione f è calcolata dal seguente programma:

```
1   Z ← Φ(X1, X2)
2   Y ← ⟨X1, X2⟩
```

Esercizio 6

Mostrare che l'insieme $K_p = \{p \in \mathbb{N} | Primo(p) \wedge HALT(p, p)\}$ è ricorsivamente enumerabile.

Soluzione: $K_p = \{p \in \mathbb{N} | Primo(p) \wedge HALT(p, p)\} = \{p \in \mathbb{N} | Primo(p) \wedge \Phi(p, p) \downarrow\}$. Tale funzione è calcolata dal seguente programma:

```
1  [A] IF ¬Primo(X) GOTO A
2      Z ← Φ(X, X)
```

Soluzione 2: $K_p = K \cap P$ dove P è l'insieme dei numeri primi, ricorsivamente enumerabile essendo $Primo$ ricorsivamente enumerabile.

9.3.2 20 Luglio 2021

Esercizio 4

Sia $B \subseteq \mathbb{N}$ ricorsivo primitivo e $C = \{(x, y) | f(x, y) \in B\}$ dove f è una funzione ricorsiva primitiva. Mostrare che C è ricorsivo primitivo.

Soluzione: Essendo B ricorsivo primitivo, f_B è ricorsiva primitiva. Allora possiamo dire che:

$$f_C = f_B(f(x, y)) \tag{9.30}$$

Essendo f_C composizione di funzioni ricorsive primitive, f_C è ricorsiva primitiva, dunque C è ricorsivo primitivo.

Esercizio 5

Sia $D \subseteq \mathbb{N}$ un insieme non ricorsivo, e sia g una funzione totale unaria tale che $x|g(x) \iff x \in D$. Mostrare che g non è calcolabile.

Soluzione: Supponiamo per assurdo che g sia calcolabile. Allora $f_D = x|g(x)$ è calcolabile, ma D non è ricorsivo, ciò porta a contraddizione.

Esercizio 6

Sia h una funzione unaria parzialmente calcolabile. Dimostrare che è ricorsivamente enumerabile l'insieme:

$$H = \{x \in \mathbb{N} | h(x) \downarrow \wedge h(x) \in W_x\} \tag{9.31}$$

Soluzione: $H = \{x \in \mathbb{N} | h(x) \downarrow \wedge h(x) \in W_x\} = \{x \in \mathbb{N} | \Phi(h(x), x) \downarrow\}$. Essendo composizione di parzialmente calcolabili, H è ricorsivamente enumerabile.

9.3.3 19 Gennaio 2021

Esercizio Moodle 2

Sia $f(x, y)$ il più piccolo intero $n > x$ che sia multiplo di $y + 1$. Mostrare che f è ricorsiva primitiva.

Soluzione: Possiamo definire $f(x, y)$ come segue:

$$f(x, y) = \min_{n \leq x(y+1)} ((y + 1)|n \wedge n > x) \tag{9.32}$$

9.4 2020

9.4.1 20 Gennaio 2020

Esercizio 4

Sia $\omega(n)$ il numero di fattori primi distinti di un numero naturale n con $\omega(1) = 0$ e $\omega(0) = 0$. Mostrare che ω è ricorsiva primitiva.

Soluzione: Possiamo definire $\omega(n)$ come segue:

$$\omega(n) = \sum_{i=0}^{n} Primo(i) \wedge i|n \tag{9.33}$$

Essendo g composizione di **funzioni ricorsive primitive** con **l'operatore di sommatoria** g è ricorsivamente enumerabile.

Esercizio 5

Sia f una funzione ricorsiva primitiva, e

$$B = \left\{1 + \sum_{i=0}^{x} f(i) | x \in \mathbb{N}\right\} \tag{9.34}$$

Dimostrare che B è ricorsivamente enumerabile.

Soluzione: B è ricorsivamente enumerabile $\iff \exists f'$ ricorsiva primitiva : $B = \left\{f'(x) | x \in \mathbb{N}\right\}$.

Ma essendo $f'(x) = 1 + \sum_{i=0}^{x} f(i)$ composizione di funzioni ricorsive primitive con l'operatore di sommatoria è anch'essa ricorsiva primitiva.

Esercizio 6

Sia $H = \left\{< x, y > | x \in W_{2y}\right\}$. L'insieme H è ricorsivamente enumerabile? Dimostrare o Confutare.

Soluzione: $H = \left\{z \in \mathbb{N} | l(z) \in W_{2r(z)} \iff \Phi(l(z), 2r(z)) \downarrow\right\}$. Essendo composizione di parzialmente calcolabili, H è ricorsivamente enumerabile.

9.4.2 18 Febbraio 2020

Esercizio 2

Sia A un insieme ricorsivo primitivo e sia $B = \{< z_1, z_2 > | z_1 \in A \wedge z_2 \notin A\}$. Mostrare che B è un insieme ricorsivo.

Soluzione 1: Scriviamo B come segue:

$$B = \{x \in \mathbb{N} | f_A(l(x)) \wedge \neq f_A(r(x))\} \tag{9.35}$$

Essendo A ricorsivo primitivo, f_A è ricorsiva primitiva. Ma allora B è ricorsivo primitivo, poichè composizione di funzioni ricorsive primitive con l'operatore di congiunzione.

Soluzione 2: B è ricorsivo $\iff$ $B \wedge \overline{B}$ sono ricorsivamente enumerabili.

B: Possiamo riscrivere B come:

$$B = \left\{x \in \mathbb{N} | l(x) \in A \wedge r(x) \in \overline{A}\right\} \tag{9.36}$$

Essendo A ricorsivo, A e $\overline{A}$ sono ricorsivamente enumerabili, dunque esistono due funzioni parzialmente calcolabili f e f' a loro associate. Possiamo dunque riscrivere B come:

$$B = \left\{x \in \mathbb{N} | f(l(x)) \wedge f'(r(x))\right\} \tag{9.37}$$

Questa funzione è parzialmente calcolabile, poichè calcolata dal seguente programma:

```
    Z1 ← l(X)
    Z2 ← r(X)
    Z3 ← f(Z1)
    Z4 ← f'(Z2)
```

$\overline{B}$: Possiamo riscrivere $\overline{B}$ usando le leggi di De Morgan come:

$$B = \left\{x \in \mathbb{N} | l(x) \notin A \vee r(x) \notin \overline{A}\right\} = \left\{x \in \mathbb{N} | l(x) \in \overline{A} \vee r(x) \in A\right\} \tag{9.38}$$

Essendo A ricorsivo, A e $\overline{A}$ sono ricorsivamente enumerabili, dunque esistono due funzioni parzialmente calcolabili f e f' a loro associate. Possiamo dunque riscrivere B come:

$$B = \left\{x \in \mathbb{N} | f'(l(x)) \vee f(r(x))\right\} \tag{9.39}$$

Siano p, q rispettivamente i numeri dei programmi che calcolano f, f'. Questa funzione è parzialmente calcolabile, poichè calcolata dal seguente programma:

```
    Z1 ← l(X)
    Z2 ← r(X)
[B] IF STP(Z1,q,Z3) GOTO E
    IF STP(Z2,p,Z3) GOTO E
    Z3 ← Z3 + 1
    GOTO B
```

Esercizio 3

Sia:

$$f(x) = \begin{cases} 1, & x \text{ è dispari e } x \in \overline{K} \\ x, & x \text{ è pari e } x \in \overline{K} \\ \uparrow, \text{altrimenti} & \end{cases} \tag{9.40}$$

La funzione f è calcolabile? Dimostrare o confutare.

Soluzione: sia f calcolabile e p il numero di programma che la calcola. Allora abbiamo:

$$\begin{aligned} &\forall x \in \mathbb{N}: \\ &\Psi^{(1)}_{\mathcal{P}_p}(x) = f(x) \uparrow \iff x \in K \iff \Phi(x,x) \downarrow \iff \Psi^{(1)}_{\mathcal{P}_x}(x) \downarrow \end{aligned} \tag{9.41}$$

In particolare però per $x = p$ abbiamo che $\Psi^{(1)}_{\mathcal{P}_p}(p) \uparrow \iff \Psi^{(1)}_{\mathcal{P}_p}(p) \downarrow$, che porta a contraddizione.

Esercizio 6

Sia $M_n = 2^n - 1$ per ogni $n \in \mathbb{N}$, e sia $g(x)$ la somma di tutti gli M_n che siano numeri primi tali che $n \leq x$. Mostrare che g è ricorsiva primitiva.

Soluzione: Essendo che $M_n =< n, 0 >$ è ricorsivamente primitiva. Possiamo definire $g(x)$ come segue:

$$g(x) = \sum_{n=0}^{x} M_n \cdot Primo(M_n) \tag{9.42}$$

Essendo g composizione di **funzioni ricorsive primitive** con **l'operatore di sommatoria**, g è **ricorsiva primitiva**.

9.4.3 23 Giugno 2020

Esercizio Moodle 1

Dimostrare che l'insieme $S = \{< x, y > | HALT(y, x)\}$ è ricorsivamente enumerabile.

Soluzione: $S = \{< x, y > | HALT(y, x)\} = \{< x, y > | \Phi(y, x) \downarrow\} = \{z \in \mathbb{N} | \Phi(r(z), l(z)) \downarrow\}$. Essendo composizione di parzialmente calcolabili, S è ricorsivamente enumerabile.

9.4.4 16 Ottobre 2020

Esercizio Moodle (?)

Qual è il predicato $p(x, z)$ che rende l'insieme $B = \{x | \exists z \in \mathbb{N} : p(x, z)\}$ ricorsivamente enumerabile ma non ricorsivo?

Soluzione: $p = STP(x, x, z)$

9.5 2019

9.5.1 20 Giugno 2019

Esercizio 4

Sia:

$$P(x, y) = \begin{cases} 1 & x \text{esistono almeno } y \text{ numeri naturali } n \text{ tali che } n \leq x \text{ e } n \text{ è quadrato perfetto} \\ 0 & \text{altrimenti} \end{cases} \tag{9.43}$$

Mostrare che $P(x, y)$ è una predicato ricorsiva primitiva.

Soluzione: Possiamo definire $P(x, y)$ come segue:

$$P(x, y) = y \geq \sum_{n=0}^{x} \exists t \leq n(t^2 = n) \tag{9.44}$$

Essendo g composizione di **funzioni ricorsive primitive** con **l'operatore di sommatoria** e di **quantificazione esistenziale limitata**, g è **ricorsiva primitiva**.

Esercizio 5

Sia $f(x) > x^2 \iff x \notin \overline{K}$. Mostrare che f non è calcoalbile.

Soluzione: $f(x) > x^2 \iff x \notin \overline{K} \iff x \in K \iff Phi(x, x) \downarrow \iff HALT(x, x)$. Supponiamo ora per assurdo che f sia calcolabile. Allora potremmo usarla come macro in un programma.

Ma il seguente programma calcolerebbe $HALT(x, x)$, che porta a assurdo:

```
IF f(X) ≤ X^2 GOTO E
Y ← Y + 1
```

Esercizio 6

Sia $R = \{< r_1, r_2 > | r_1 \in W_{(r_1+r_2)}\}$. Mostrare che R è ricorsivamente enumerabile.

Soluzione: Dalla definizione di W_n abbiamo che

$$R = \{< r_1, r_2 > | r_1 \in W_{(r_1+r_2)} \iff \Phi(r_1, r_1 + r_2) \downarrow\} = n \in \mathbb{N} | \Phi(l(n), l(n) + r(n)) \downarrow \tag{9.45}$$

Essendo composizione di parizalmente calcolabili, R è ricorsivmane enumerabile

9.5.2 17 Luglio 2019

Esercizio 4

Sia $A = \{(x_1, x_2) | f(x_1, x_2) \downarrow\}$ con f funzione parzialmente calcolabile. Mostrare che $B = \{< x_1, x_2 > | (x_1, x_2) \in A\}$ è ricorsivamente enumerabile.

Soluzione: B è ricorsivamente enumerabile $\iff \exists g$ parzialmente calcolabile : $g(x) \downarrow \iff x \in B$.

Consideriamo allora il seguente programma $\mathcal{P}$:

```
Z1 ← l(X)
Z2 ← r(X)
Y ← f(Z1, Z2)
```

Questo programma termina $\iff x \in B$. Dunque con $g = \Psi_{\mathcal{P}}^{(1)}$ abbiamo la tesi.

Esercizio 5

Sia c una costante e sia $f(x)$ il più grande numero naturale $y \leq x$ che è multiplo di c. Mostrare che f è ricorsiva primitiva.

Soluzione: Possiamo definire $f(x)$ come segue:

$$f(x) = \begin{cases} x \dot{-} \min_{t \leq x}(c | (x \dot{-} t)), & \exists t \leq x : c | (x \cdot t) \\ 0, \text{altrimenti} \end{cases} \tag{9.46}$$

Essendo che 0 è multiplo di ogni numero, in caso non venga trovato alcun multiplo di c è corretto che la minimalizzazione restituisca 0.

Esercizio 6

Sia $f(x)$ una funzione che soddisfa la condizione:

$$f(x) \downarrow \iff l(x) \notin W_{r(x)} \tag{9.47}$$

f è parzialmente calcolabile? Dimostrare o confutare.

Soluzione: Sia $f(x)$ parizlamente calcoalbile, allora $\exists p \in \mathbb{N} : \forall x \in \mathbb{N} : \Psi_{\mathcal{P}_p}^{(1)}(x) = f(x)$.

Sapendo che anche la funzione $\Phi(l(x), r(x))$ è parzialmente calcolabile sia q il numero del programma che la calcola.

Vediamo ora che il seguente programma calcola $HALT(x, q)$:

```
[B] IF STP(X,p,Z) GOTO E
    IF STP(X,q,Z) GOTO A
    Z ← Z + 1
    GOTO B
[A] Y ← Y + 1
```

Il programma termina necessariamente poichè, $\forall x \in \mathbb{N}, l(x) \in W_{r(x)} \vee l(x) \notin W_{r(x)}$.

9.6 2018

9.6.1 2 Luglio 2018

Esercizio 4

Sia $g(x)$ la somma di tutti i numeri naturali $n \leq x$ tali che n è un numero primo o n è un numero pari. Mostrare che g è una **funzione ricorsiva primitiva**.

Soluzione: Possiamo definire $g(x)$ come segue:

$$g(x) = \sum_{n=0}^{x} n(Primo(n) \vee 2|n) \tag{9.48}$$

Essendo g composizione di **funzioni ricorsive primitive** con **l'operatore di sommatoria**, g è **ricorsiva primitiva**.

Esercizio 5

Sia $B = \{(x_1, \cdots, x_m) \in \mathbb{N}^m | g(x_1, \cdots, x_m) \downarrow\}$, dove g è una funzione **parzialmente calcolabile**.

Mostrare che $B' = \{[x_1, \cdots, x_m] \in \mathbb{N} | (x_1, \cdots, x_m) \in B\}$ è ricorsivamente enumerabile.

Soluzione: B' è ricorsivamente enumerabile $\iff \exists g'$ parzialmente calcolabile : $g'(x) \downarrow \iff x \in B'$.

Consideriamo allora il seguente programma $\mathcal{P}$:

```
[A] IF (X = 0 ∨ Lt(x) > m) GOTO A
    Z1 ← (X)_1
    ⋮
    Zm ← (X)_m
    Y ← g(Z1, \(\cdots\), Zm)
```

Questo programma termina $\iff x \in B'$. Dunque con $g' = \Psi_{\mathcal{P}}^{(1)}$ abbiamo la tesi.

Esercizio 6

Mostrare che non esiste nessuna funzione calcolabile $f(x)$ tale che $f(x) = 0 \iff x \in W_x$.

Soluzione: $x \in W_x \iff \Phi(x, x) \downarrow \iff HALT(x, x)$. Supponiamo per assurdo che esista una $f(x)$ calcolabile. Allora potremmo usarla come macro in un programma.

Ma il seguente programma calcolerebbe $HALT(x, x)$, che porta ad assurdo:

```
IF f(X)≠0 GOTO E
Y ← Y + 1
```